УТВЕРЖДАЮ
Руководитель по предмету «АСОИиУ»
_____ Д. С. Шипицин

«___» _____ 2026 г.
Печать

УТВЕРЖДАЮ
студент гр. ИСИб-23-1 ИРНИТУ
_____ Л. А. Демонов

«___» _____ 2026 г.
Печать

ИРНИТУ, институт ИТиАД

наименование организации - разработчика ТП на АС

Разработка мультиагентной системы ситуативного обучения с динамическим синтезом и верификацией адаптивных образовательных сценариев

наименование вида АС

Автономная цифровая среда интеллектуального тренинга

наименование объекта автоматизации

МАССО

сокращенное наименование АС

ТЕХНИЧЕСКИЙ ПРОЕКТ

Редакция 3 (по результатам реализации первой очереди)

На 81 листах

Действует с «___» _____ 2026 г.

СОГЛАСОВАНО
Руководитель

_____ Д. С. Шипицин

«___» _____ 2026 г.
Печать

Содержание

1. Постановка проектных решений 3
2. Модель сущность-связь 7
3. Модели процессов To-Be 10
4. Диаграммы последовательности функциональностей 17
5. Модели данных 29
6. Архитектура системы в нотации C4 39
7. Модели информационных потоков данных 47
8. Шаблоны справочников и НСИ 51
9. Проектирование API 54
10. Модель структуры интерфейса 58
11. Макеты интерфейсов системы 60
12. Контрольные проектные сценарии 79
13. Вывод 80

1. Постановка проектных решений

Технический проект разработан для автоматизированной системы «МАССО» - мультиагентной системы ситуативного обучения с динамическим синтезом и верификацией адаптивных образовательных сценариев. Проектные решения синхронизированы с техническим заданием (редакция 3) и с реализованным опытным образцом: сохраняются роли пользователей, трехагентная схема, гибридный LLM-контур, изолированные учебные среды Docker, автоматическая верификация и отчетность.

Объект проектирования - цифровая среда практического обучения, в которой обучающийся получает индивидуальный сценарий, работает в изолированной среде, документном тренажере или редакторе задачи на код, получает теорию по навыкам сценария, а система фиксирует цифровой след, выдает ограниченные подсказки, проверяет результат и обновляет граф компетенций.

Настоящая редакция описывает первую очередь создания системы (пункт 1.9 ТЗ) в том виде, в котором она реализована. Все компоненты реализуются в одном приложении Backend/API (модульный монолит), все данные - включая граф компетенций, векторные представления и состояния фоновых задач - хранятся в PostgreSQL 18 с расширением pgvector, фоновые задачи выполняет worker - асинхронная задача того же процесса. Решения второй очереди (выделение агентов, Sandbox Manager и Verification Engine в отдельные сервисы, сетевой allowlist учебных сред) указаны в тексте как направления развития и не меняют публичного API и модели данных. Хранилище в обеих очередях одно - PostgreSQL с pgvector, учебные среды работают на Docker Engine.

Назначение проектных артефактов.

Артефакты технического проекта используются для перехода от требований к реализации: модель сущность-связь задает границы предметной области, To-Be модели уточняют работу модулей, диаграммы последовательности закрепляют взаимодействия, модели данных определяют

структуру хранилища, C4 фиксирует архитектуру, DFD описывает информационные потоки, а макеты показывают интерфейс системы.

В таблице 1 приведена трассировка между требованиями ТЗ и решениями настоящего ТП.

Таблица 1 - Трассировка ТЗ и технического проекта

Требование ТЗ	**Проектное решение**	**Где раскрыто в ТП**	**Артефакты**
Очередность создания (1.9)	Модульный монолит, одно хранилище PostgreSQL + pgvector, фоновый worker в процессе Backend/API; во второй очереди компоненты выделяются в сервисы без смены хранилища.	Разделы 1, 5, 6.	Рис. 22-24, Рис. 27, Рис. 28
Ролевая модель и RBAC	JWT, ротируемый refresh-токен в HttpOnly-cookie, блокировка перебора паролей, матрица прав и аудит изменения ролей.	Разделы 3, 4, 8, 9.	Рис. 2, Рис. 9, Рис. 48
Граф компетенций	Таблицы domains, skills, skill_requires, user_skills, track_targets; обход prerequisite-связей в ProfileAgent; карта компетенций обучающегося.	Разделы 2, 3, 5, 11.	Рис. 1, Рис. 3, Рис. 23, Рис. 42, Рис. 45, Рис. 47
Генерация сценариев	ScenarioAgent формирует легенду и параметры, выбирает инциденты из пула шаблона, проверяет уникальность через pgvector и достижимость прогоном.	Разделы 3, 4, 5, 8.	Рис. 4, Рис. 11, Рис. 16, Рис. 46
Учебные сессии и sandbox	Sandbox Manager запускает контейнер «учебный сервер» без сети и привилегий, с лимитами ресурсов; пауза, лимит времени и автоотправка.	Разделы 3, 4, 5, 6.	Рис. 5, Рис. 12, Рис. 25, Рис. 31, Рис. 38

Требование ТЗ	Проектное решение	Где раскрыто в ТП	Артефакты
Документный тренажер	Пакет документов и дефекты формируются программно по параметрам шаблона, ответ сохраняется автоматически, проверка по ключу варианта.	Разделы 3, 4, 8, 11.	Рис. 4, Рис. 6, Рис. 39
Задачи на программирование	Редактор кода, запуск кода в одноразовом контейнере песочницы, скрытые тесты по группам, проверка собственных тестов на эталоне и типичных ошибках.	Разделы 4, 6, 8, 9, 11.	Рис. 19, Рис. 29, Рис. 40
Подсказки, инциденты, теория	Worker вносит инцидент по сроку, AssessmentAgent выдает не более 3 подсказок со штрафом 10 %, самопроверка без штрафа, теория по навыкам сценария.	Разделы 3, 4, 8, 11.	Рис. 6, Рис. 13, Рис. 38
Автоматическая проверка	Verification Engine выполняет checks, AssessmentAgent рассчитывает балл и обновляет профиль.	Разделы 3, 4, 5, 8.	Рис. 6, Рис. 14, Рис. 41
Отчеты и экспертная оценка	Отчеты с обязательным периодом, экспорт CSV/JSON, печатная форма отчета, корректировка с причиной.	Разделы 3, 4, 9, 11.	Рис. 7, Рис. 15, Рис. 18, Рис. 43, Рис. 44
Гибридный LLM-контур	LLM Gateway с OpenAI-совместимым API (GigaChat - через адаптер gpt2giga) переключает внешний, локальный и шаблонный режимы без остановки сессий.	Разделы 3, 4, 6, 11.	Рис. 8, Рис. 17, Рис. 49

Требование ТЗ	Проектное решение	Где раскрыто в ТП	Артефакты
Мониторинг и безопасность	Метрики /metrics, журналы аудита и security_events, HTTPS/WSS в эксплуатационном режиме, ручная очистка сред.	Разделы 3, 6, 11.	Рис. 8, Рис. 31, Рис. 50

2. Модель сущность-связь

Модель сущность-связь определяет основные объекты МАССО: пользователей, роли, предметные области, компетенции, сценарии, назначения, сессии, события, подсказки, результаты проверки, отчеты и эксплуатационные настройки. Центральной операционной сущностью является учебная сессия: она связывает обучающегося, индивидуальный запуск сценария, цифровой след, проверки и итоговый отчет.

ER-модель представлена на Рис. 1. Справочные данные отделены от операционных: роли, предметные области, профили песочниц и настройки LLM ведутся отдельно, а события сессии и результаты проверки имеют обязательные корреляционные идентификаторы.

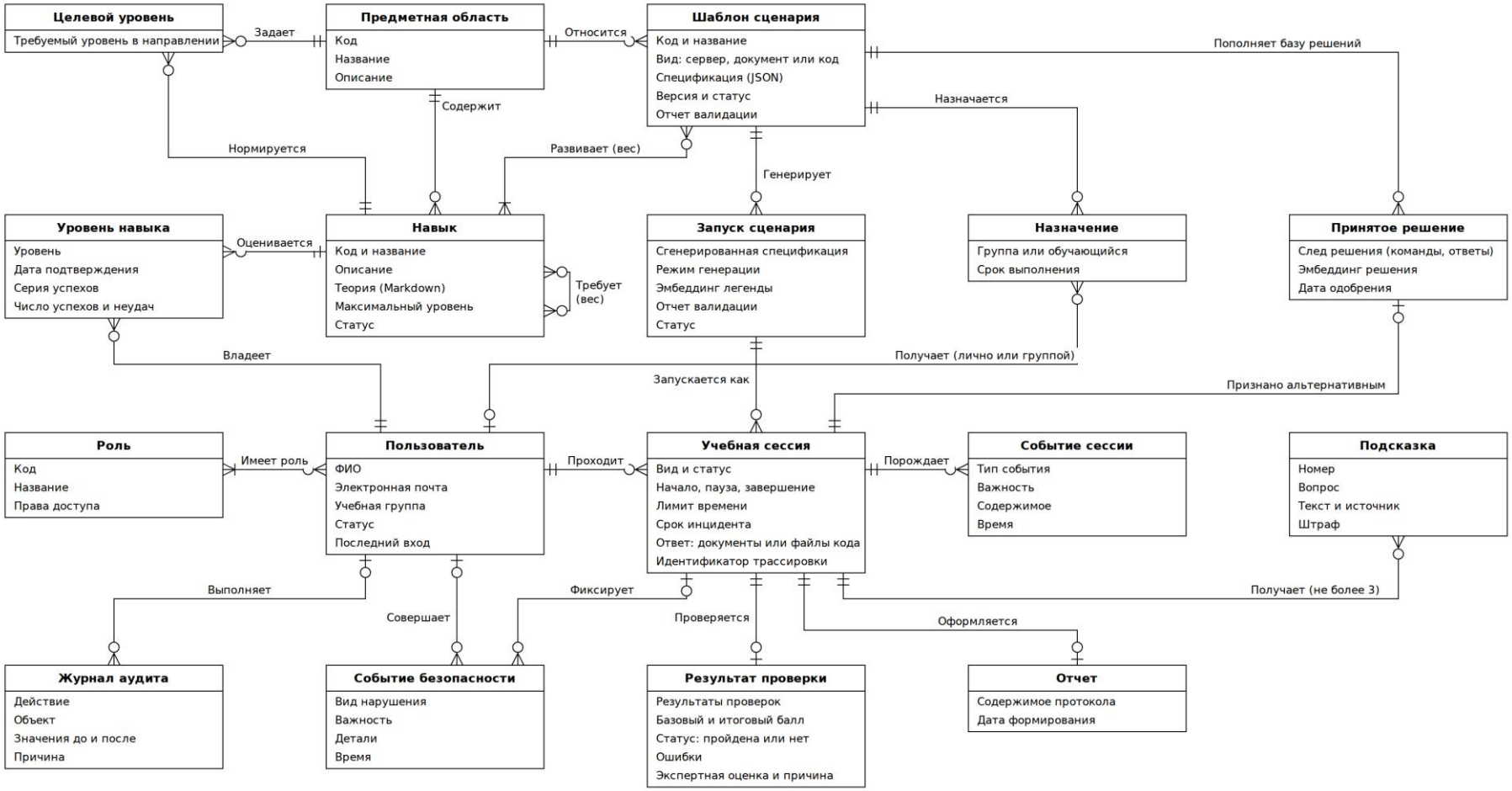


Рис. 1. Модель сущность-связь MACCO

Связи модели:

- один пользователь может иметь несколько ролей через связующую сущность UserRole;
- предметная область (Domain) объединяет навыки и шаблоны сценариев и задает направление обучения с целевыми уровнями навыков (TrackTarget, таблица track_targets);
- навыки связаны между собой отношением REQUIRES (таблица skill_requires) с весом от 0 до 1; навык содержит теорию для обучающегося;
- пользователь и навык связаны отношением HAS_SKILL (таблица user_skills) с уровнем и счетчиками подтверждений;
- шаблон сценария связан с навыками отношением TARGETS (таблица template_skills) и имеет несколько индивидуальных запусков ScenarioRun;
- назначение (Assignment) связывает шаблон сценария с группой или обучающимся и сроком выполнения;
- один запуск сценария порождает одну или несколько учебных сессий;
- одна учебная сессия содержит события, подсказки, результат проверки и итоговый отчет;
- подтвержденное альтернативное решение (AcceptedSolution) связано с шаблоном сценария и сессией, в которой оно получено;
- изменения оценок, ролей, LLM-режима, sandbox-профилей, справочника предметных областей и шаблонов фиксируются в журнале аудита.

3. Модели процессов To-Be

Процессы To-Be описывают целевую работу каждого ключевого модуля системы. В отличие от ТЗ, где сформулированы требования, здесь зафиксирован проектный порядок обработки данных, ответственности модулей и точки сохранения состояния.

Процесс аутентификации и разграничения прав приведен на Рис. 2. Пользователь проходит вход, модуль Auth/RBAC проверяет учетные данные и временную блокировку после серии неудачных попыток, выдает токены и набор ролей, затем клиентское приложение открывает только те разделы, которые разрешены назначенными ролями.

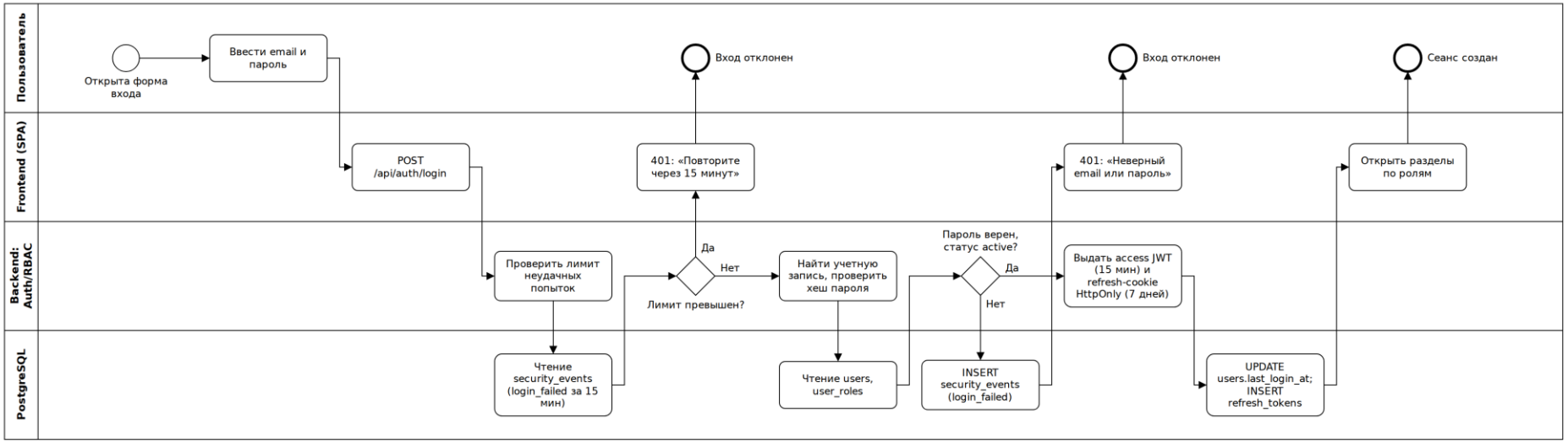


Рис. 2. To-Be процесс аутентификации и RBAC

Процесс формирования профиля компетенций приведен на Рис. 3. ProfileAgent читает уровни навыков из user_skills и целевые уровни направления из track_targets, обходом графа skill_requires (с учетом транзитивных связей) определяет

зависимые навыки, вычисляет и ранжирует дефициты и возвращает рекомендации по сценариям; тот же профиль отображается обучающемуся в карте компетенций.

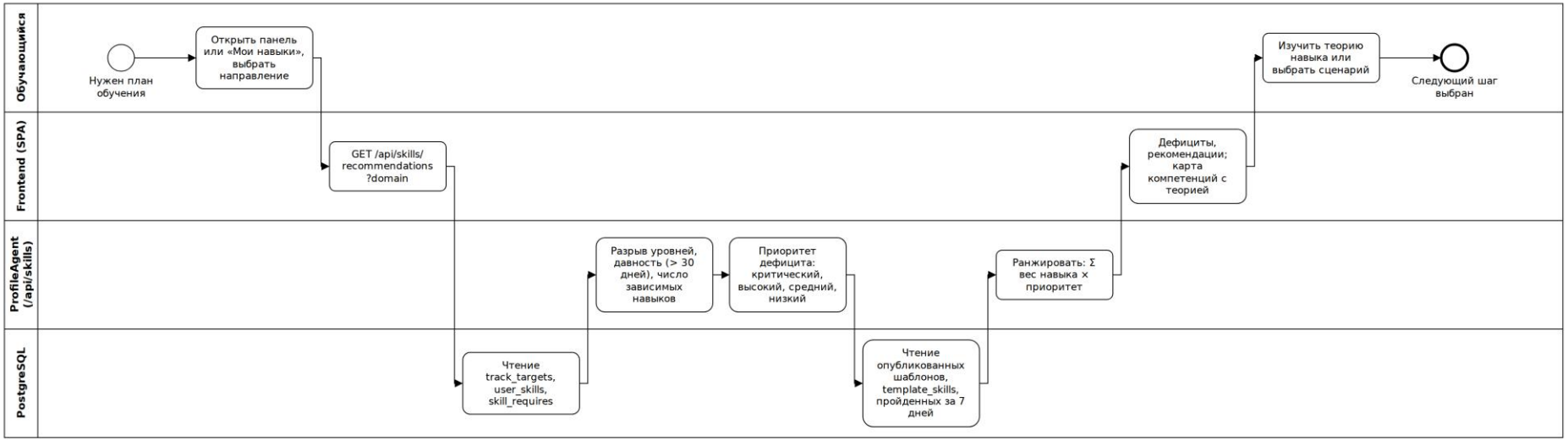


Рис. 3. To-Be процесс формирования профиля компетенций

Процесс генерации сценария приведен на Рис. 4. ScenarioAgent выбирает параметры варианта: для контейнерного сценария - начальный сбой и инциденты из пула шаблона, для документного - пакет документов с дефектами, для задачи на код - параметры условия и тестовые данные; получает легенду через LLM Gateway, исключает повторы по векторному сходству (pgvector) и до публикации подтверждает путь решения прогоном.

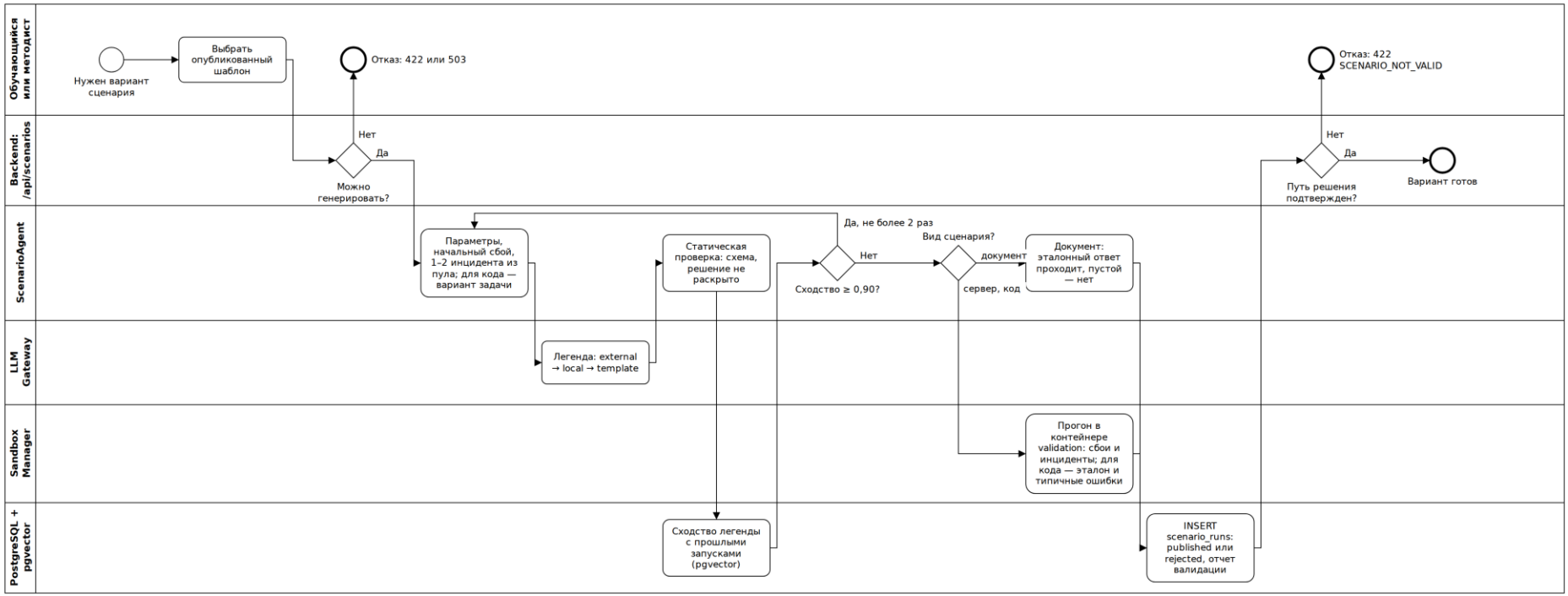


Рис. 4. To-Be процесс генерации и валидации сценария

Процесс запуска учебной среды приведен на Рис. 5. Модуль сессий создает запись сессии, Sandbox Manager запускает контейнер «учебный сервер» с ограничениями и применяет начальный сбой либо открывается документный тренажер или редактор задачи на код, а клиентское приложение получает статус по WebSocket и открывает рабочее место.

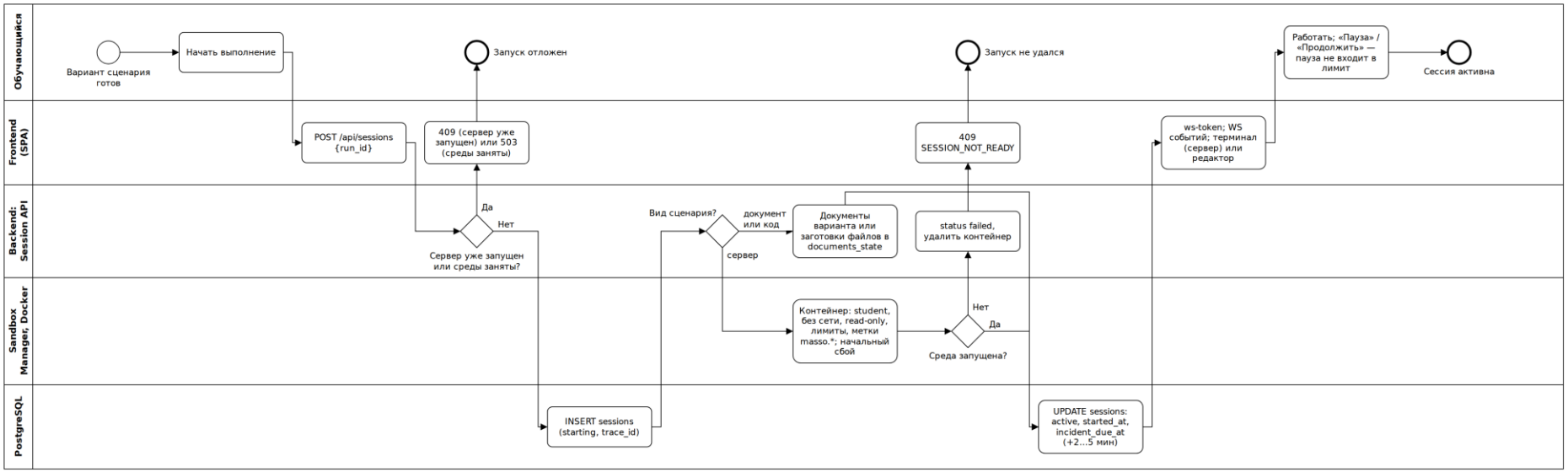


Рис. 5. To-Be процесс запуска учебной среды

Процесс сопровождения и проверки приведен на Рис. 6. Worker вносит инцидент по сроку, контролирует бездействие и лимит времени; обучающийся может запустить промежуточную самопроверку, AssessmentAgent выдает не более трех подсказок, по отправке запускает Verification Engine, рассчитывает балл и сохраняет результат и обновленный профиль в PostgreSQL.

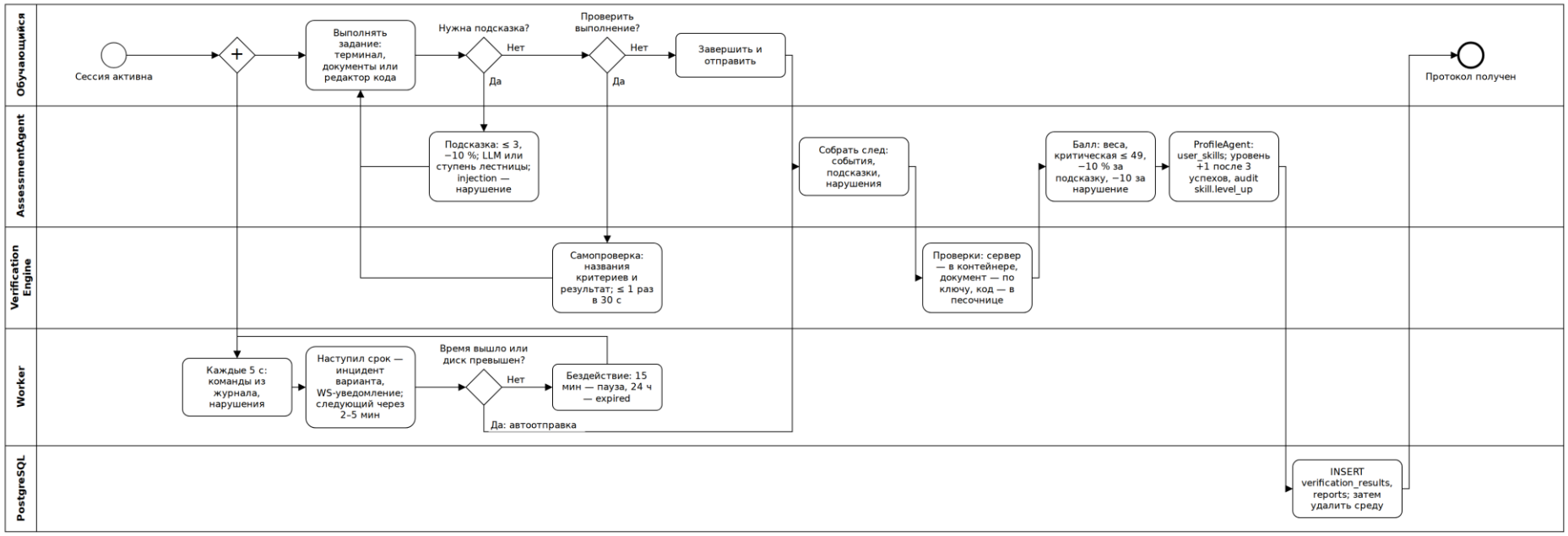


Рис. 6. To-Be процесс сопровождения и автоматической проверки

Процесс отчетности и экспертной корректировки приведен на Рис. 7. Преподаватель задает группу и период, анализирует цифровой след и при необходимости корректирует оценку с обязательной причиной.

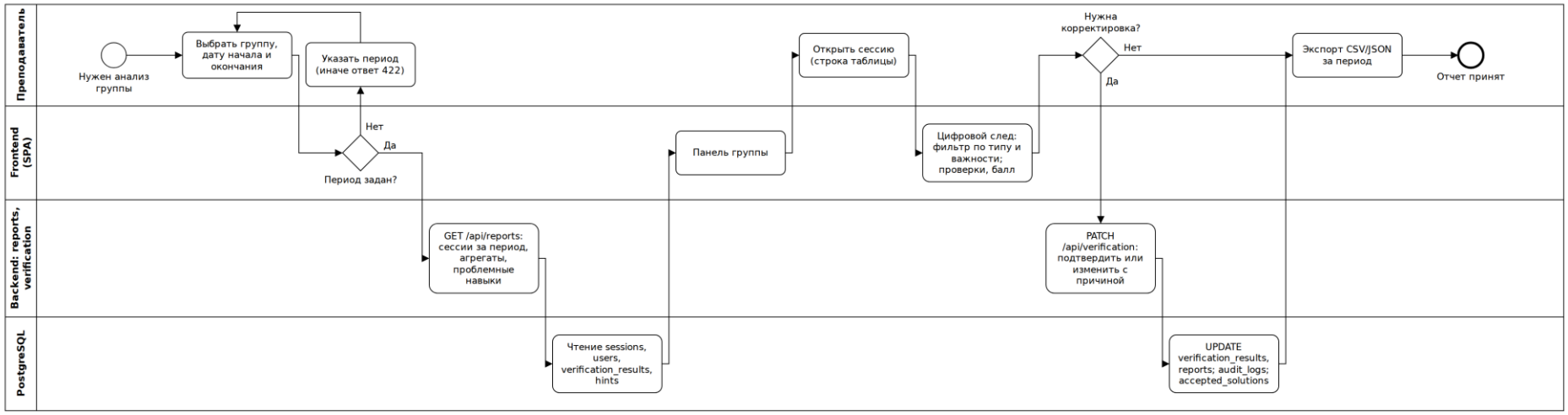


Рис. 7. To-Be процесс отчетности и экспертной корректировки

Процесс администрирования и мониторинга приведен на Рис. 8. Администратор управляет пользователями, LLM-провайдерами и профилями песочниц, системный администратор контролирует состояние компонентов, учебные среды с ручной очисткой и события безопасности.

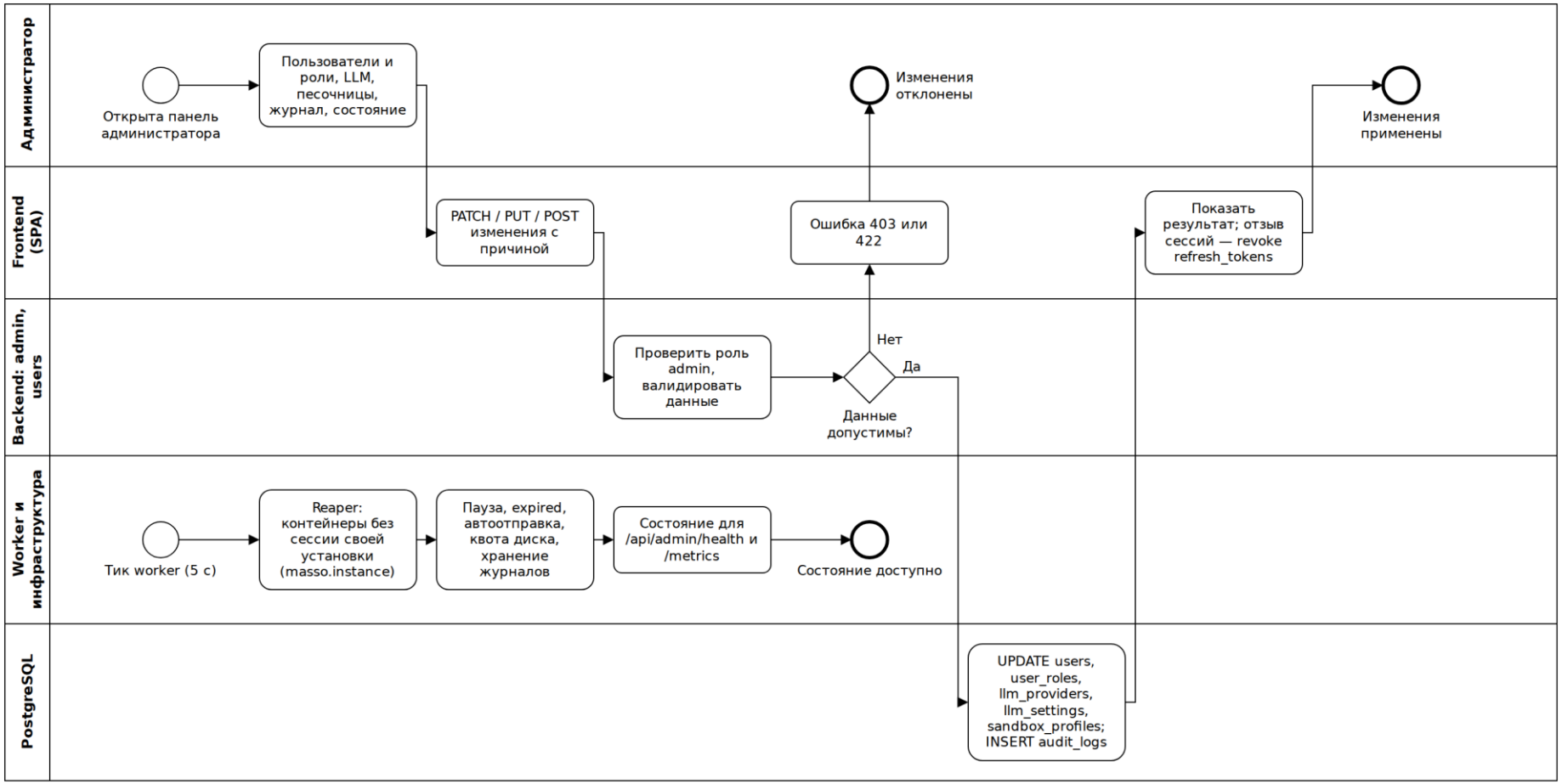


Рис. 8. To-Be процесс администрирования и мониторинга

4. Диаграммы последовательности функциональностей

Диаграммы последовательности уточняют вызовы между пользователем, клиентским приложением, модулями Backend/API, агентами, PostgreSQL и Docker Engine. Каждая диаграмма соответствует функциональности из ТЗ; номера сообщений и обозначения выполнены в нотации UML: синхронный вызов - закрашенная стрелка, асинхронный - открытая, ответ - пунктир, REST-вызовы подписаны методом и URL.

Последовательность входа пользователя показана на Рис. 9. После проверки учетных данных система сохраняет хеш refresh-токена и возвращает access-токен, refresh-cookie и набор ролей; неуспешная попытка фиксируется в security_events, серия неудачных попыток временно блокирует вход.

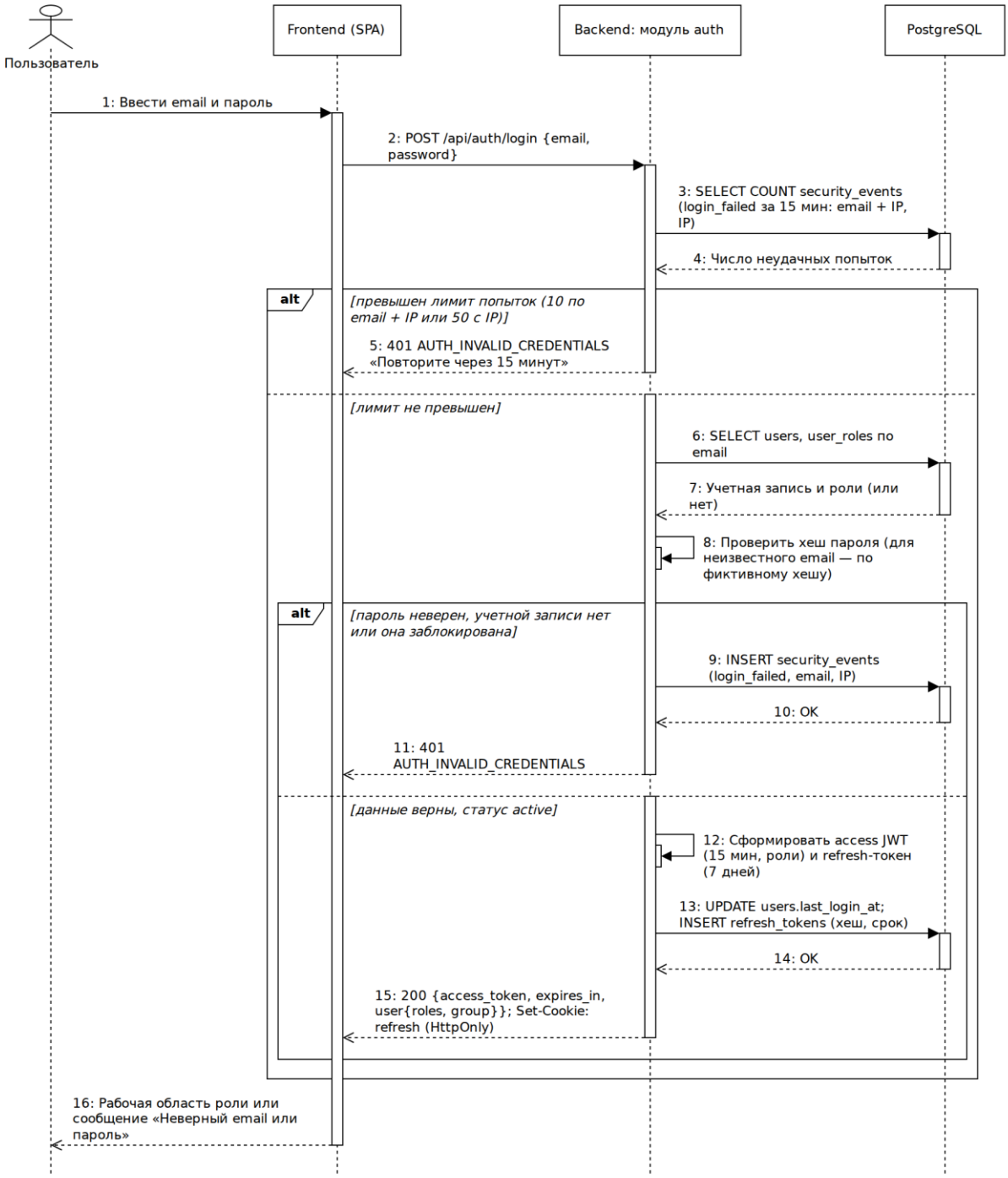


Рис. 9. Диаграмма последовательности входа пользователя

Подбор сценария по профилю компетенций показан на Рис. 10. Вызов начинается с запроса рекомендаций и завершается списком дефицитов навыков с приоритетами и подходящими шаблонами.

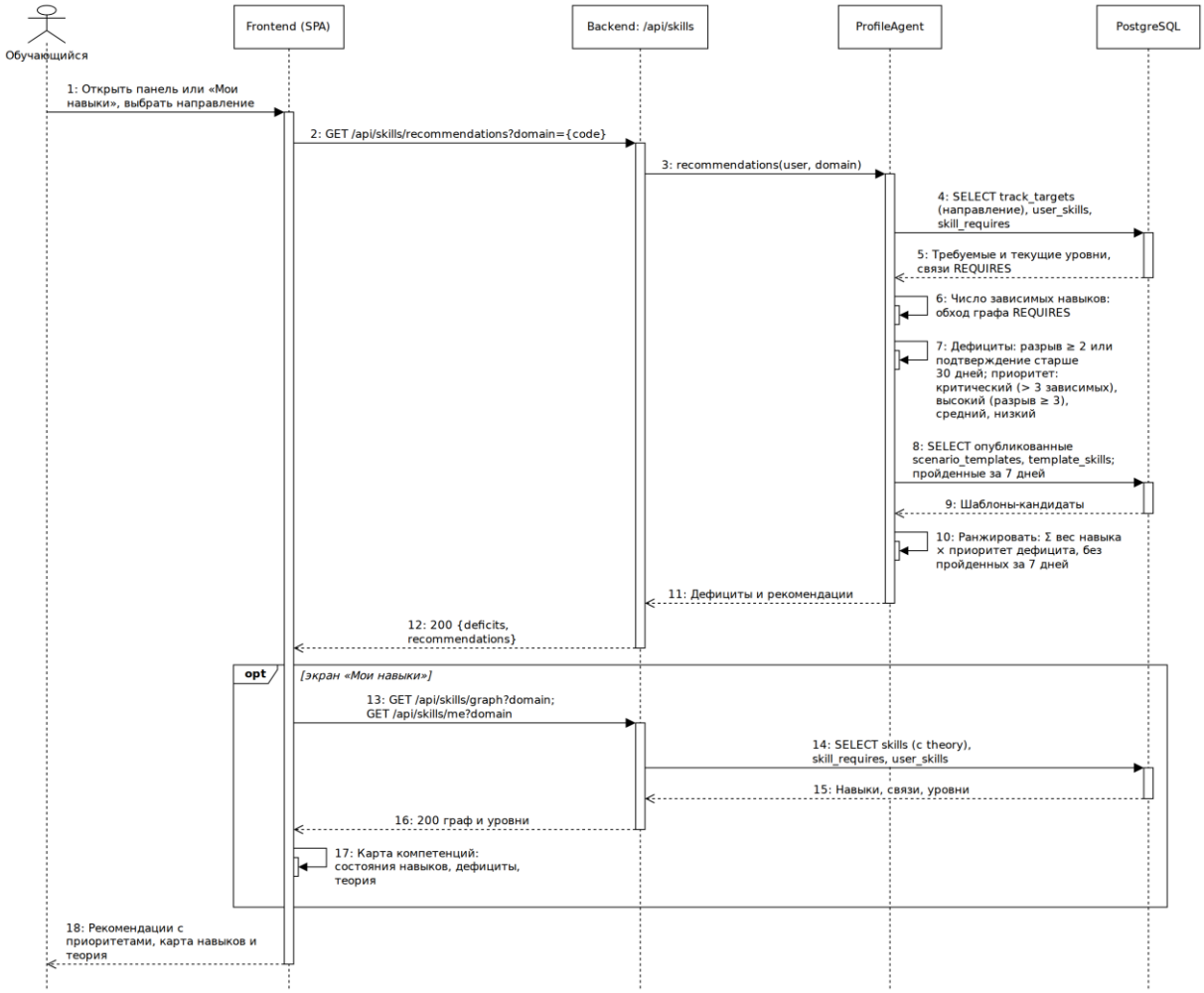


Рис. 10. Диаграмма последовательности подбора сценария по профилю

Генерация сценария показана на Рис. 11. ScenarioAgent обращается к LLM Gateway за легендой, проверяет сходство с ранее выданными легендами в pgvector, подтверждает путь решения прогоном в Sandbox Manager и сохраняет запуск с отчетом валидации.

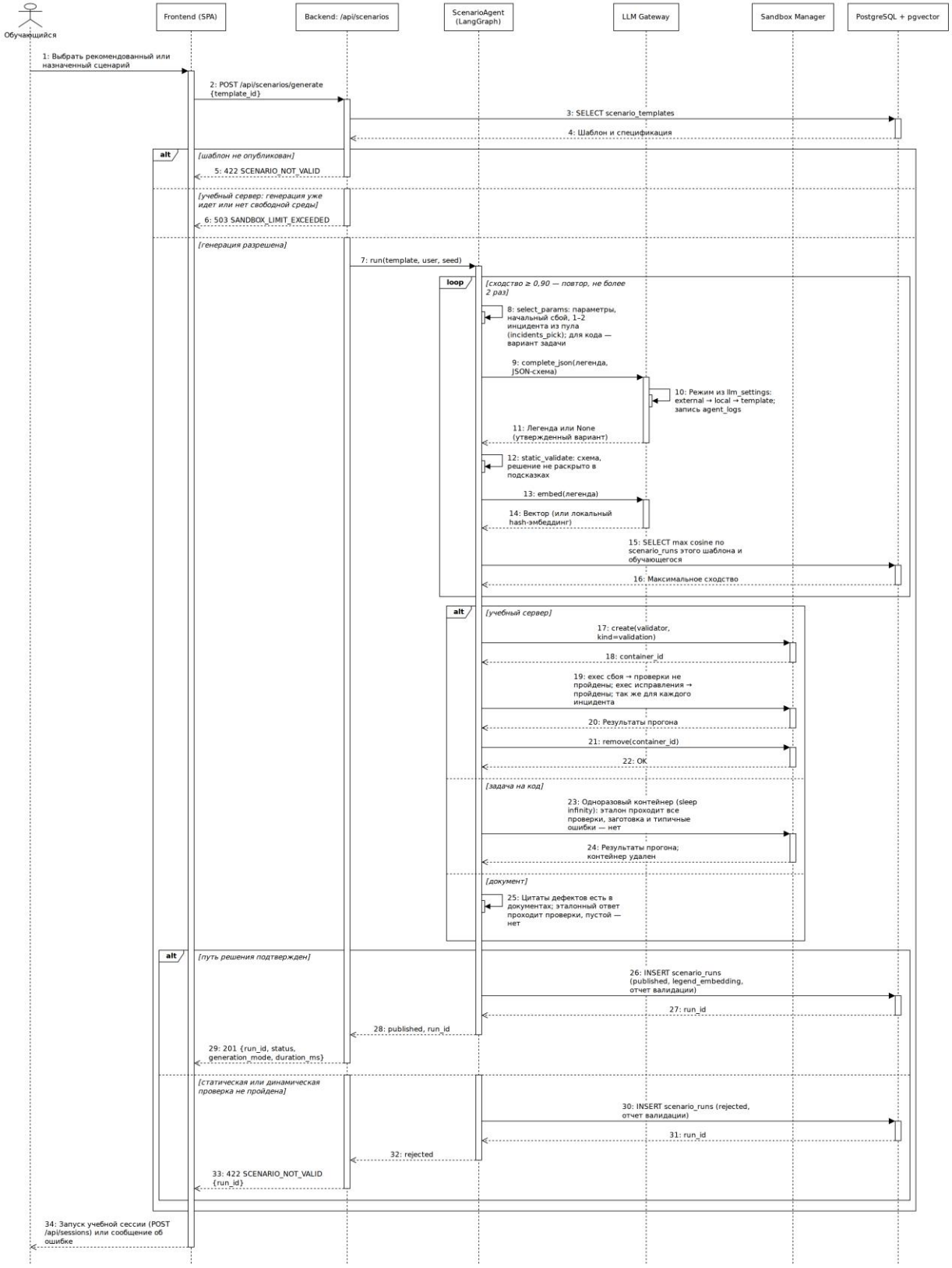


Рис. 11. Диаграмма последовательности генерации сценария

Запуск учебной сессии показан на Рис. 12. Backend создает session_id, Sandbox Manager создает и запускает контейнер через Docker API, применяет начальный сбой, после чего клиентское приложение открывает каналы

терминала и событий по краткоживущему токену. При паузе и возобновлении сроки started_at и incident_due_at сдвигаются на длительность паузы.

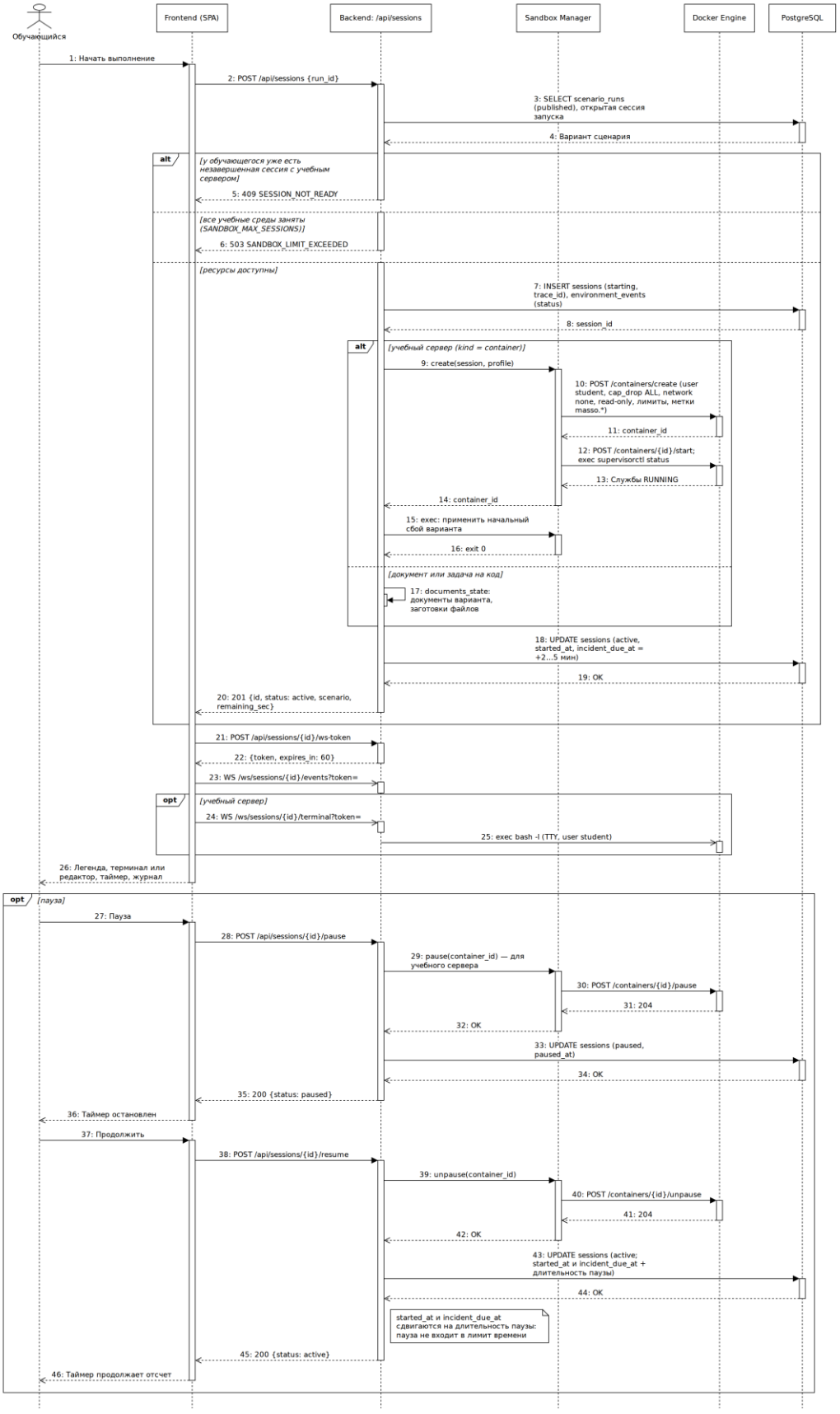


Рис. 12. Диаграмма последовательности запуска учебной сессии

Внесение инцидента и выдача подсказки показаны на Рис. 13. Worker по сроку incident_due_at применяет сбой и публикует событие; подсказка формируется по проваленным проверкам, проходит фильтр и сохраняется с номером и штрафом. Запрос с попыткой prompt injection фиксируется в security_events и в LLM Gateway не передается: выдается очередная ступень лестницы подсказок.

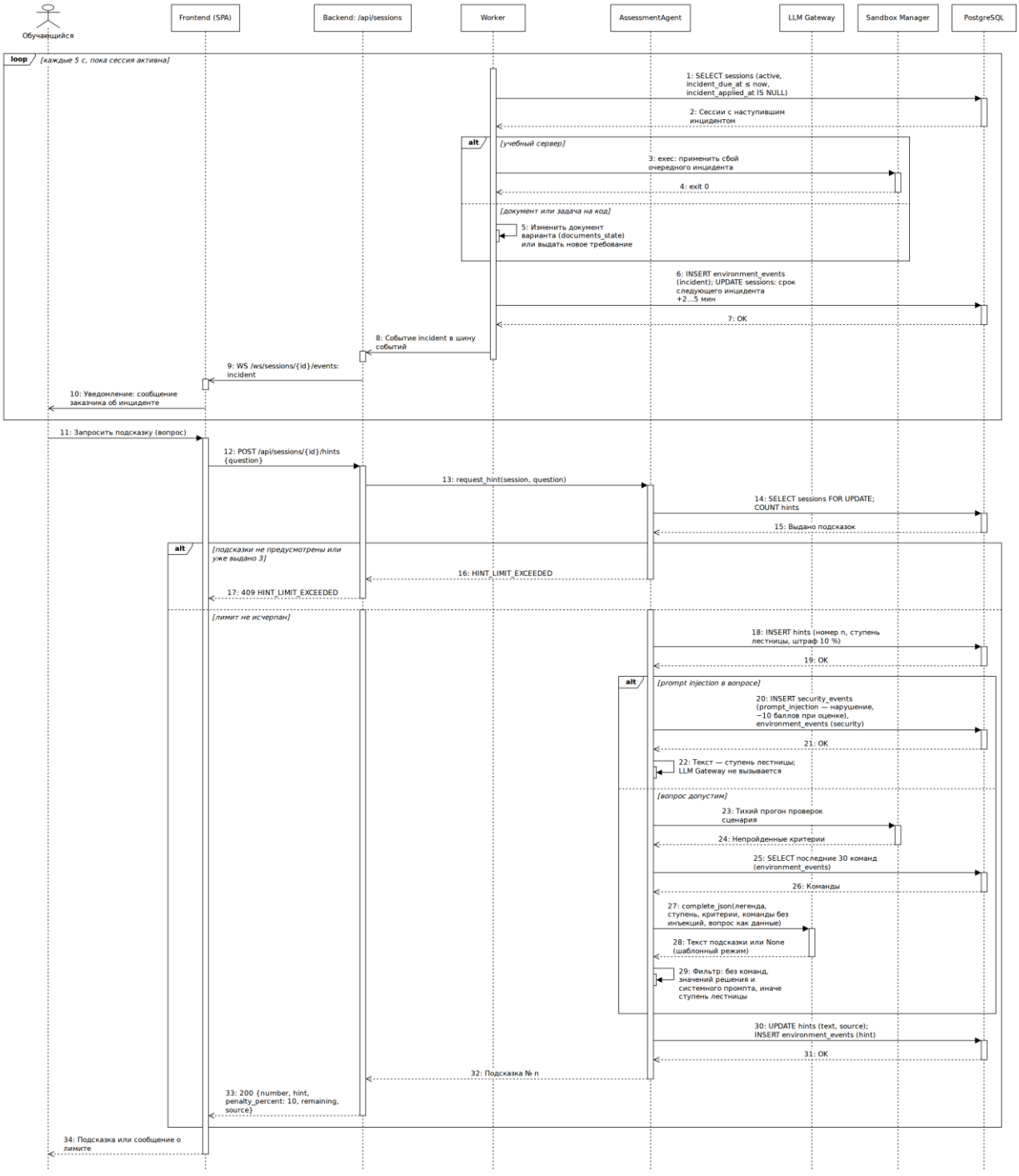


Рис. 13. Диаграмма последовательности обработки инцидента и подсказки

Проверка результата и формирование отчета показаны на Рис. 14. Отправку инициирует обучающийся или worker по истечении лимита времени; Verification Engine выполняет checks, AssessmentAgent рассчитывает балл, ProfileAgent обновляет user_skills (повышение уровня записывается в audit_logs), формируется отчет, после чего среда удаляется.

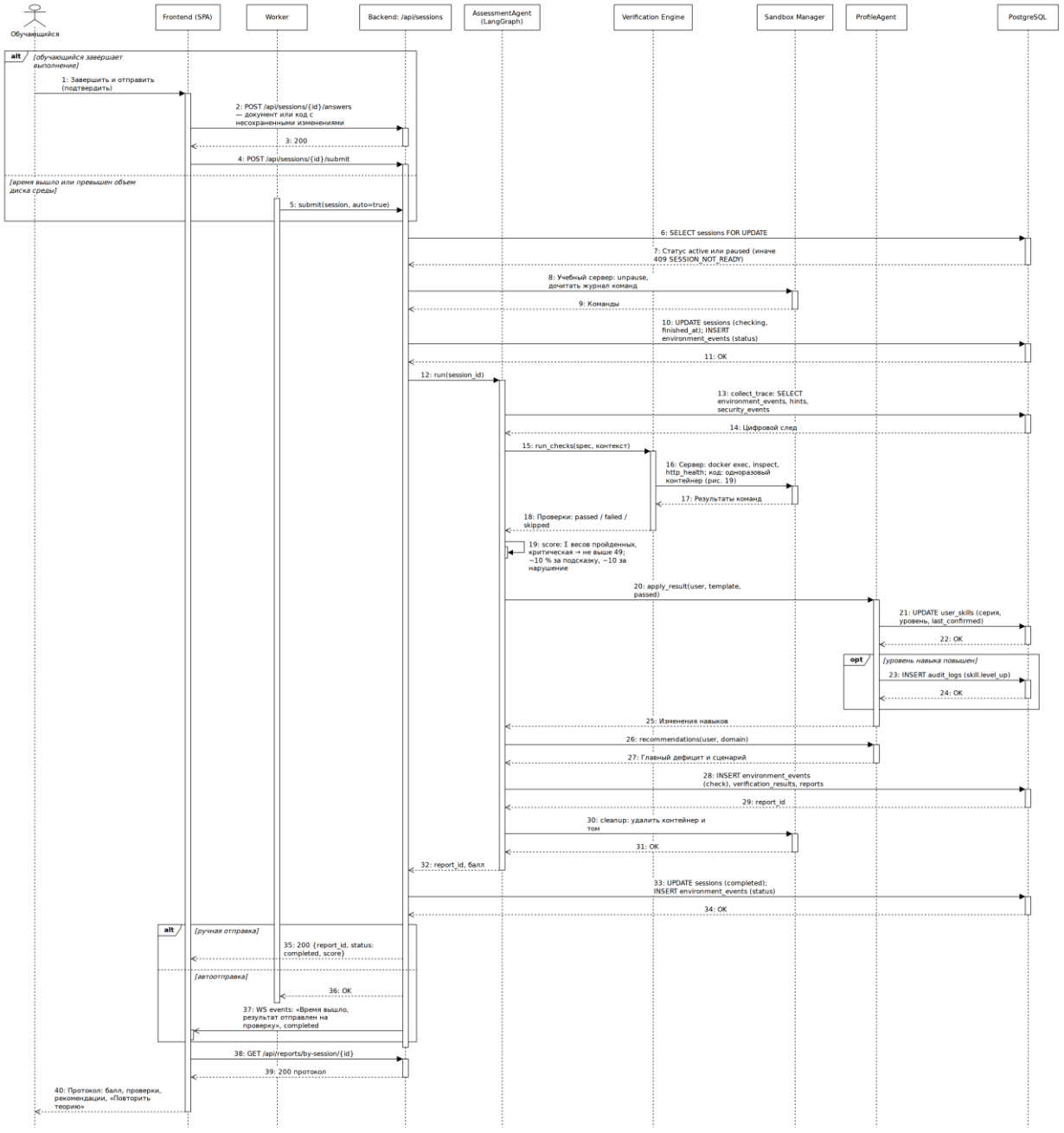


Рис. 14. Диаграмма последовательности проверки результата и отчета

Экспертная проверка преподавателем показана на Рис. 15. При корректировке оценки причина изменения обязательна и попадает в audit_logs.

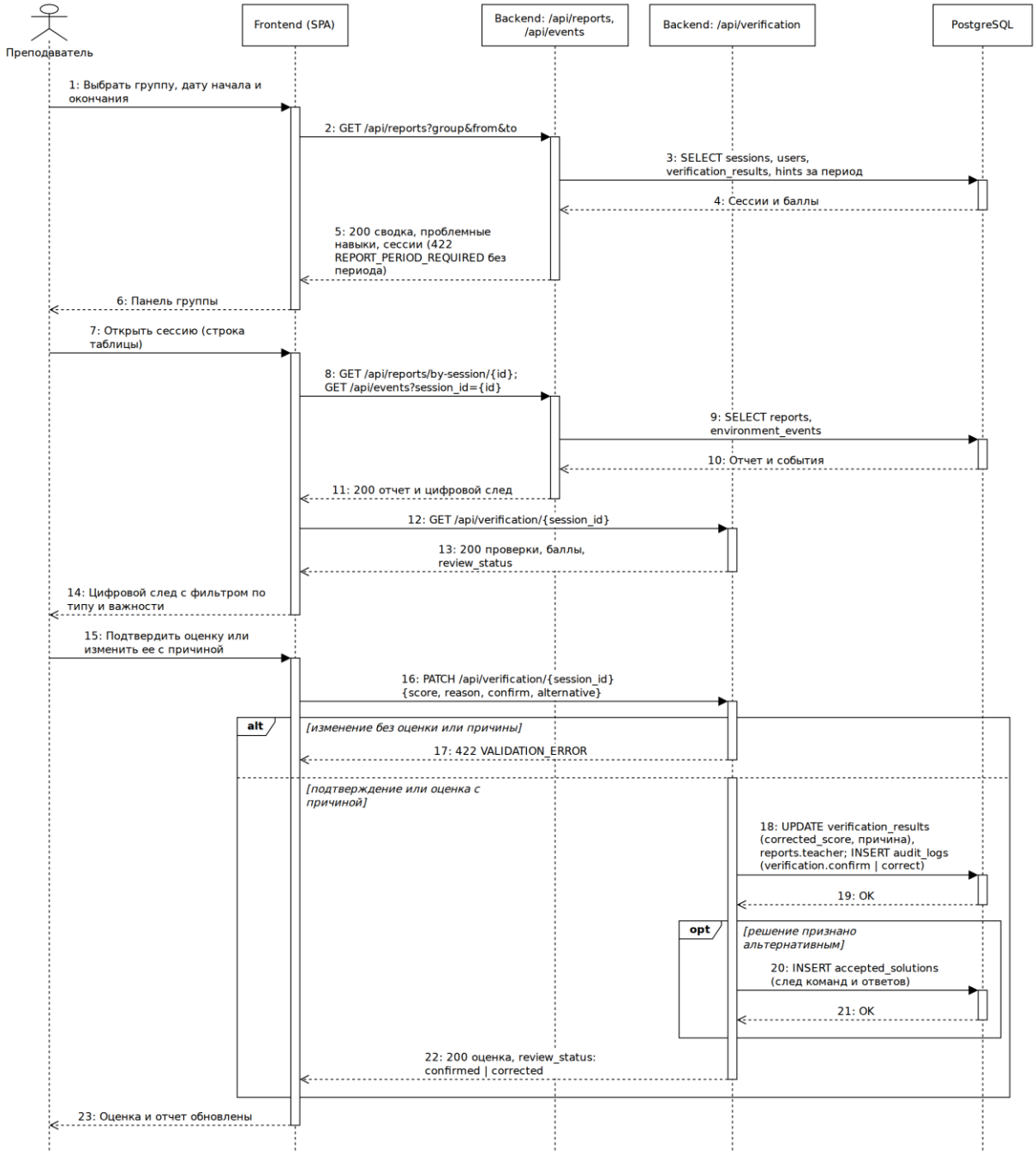


Рис. 15. Диаграмма последовательности экспертной проверки преподавателем

Создание шаблона сценария методистом показано на Рис. 16. Методист задает навык и prerequisite-связи, сбои и пул инцидентов, checks и правила подсказок; публикация возможна только после успешной валидации прогоном.

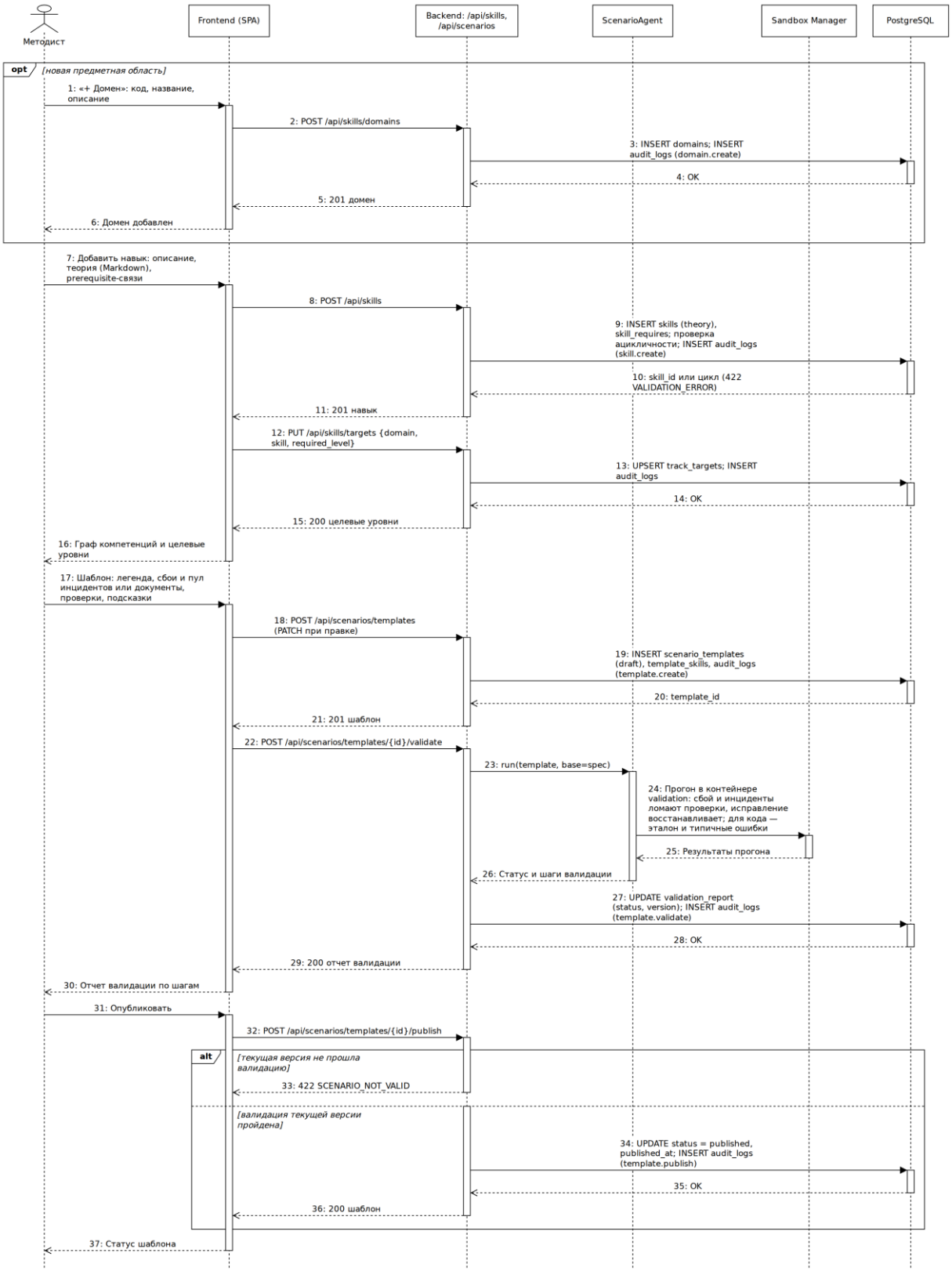


Рис. 16. Диаграмма последовательности создания шаблона сценария

Переключение LLM-контура показано на Рис. 17. Новые задачи переводятся на выбранный режим, активные учебные сессии не останавливаются, преподаватели получают предупреждение.

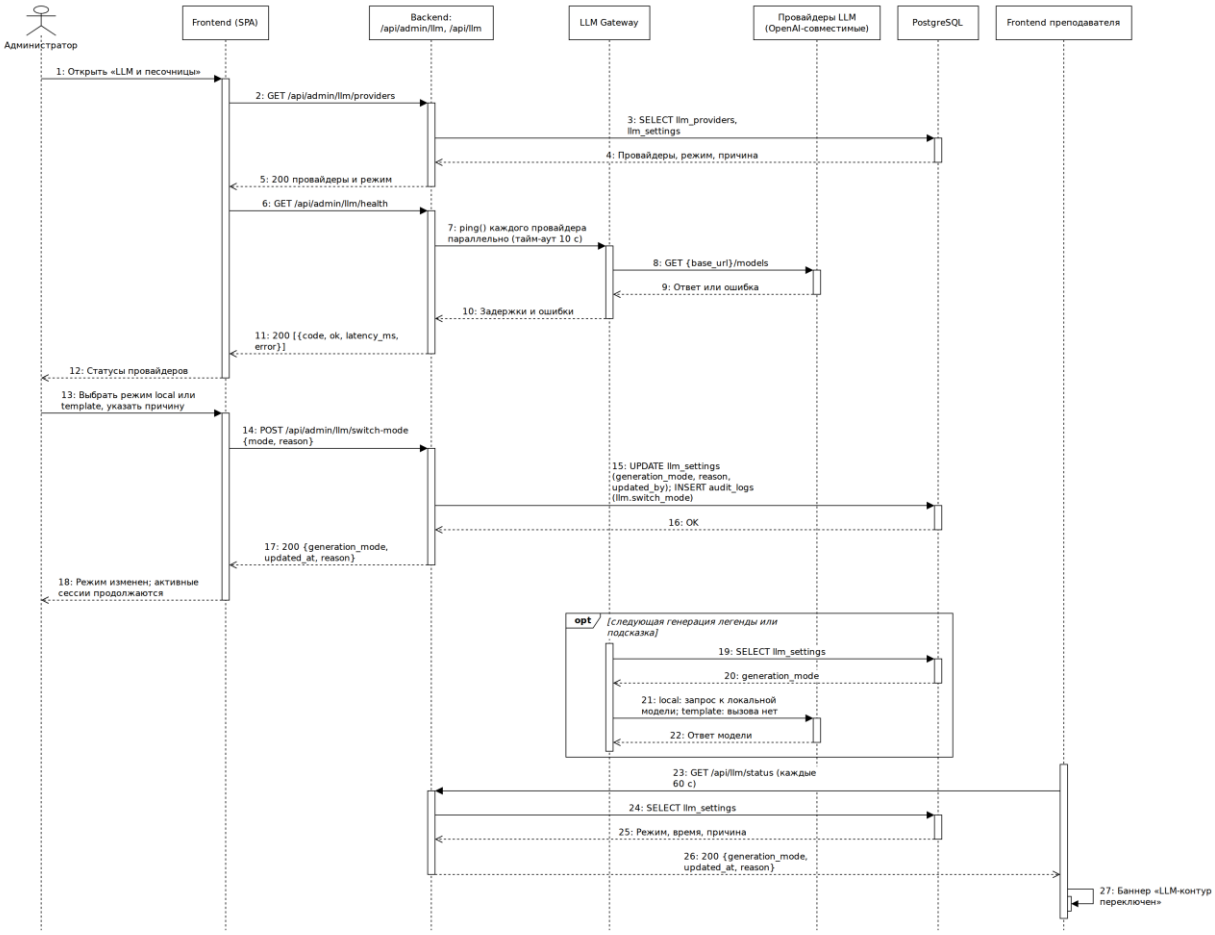


Рис. 17. Диаграмма последовательности переключения LLM-контура

Экспорт отчетности показан на Рис. 18. Обязательными входными параметрами являются начало и конец периода, а также формат выгрузки (CSV или JSON).

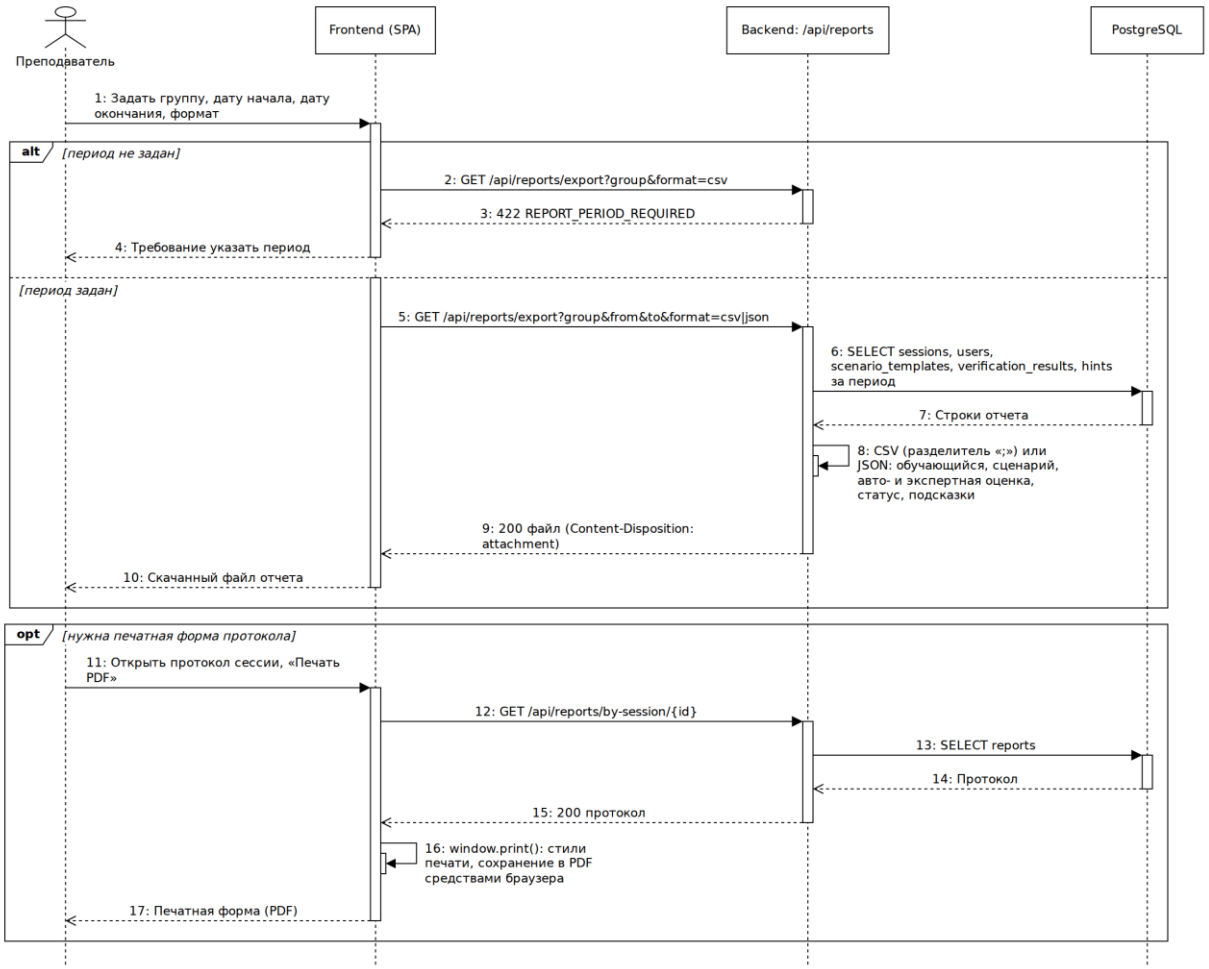


Рис. 18. Диаграмма последовательности экспорта отчетности

Выполнение задачи на программирование показано на Рис. 19. Файлы решения и тестов сохраняются автоматически; запуск примеров и собственных тестов выполняется в одноразовом контейнере песочницы без начисления баллов; самопроверка возвращает только названия критериев и результат; при отправке Verification Engine прогоняет решение на скрытых тестах (ожидаемые значения вычисляет эталонное решение на backend) и проверяет собственные тесты обучающегося на эталоне и типичных ошибочных решениях.

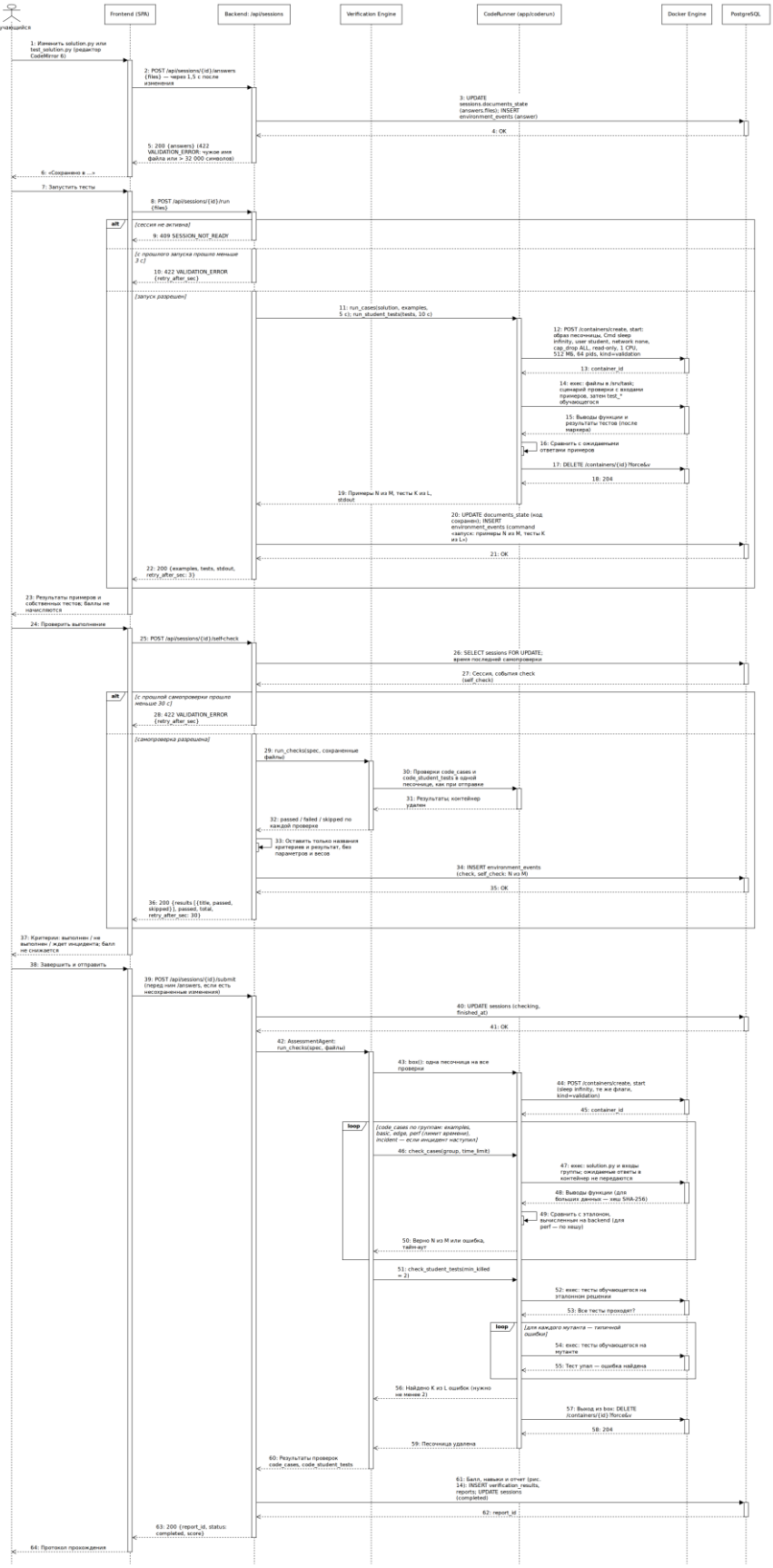


Рис. 19. Диаграмма последовательности выполнения задачи на код

5. Модели данных

Модели данных представлены на трех уровнях: концептуальном, логическом и физическом. В первой очереди физический уровень реализуется в одной СУБД PostgreSQL 18 с расширением pgvector, поэтому физическая модель разделена не по хранилищам, а по предметным группам таблиц.

Концептуальная модель на Рис. 20 показывает крупные классы данных: субъекты, обучение, оценивание и эксплуатацию.

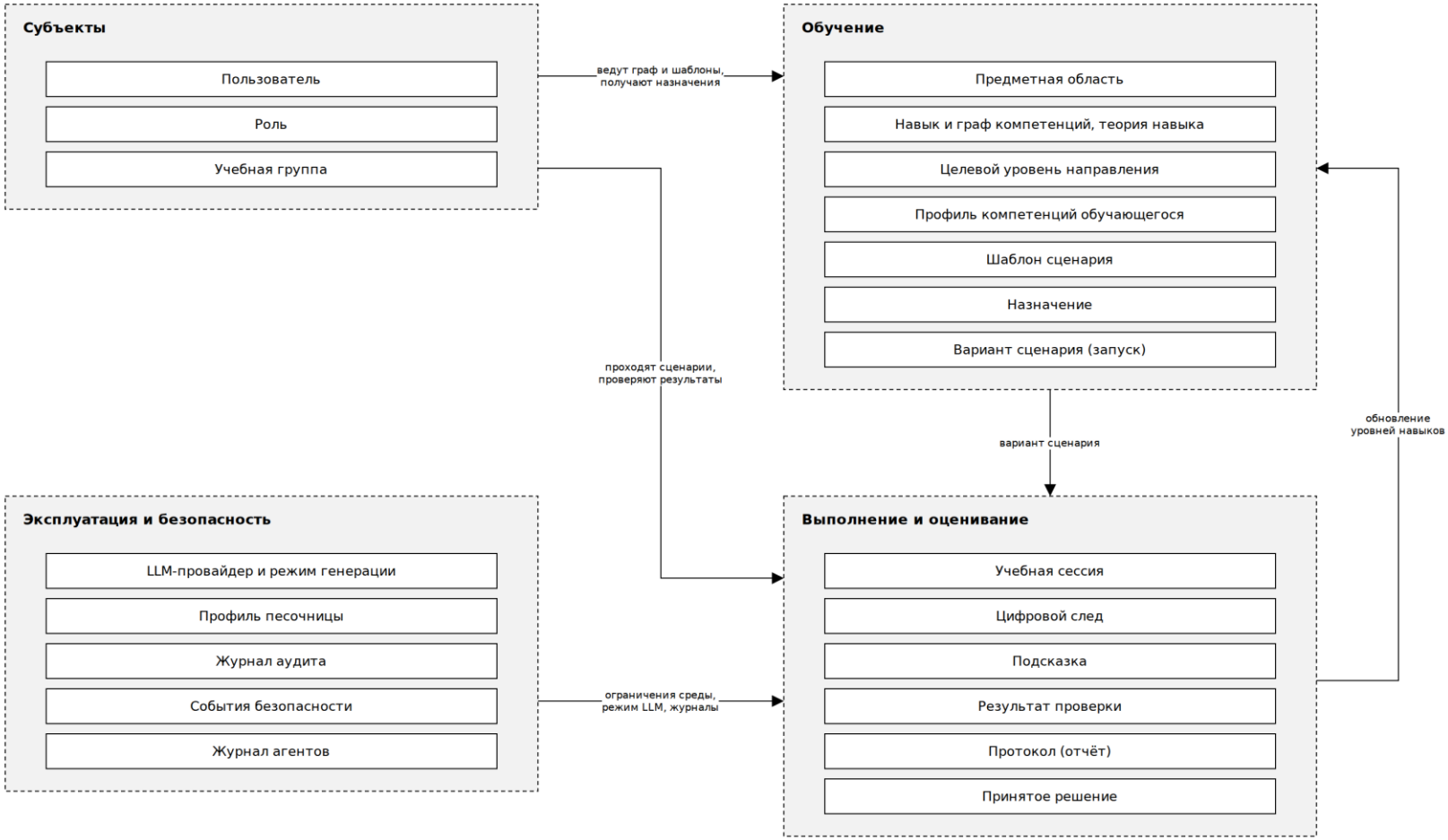


Рис. 20. Концептуальная модель данных

Логическая модель на Рис. 21 показывает состав сущностей, ключей и связей, которые затем уточняются в физической модели.

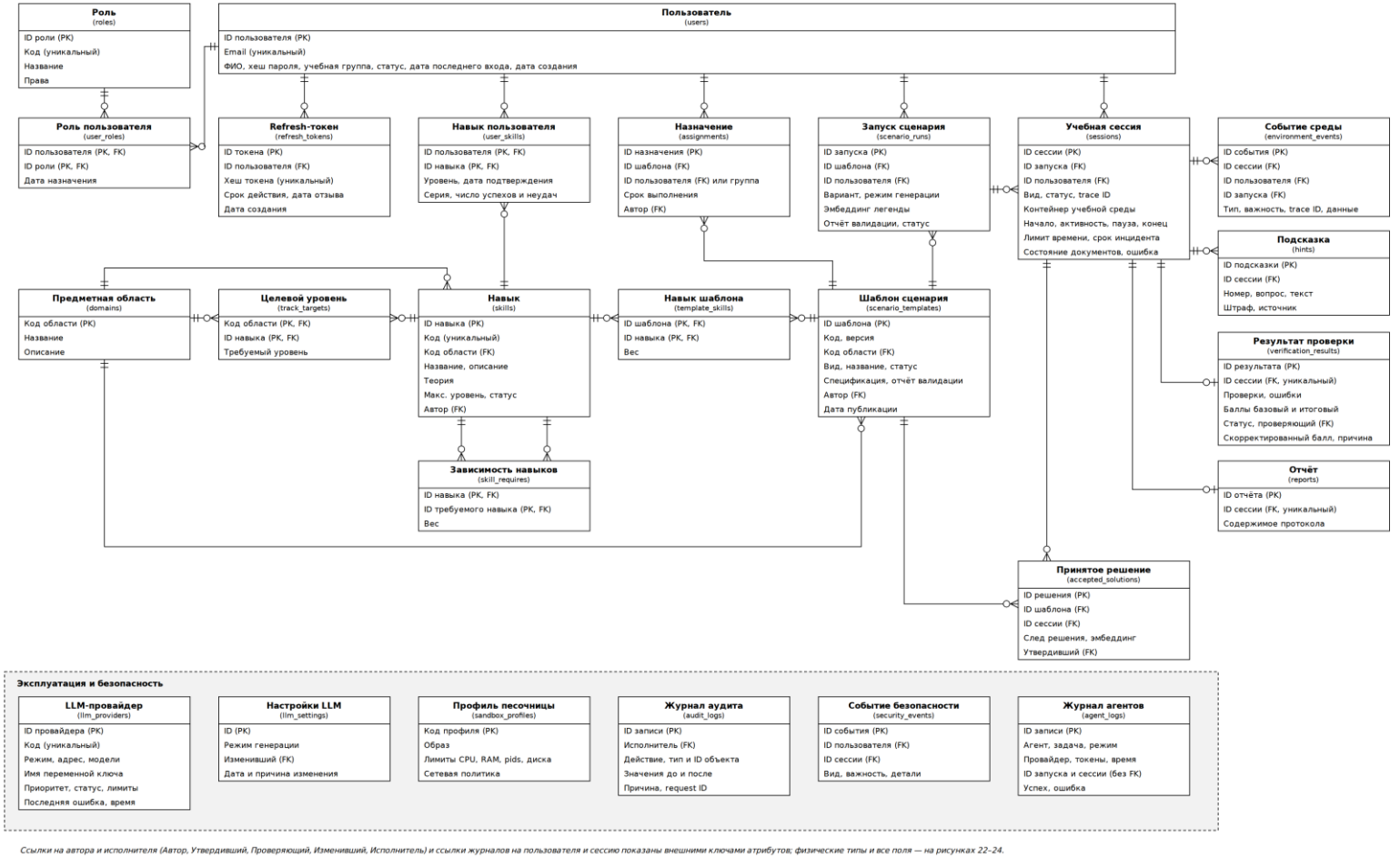
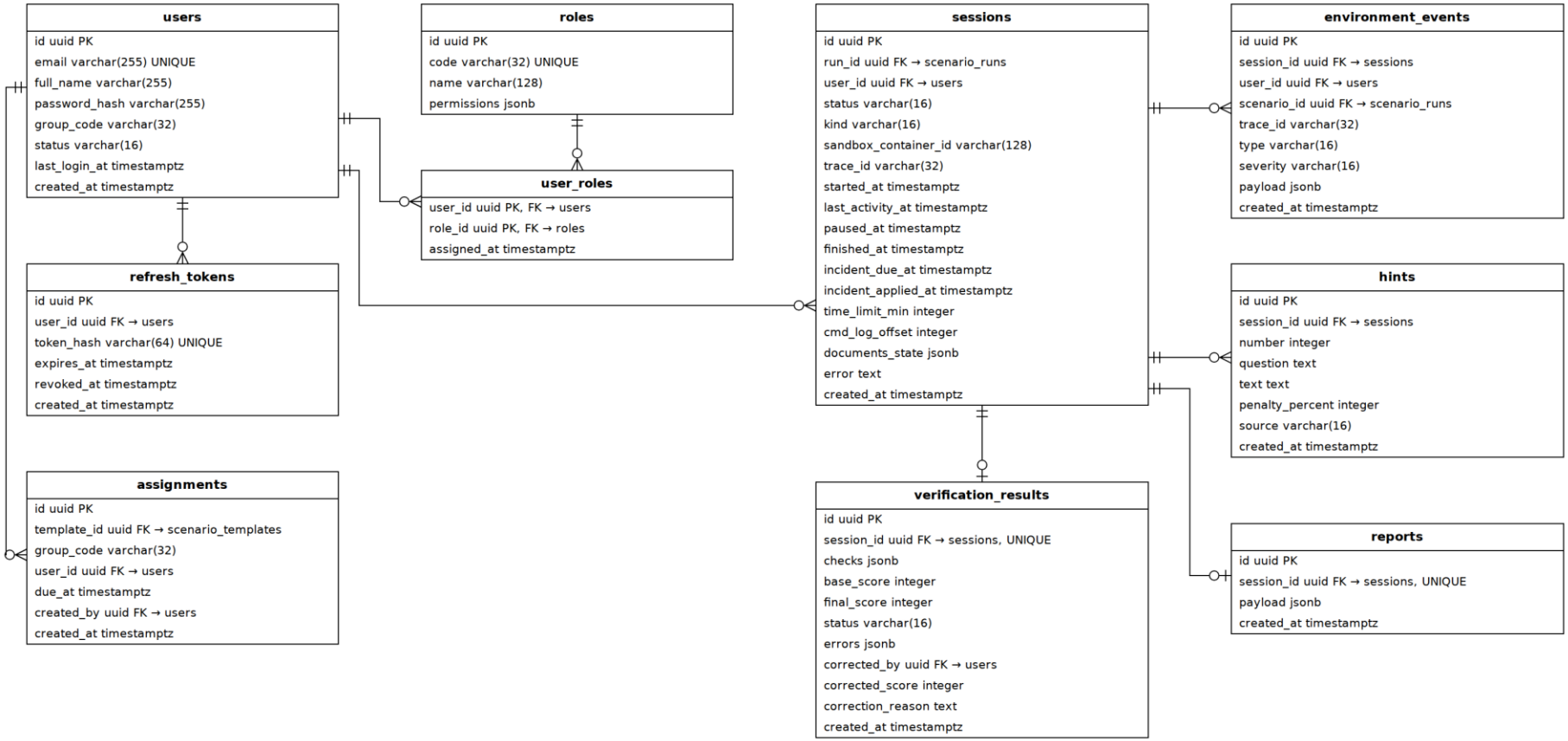


Рис. 21. Логическая модель данных

Физическая модель ядра приведена на Рис. 22: пользователи, роли, refresh-токены, назначения, учебные сессии, цифровой след, подсказки, результаты проверки и отчеты.



Таблицы scenario_templates и scenario_runs приведены на рисунке 23. Линиями показаны основные связи; ссылки environment_events.user_id, assignments.created_by и verification_results.corrected_by на users указаны в описании полей (FK → таблица).

Рис. 22. Физическая модель PostgreSQL: пользователи, сессии и оценивание

Физическая модель графа компетенций и сценариев приведена на Рис. 23. Граф хранится таблицами domains, skills (с теорией навыка) и skill_requires, профиль обучающегося - таблицей user_skills, целевые уровни направления - таблицей

track_targets; векторные представления легенд хранятся в колонке legend_embedding типа vector(1024), подтвержденные решения - в accepted_solutions.

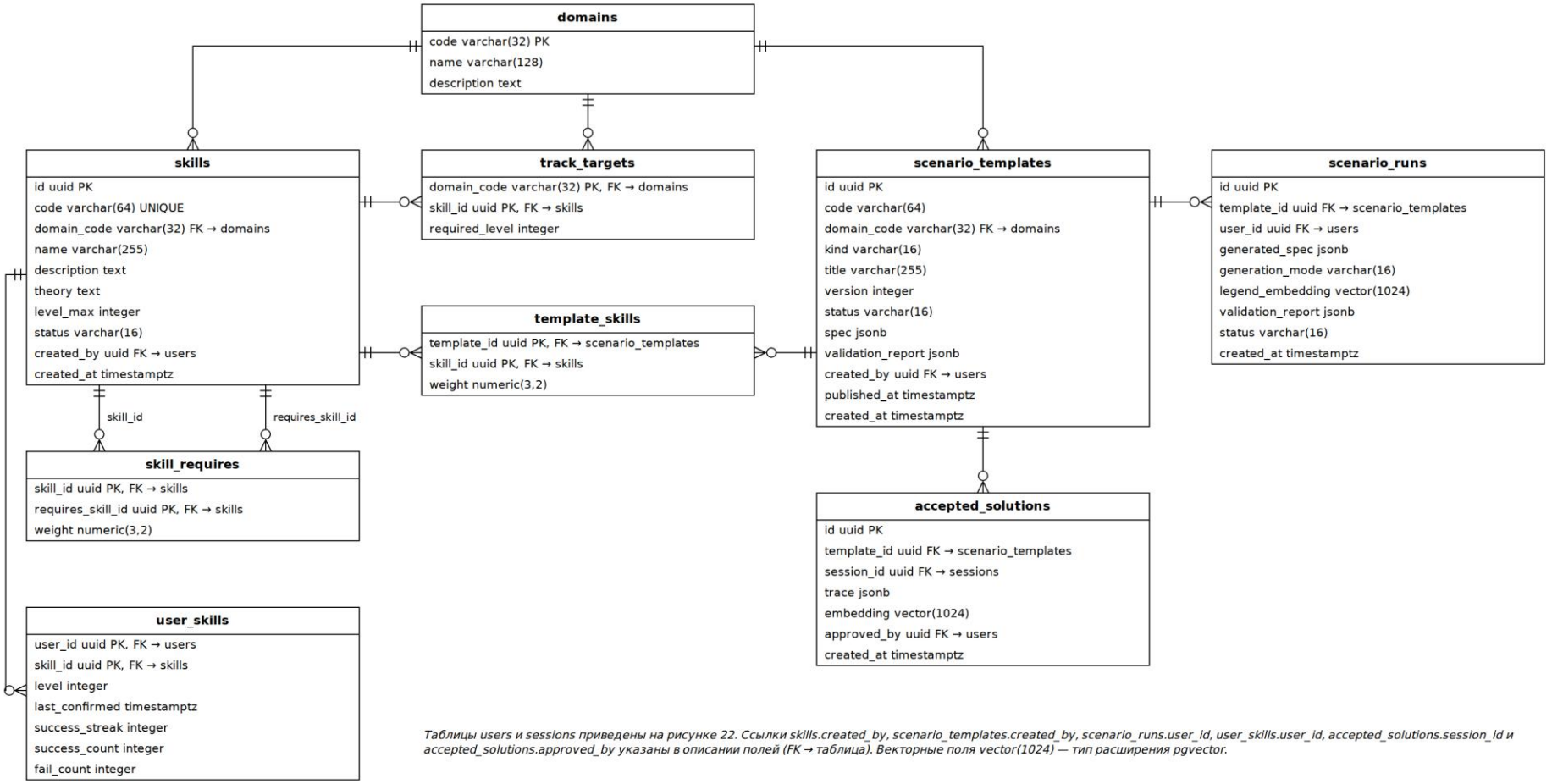


Рис. 23. Физическая модель PostgreSQL: граф компетенций и сценарии

Физическая модель эксплуатационных таблиц приведена на Рис. 24: провайдеры и режим LLM, профили песочниц, журналы аудита, событий безопасности и вызовов агентов.

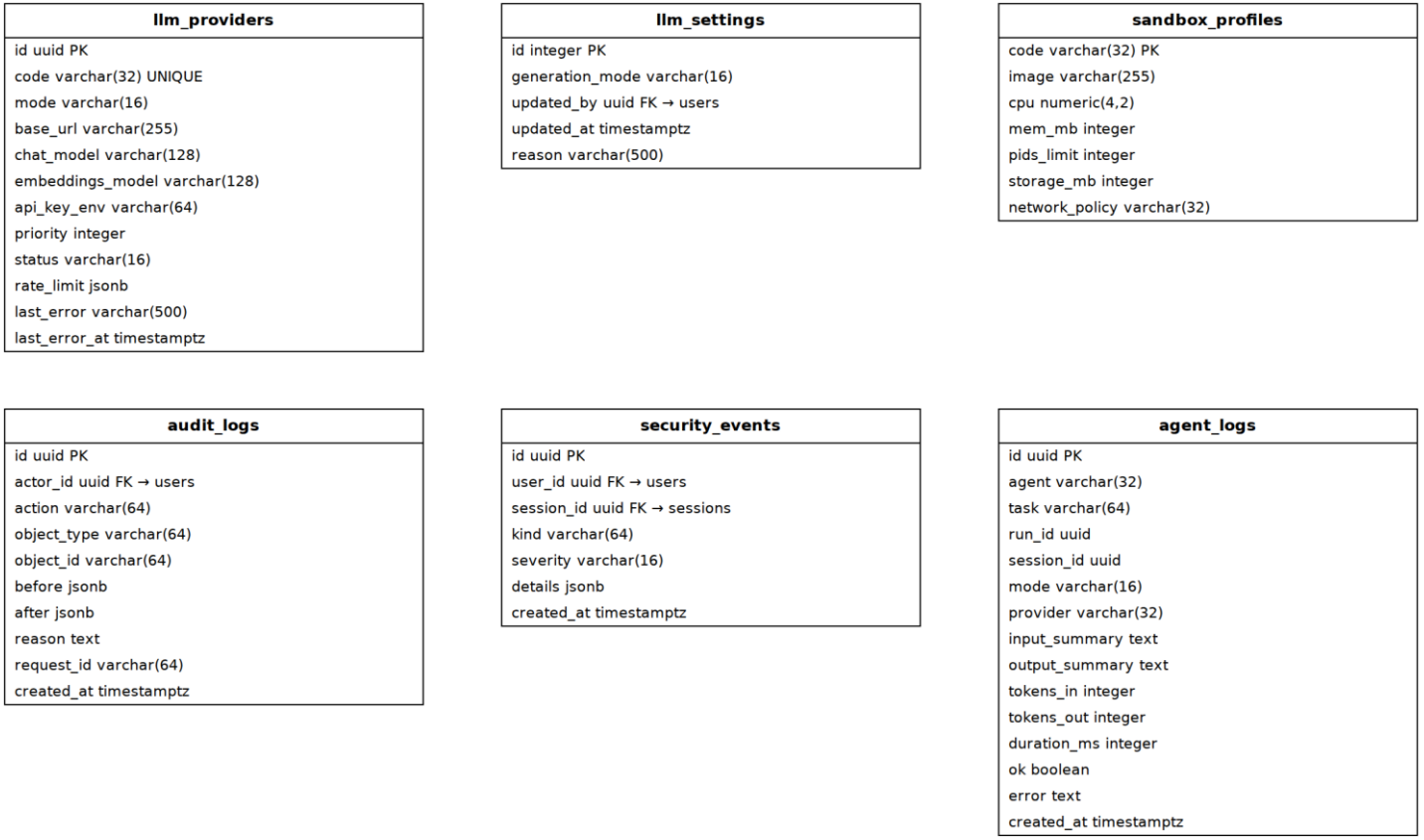


Рис. 24. Физическая модель PostgreSQL: настройки и журналы

Жизненный цикл учебной сессии приведен на Рис. 25. Переходы между статусами выполняют модуль сессий (запуск, пауза, возобновление, отправка), worker (пауза при бездействии, автоотправка по истечении лимита времени, истечение срока) и AssessmentAgent (проверка и завершение).

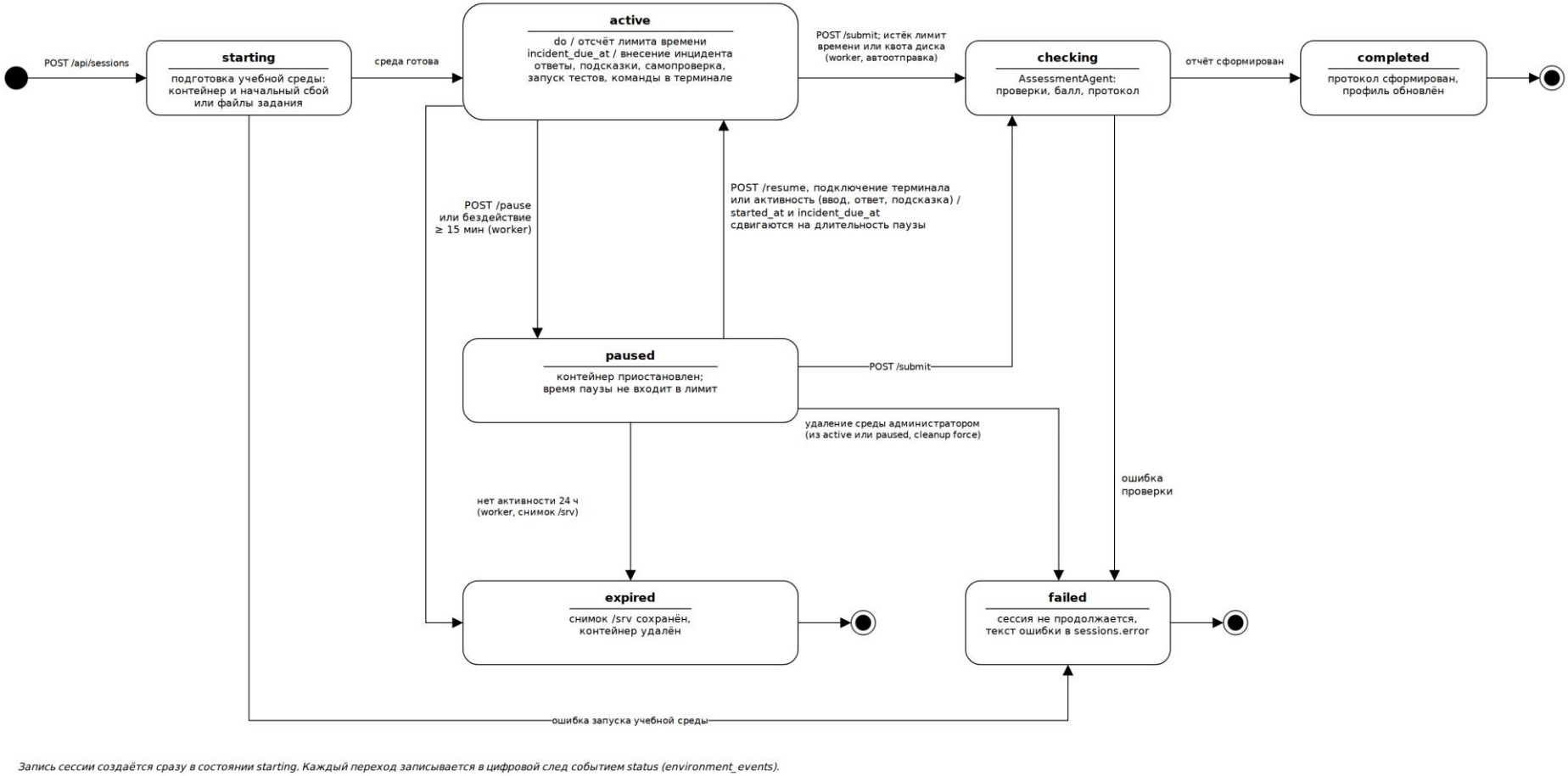


Рис. 25. Диаграмма состояний учебной сессии

Состав основных сущностей приведен в таблице 2. Граф компетенций, векторные представления и состояния фоновых задач хранятся в PostgreSQL; отдельные графовая и векторная СУБД и брокер очередей не используются.

Таблица 2 - Основные сущности API и данных

Сущность	Назначение	Ключевые поля	Таблица
User	Учетная запись пользователя.	id, email, full_name, password_hash, group_code, status, last_login_at	users
Role	Роль доступа.	id, code, name, permissions; user_roles: user_id, role_id, assigned_at	roles, user_roles
RefreshToken	Ротация и отзыв refresh-токенов.	id, user_id, token_hash, expires_at, revoked_at	refresh_tokens
Domain	Предметная область (направление обучения).	code, name, description	domains
Skill	Компетенция предметной области.	id, code, domain_code, name, description, theory, level_max, status, created_by	skills
Competency Graph	Prerequisite-связи, уровни и целевые уровни навыков.	REQUIRES.weight; HAS_SKILL.level, last_confirmed, success_streak, success_count, fail_count; TARGET.required_level	skill_requires, user_skills, track_targets

Сущность	Назначение	Ключевые поля	Таблица
Scenario Template	Утвержденный шаблон сценария.	id, code, domain_code, kind, title, version, status, spec, validation_report, published_at; TARGETS.weight	scenario_templates, template_skills
Scenario Run	Индивидуальный запуск сценария.	id, template_id, user_id, generated_spec, generation_mode, legend_embedding, validation_report, status	scenario_runs
Assignment	Назначение сценария группе или обучающемуся.	id, template_id, group_code, user_id, due_at, created_by	assignments
Learning Session	Факт выполнения задания.	id, run_id, user_id, kind, status, sandbox_container_id, trace_id, started_at, paused_at, finished_at, time_limit_min, incident_due_at, incident_applied_at, last_activity_at, cmd_log_offset, documents_state, error	sessions

Сущность	Назначение	Ключевые поля	Таблица
Session Event	Цифровой след действий.	id, session_id, user_id, scenario_id, type, severity, payload, trace_id	environment_events
Hint	Ограниченная подсказка.	id, session_id, number, question, text, penalty_percent, source	hints
Verification Result	Результат автоматической проверки.	id, session_id, checks, base_score, final_score, status, errors, corrected_by, corrected_score, correction_reason	verification_results
Report	Итоговый отчет.	id, session_id, payload	reports
Accepted Solution	Подтвержденное альтернативное решение.	id, template_id, session_id, trace, embedding, approved_by	accepted_solutions
LLM Provider	Провайдер генерации и режим.	id, code, mode, base_url, chat_model, embeddings_model, api_key_env, priority, status, rate_limit, last_error, last_error_at; llm_settings: generation_mode, reason, updated_by	llm_providers, llm_settings
Sandbox Profile	Лимиты и сетевая политика среды.	code, image, cpu, mem_mb, pids_limit, storage_mb, network_policy	sandbox_profiles

Сущность	Назначение	Ключевые поля	Таблица
Audit / Security	Аудит действий и события безопасности.	actor_id, action, object_type, object_id, before, after, reason, request_id; user_id, session_id, kind, severity, details	audit_logs, security_events
Agent Log	Трассировка вызовов агентов и LLM.	agent, task, run_id, session_id, mode, provider, tokens_in, tokens_out, duration_ms, ok, error	agent_logs

6. Архитектура системы в нотации C4

Архитектура проектируется как модульный монолит с четкими границами компонентов: клиентское веб-приложение, Backend/API, агенты на LangGraph, LLM Gateway, Sandbox Manager, Verification Engine, фоновый worker и слой данных. Границы модулей совпадают с компонентами C4, что позволяет во второй очереди выделить их в отдельные сервисы.

Контекстная диаграмма C4 приведена на Рис. 26. Она показывает пользователей системы по ролям, внешнего LLM-провайдера (GigaChat через адаптер gpt2giga или другой OpenAI-совместимый API), локальную модель и среду исполнения учебных контейнеров.

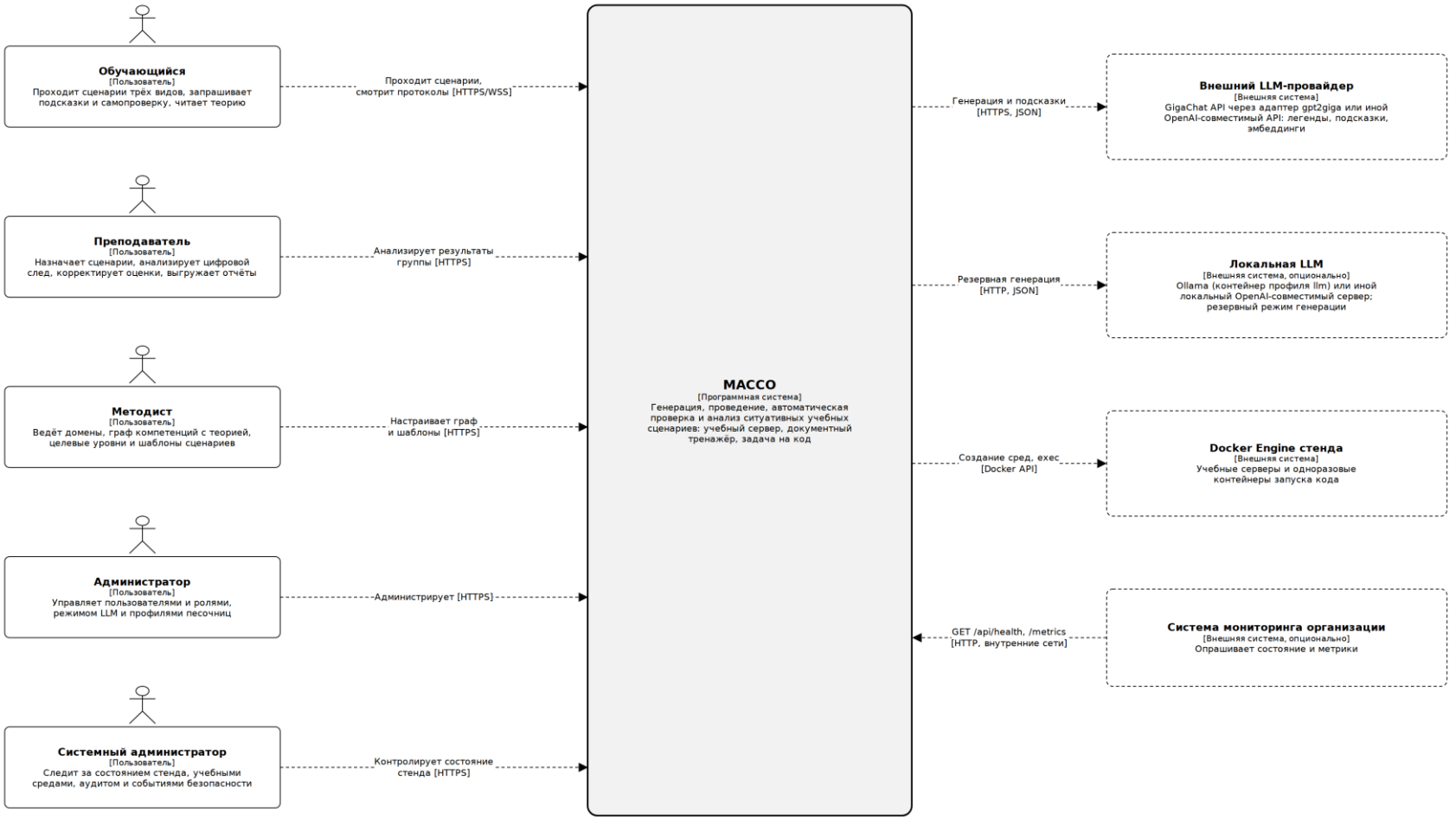


Рис. 26. C4 Context: MACCO во внешней среде

Диаграмма контейнеров приведена на Рис. 27. В ней закреплены контейнеры Frontend (SPA за Nginx), Backend/API (включая фоновый worker как задачу того же процесса), PostgreSQL с pgvector, учебные среды, создаваемые через Docker Engine, а также необязательные контейнеры gpt2giga (адаптер GigaChat) и Ollama (локальная модель).

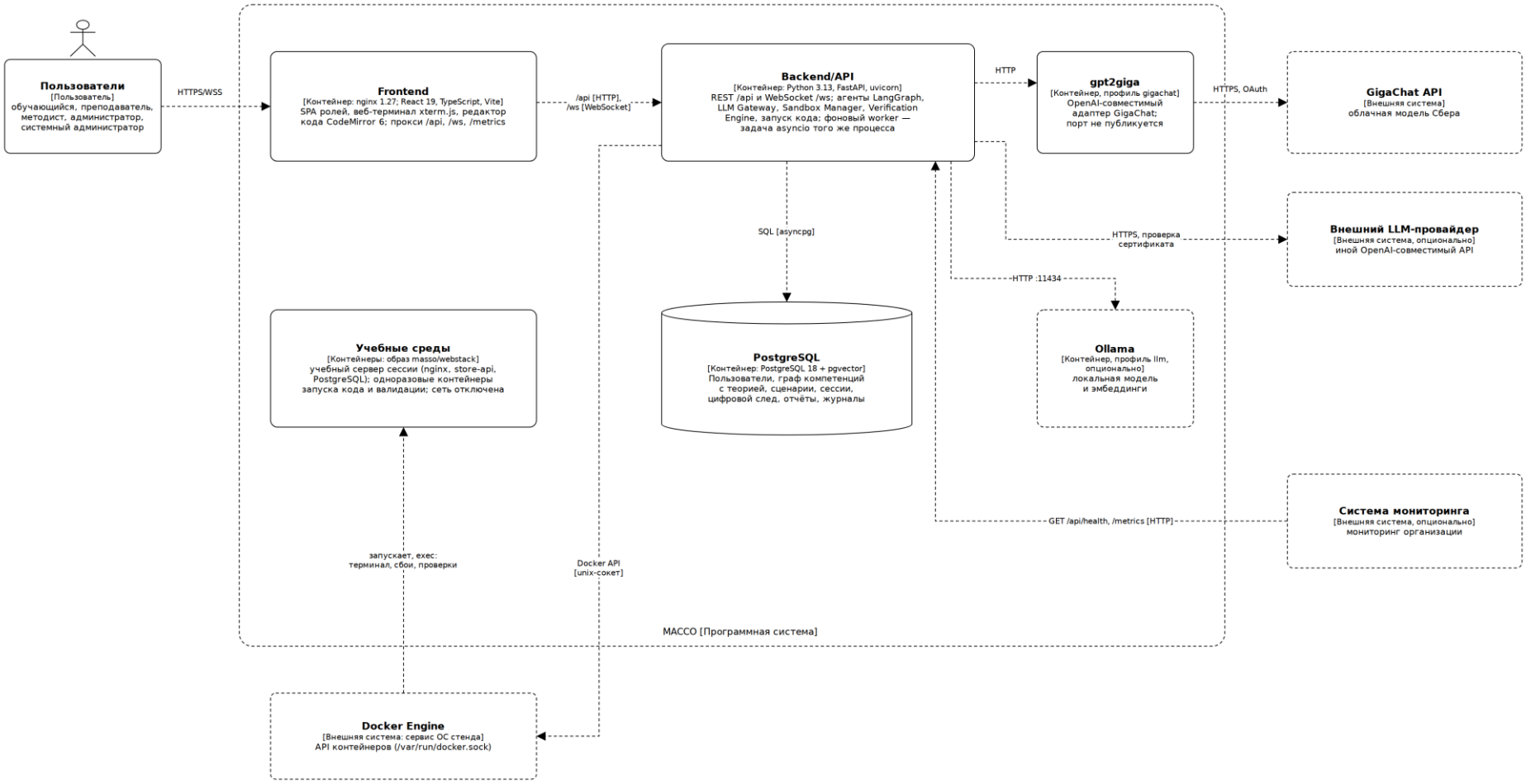


Рис. 27. C4 Container: контейнеры системы

Соотношение первой очереди и целевой архитектуры приведено на Рис. 28. Во второй очереди агенты, Sandbox Manager и Verification Engine выделяются в отдельные сервисы, а учебные среды получают сетевой allowlist и Sysbox; хранилище PostgreSQL с pgvector, Docker Engine и мониторинг через /metrics остаются прежними.

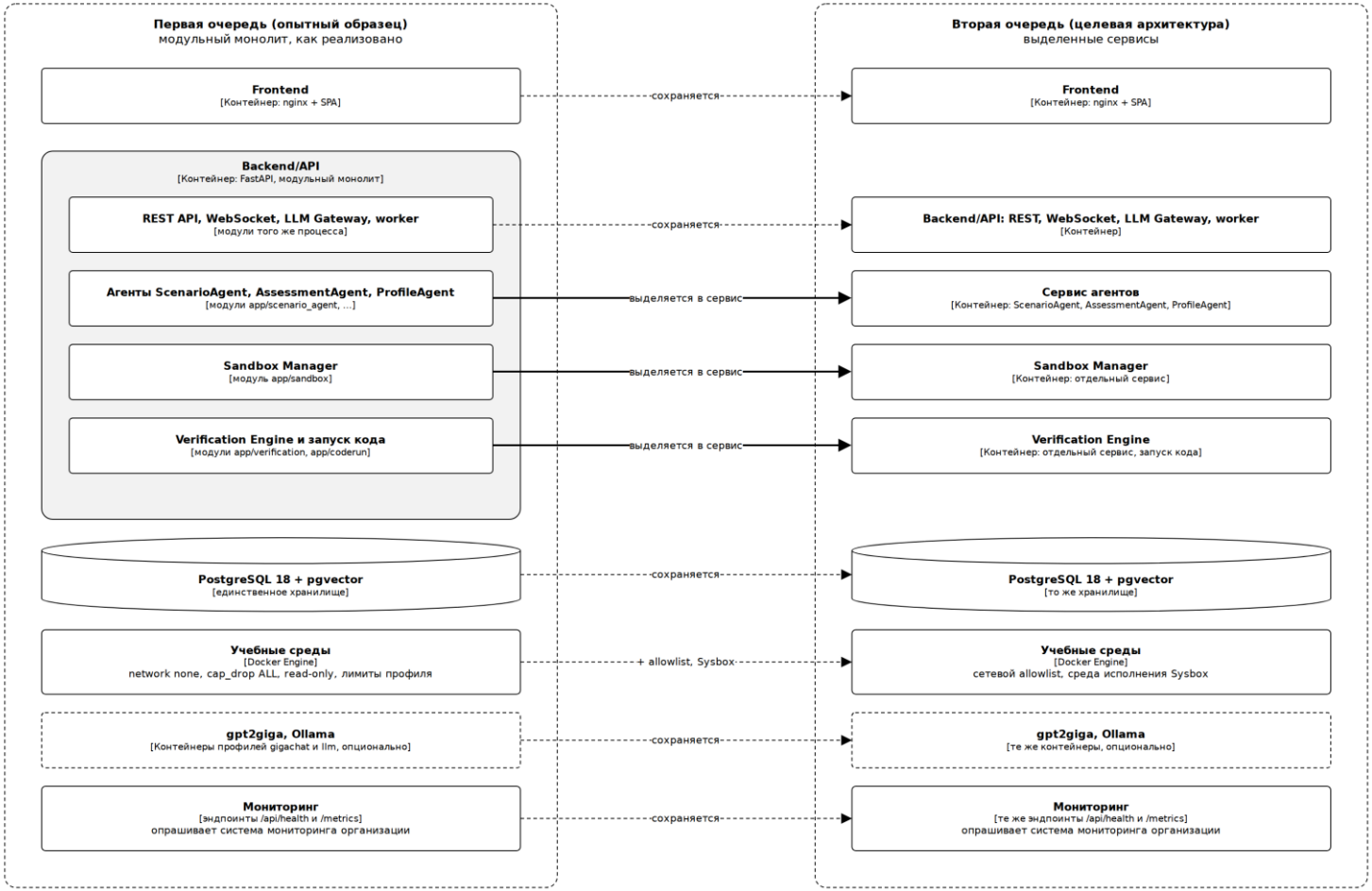


Рис. 28. Первая очередь и целевая архитектура

Компонентные диаграммы Backend/API и агентов приведены на Рис. 29 и Рис. 30. Они показывают модули API, сервисный слой, агенты, модуль запуска кода и адаптеры к данным и инфраструктуре.

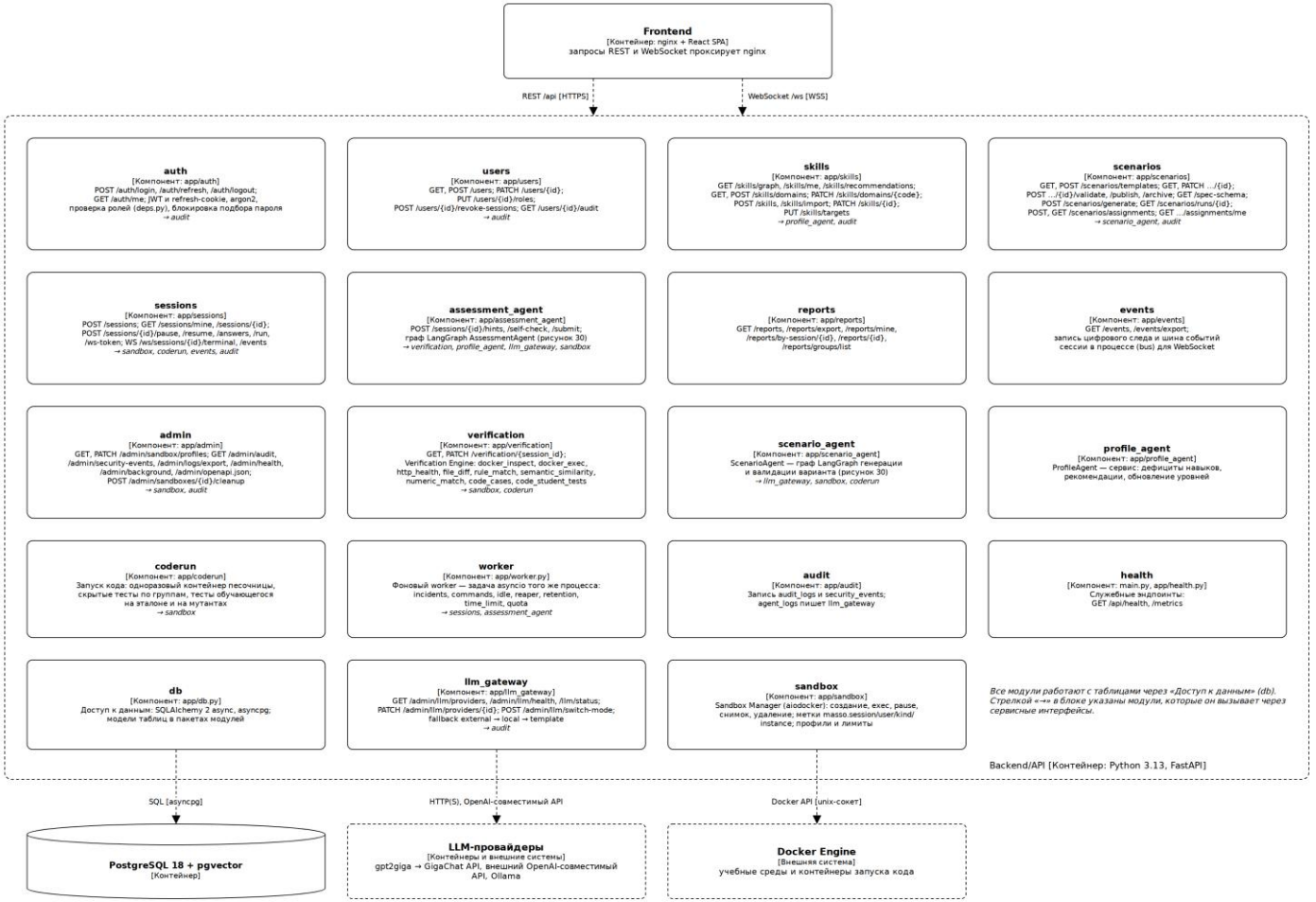


Рис. 29. C4 Component: Backend/API

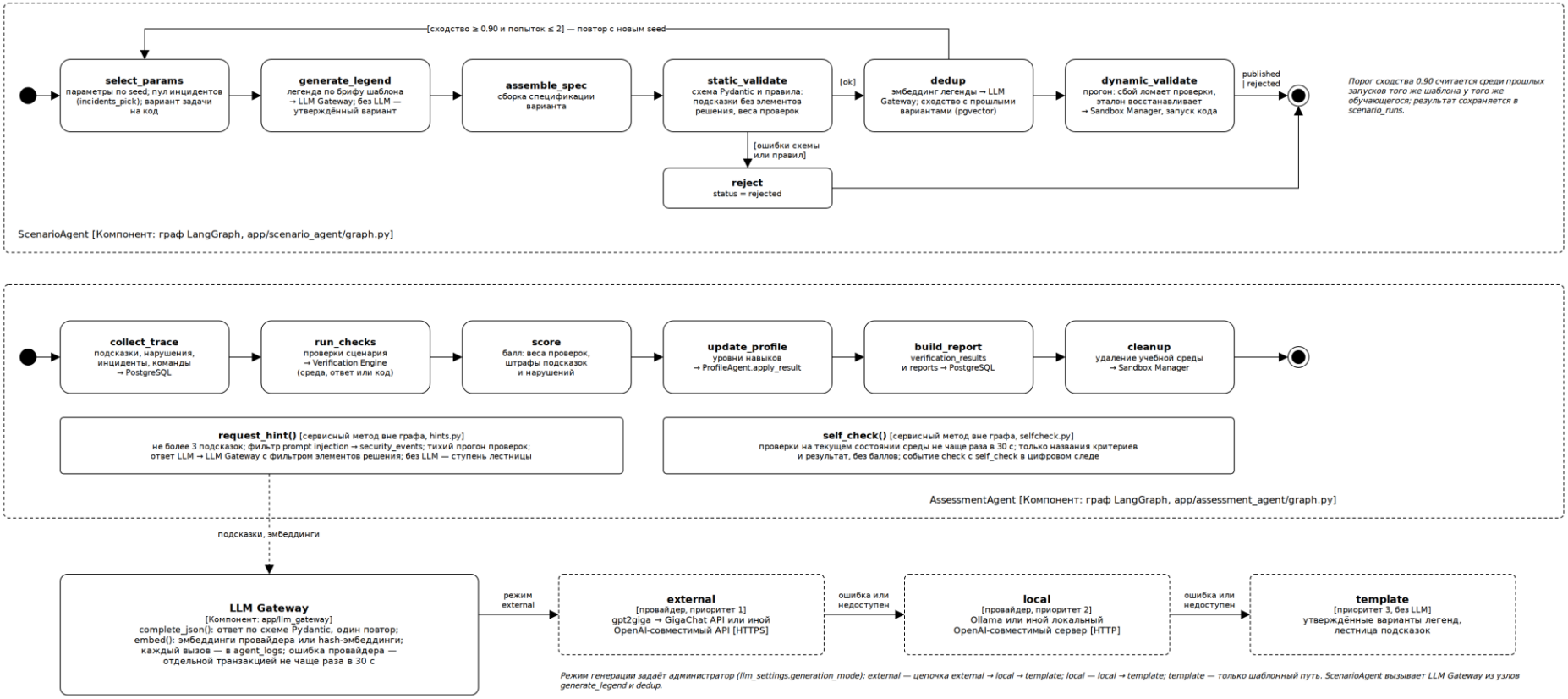


Рис. 30. C4 Component: агенты и LLM Gateway

Диаграмма размещения приведена на Рис. 31. Она разделяет клиентскую зону, прикладной сервер с контейнерами системы и учебными средами, а также внешние LLM-провайдеры; в эксплуатационном режиме браузер обращается к системе по HTTPS/WSS, в тестовом - по HTTP на localhost.

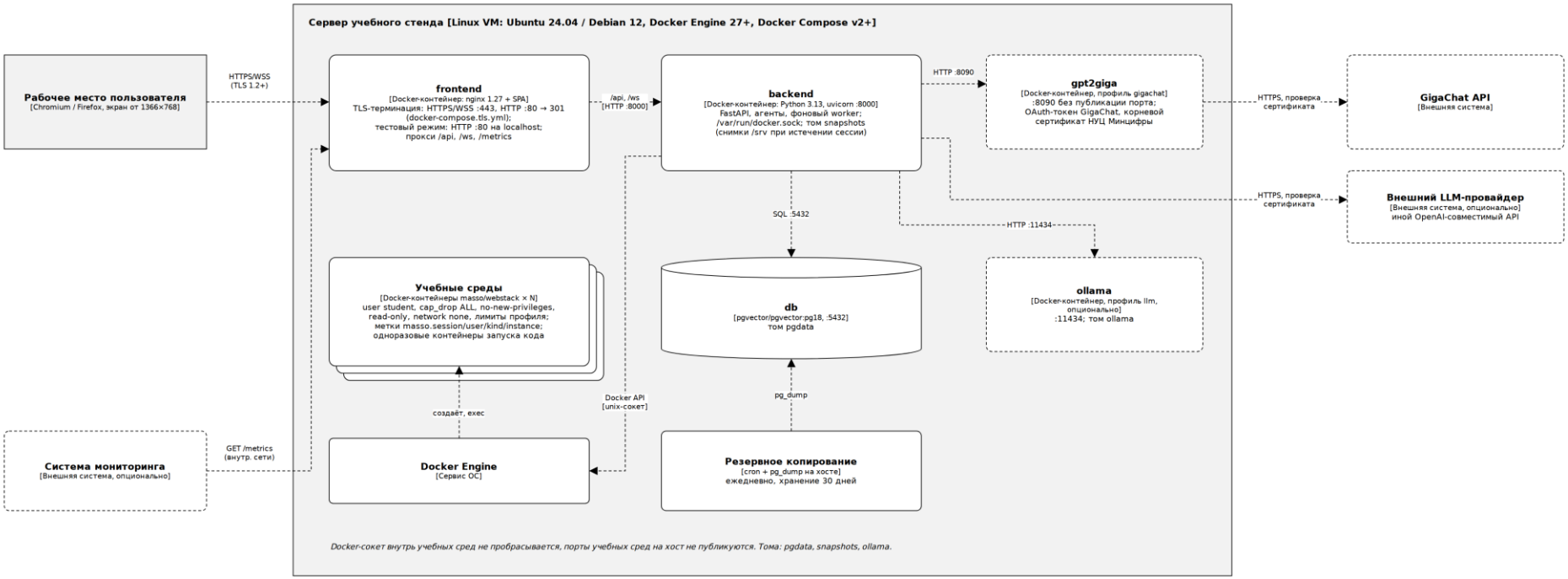


Рис. 31. C4 Deployment: размещение системы

Архитектурные решения.

- Frontend реализуется как одностраничное приложение на Vite, React 19 и TypeScript с интерфейсами обучающегося, преподавателя, методиста, администратора и системного администратора; веб-терминал - xterm.js, графы - React Flow, редактор кода - CodeMirror 6, светлая и темная тема оформления.
- Backend/API реализуется на FastAPI как модульный монолит и предоставляет REST, WebSocket и OpenAPI-спецификацию; модули взаимодействуют через сервисные интерфейсы, а не через HTTP.

- Внешний периметр - Nginx в контейнере frontend: в эксплуатационном режиме он завершает TLS (HTTPS и WSS, TLS 1.2 и 1.3, HSTS), перенаправляет HTTP на HTTPS и передает backend адрес клиента и схему (X-Forwarded-For, X-Forwarded-Proto). Порты backend и PostgreSQL наружу не публикуются, трафик между контейнерами идет во внутренней сети Docker. Тестовый режим опытного образца работает по HTTP на localhost; режим включается конфигурацией развертывания без изменения кода.
- LangGraph используется для графов ScenarioAgent (генерация и валидация) и AssessmentAgent (проверка, балл, отчет); ProfileAgent реализован как сервис.
- LLM Gateway обращается ко всем провайдерам через OpenAI-совместимый API и скрывает различия внешнего провайдера, локальной модели и шаблонного режима деградации. Ошибки провайдеров записываются в llm_providers отдельной короткой транзакцией, не чаще раза в 30 с, чтобы при недоступном провайдере параллельные генерации не блокировали друг друга. Внешний провайдер GigaChat подключается через контейнер-адаптер gpt2giga во внутренней сети Docker (обновление OAuth-токена, преобразование запросов в OpenAI-совместимый формат); в эксплуатационном режиме адаптер проверяет сертификат GigaChat по корневому сертификату НУЦ Минцифры. Режим JSON mode включается для каждого провайдера параметром конфигурации; число одновременных запросов к модели ограничивается адаптером по тарифу провайдера (для физических лиц - один), остальные запросы ждут в очереди.
- Sandbox Manager создает учебные среды через Docker API без привилегированного режима, от пользователя student, с cap-drop ALL, no-new-privileges, файловой системой только для чтения и отключенной сетью. Контейнеры помечаются метками masso.session, masso.user, masso.kind и masso.instance; очистка осиротевших контейнеров учитывает только метку своей установки.

- Модуль запуска кода (coderun) выполняет код обучающегося в одноразовом контейнере образа песочницы с теми же флагами безопасности и профилем code; в контейнер передаются только файлы обучающегося и проверочные скрипты, ожидаемые значения скрытых тестов вычисляет эталонное решение на backend, крупные результаты сравниваются по хешу.
- Verification Engine выполняет проверки контейнерных, документных сценариев и задач на код; команды проверок передаются в среду в момент выполнения и не хранятся в образе.
- Фоновый worker вносит инциденты, контролирует бездействие, лимит времени и объем диска учебных сред, удаляет среды и устаревшие данные; состояние задач хранится только в PostgreSQL.
- Наблюдаемость обеспечивают эндпоинты /health и /metrics, журналы audit_logs, security_events и agent_logs.

7. Модели информационных потоков данных

Модели информационных потоков выполнены в нотации DFD. Они фиксируют внешние сущности, процессы, хранилища и направления движения данных; хранилища D1-D7 соответствуют группам таблиц PostgreSQL.

Контекстная DFD приведена на Рис. 32. На ней система MACCO показана единым процессом, взаимодействующим с пользователями, LLM-провайдером и средой исполнения учебных контейнеров.



Рис. 32. Контекстная DFD MACCO

DFD уровня 1 для жизненного цикла обучения приведена на Рис. 33. Поток начинается с профиля компетенций, проходит генерацию сценария, выполнение задания с теорией, самопроверкой и запуском кода и завершается автоматической проверкой.

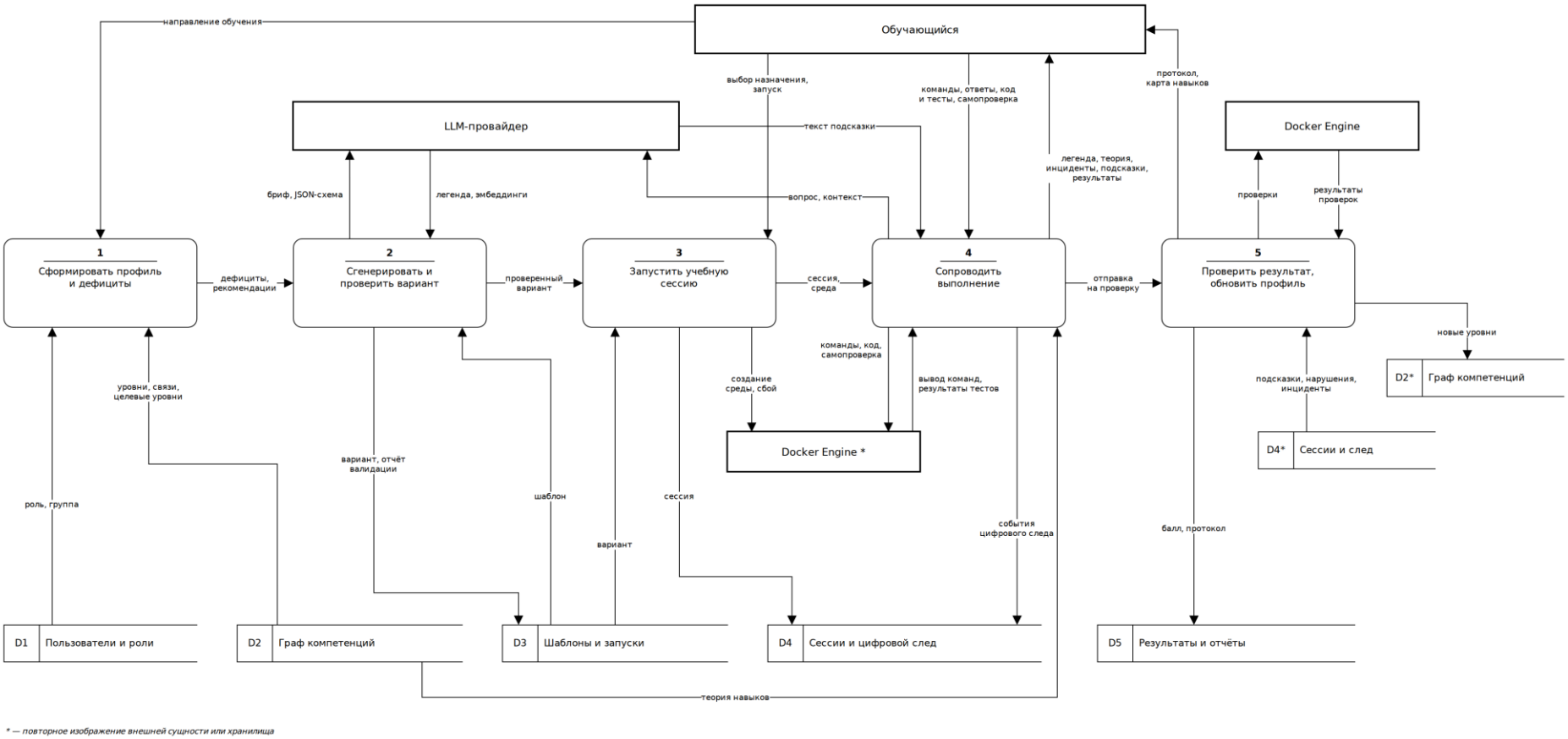


Рис. 33. DFD уровня 1: жизненный цикл обучения

DFD уровня 2 для отчетности и администрирования приведена на Рис. 34. На ней выделены фильтры периода, выгрузка отчетов, экспертная корректировка, настройка LLM и песочниц.

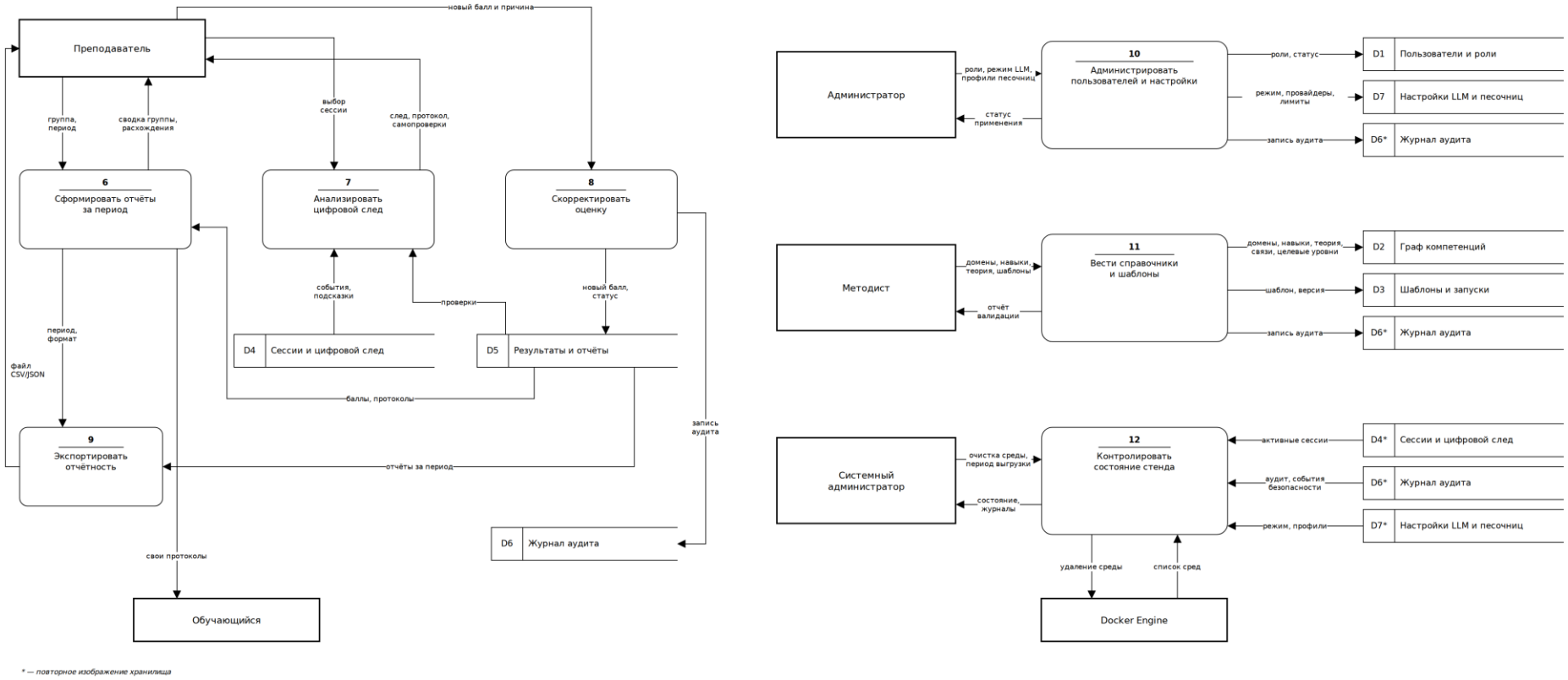


Рис. 34. DFD уровня 2: отчетность и администрирование

8. Шаблоны справочников и НСИ

Справочники используются при вводе и фильтрации информации, чтобы снизить число ошибок, обеспечить единообразие данных и выполнить требования методических рекомендаций к техническому проекту. Справочники, которые ведутся в интерфейсе, хранятся в таблицах: domains (code, name, description), roles (code, name, permissions), sandbox_profiles (code и лимиты), llm_providers; статусы, типы и severity - фиксированные перечисления системы. Права ролей закреплены в системе. Изменения предметных областей и профилей песочниц аудитируются.

Таблица 3 - Роли и права

Код	Роль	Основные права
student	Обучающийся	Запуск назначенных сценариев, работа в среде, запрос подсказок, самопроверка, отправка результата, теория, свои отчеты и карта компетенций.
teacher	Преподаватель	Просмотр своей группы (поле group_code учетной записи; без группы - все группы), цифрового следа, отчетов, экспертное подтверждение или корректировка оценки, назначение сценариев, экспорт отчетов.
methodist	Методист	Справочник предметных областей, граф компетенций и теория навыков, целевые уровни, шаблоны сценариев, checks, правила подсказок.
admin	Администратор	Пользователи, роли, отзыв сессий входа, LLM-провайдеры и режим, лимиты sandbox-профилей, аудит и события безопасности; доступ к экранам методиста и состоянию системы.
sysadmin	Системный администратор	Мониторинг инфраструктуры, учебных сред и событий безопасности, ручная очистка сред, резервное копирование, обновление сервисов.

Таблица 4 - Предметные области и виды сценариев

Код	Название	Пример использования
devops	DevOps и администрирование	Контейнерный движок: восстановление веб-сервиса (nginx, приложение, PostgreSQL) на учебном сервере с инцидентами из пула.
security	Информационная безопасность	Направление справочника для шаблонов контейнерного движка (анализ активности, ограничение прав); шаблоны направления создает методист.
legal	Юриспруденция	Документный движок: поиск противоречий в договоре и учет новой нормативной вводной.
audit	Экономика и аудит	Документный движок: сверка первичных документов, обнаружение расхождения, расчет суммы.
algorithms	Алгоритмы и структуры данных	Движок задач на код: решение и собственные тесты (поиск, разбор вложенности, обход графа).

Таблица 5 - Статусы и события

Справочник	Значения	Назначение
Статусы навыка	draft, active, deprecated, archived	Управление жизненным циклом навыков в графе компетенций.
Статусы шаблона и запуска	шаблон: draft, published, archived; запуск сценария: published, rejected	Контроль публикации и версии шаблона; результат валидации варианта.
Статусы сессии	starting, active, paused, checking, completed, failed, expired	Отображение состояния учебной сессии (рис. 25).
Типы событий	command, answer, incident, hint, check, status, security	Классификация цифрового следа; status - смена состояния сессии (ошибки среды - status с severity error).
Severity	события: info, warning, error, critical; события безопасности: info, warning, medium, high	Фильтрация событий и сообщений в интерфейсе.

Справочник	Значения	Назначение
Результат и экспертная проверка	status: passed, failed; статус экспертной проверки (вычисляется): pending, confirmed, corrected	Отображение статуса оценки и отбор сессий с расхождением автоматической и экспертной оценки.
Виды событий безопасности	privilege_escalation, sandbox_escape, network_scan, forbidden_file, prompt_injection, log_tampering, sandbox_quota, login_failed, refresh_reuse	Классификация security_events и фильтр журнала безопасности.
Статусы пользователя и провайдера	пользователь: active, blocked; LLM-провайдер: active, standby, disabled	Блокировка учетной записи; участие провайдера в цепочке external → local → template.

Таблица 6 - Проверки, подсказки, отчеты и инфраструктура

Справочник	Значения	Использование
Политики подсказок	none, limited_3, teacher_allowed (в первой очереди действует как limited_3); лестница из 3 ступеней	Ограничение числа подсказок и расчет штрафа.
Типы проверок	docker_inspect, docker_exec, http_health, file_diff, rule_match, semantic_similarity, numeric_match, code_cases, code_student_tests	Запуск Verification Engine; numeric_match - сравнение суммы с эталоном с допуском; code_cases - скрытые тесты задачи на код по группам; code_student_tests - проверка собственных тестов обучающегося на эталоне и типичных ошибках.
Критерии оценки	вес проверки, признак critical (в шаблоне не менее одной критической проверки), порог зачета (по умолчанию 75)	Расчет итогового балла; провал critical ограничивает базовый балл значением 49.
Приоритеты дефицитов	critical, high, medium, low	Ранжирование рекомендаций ProfileAgent.

Справочник	Значения	Использование
Каталог сбоев	код сбоя, команда внесения, эталонное исправление; для документов - тип дефекта и ключ ответа; пул инцидентов и число инцидентов варианта	Начальный сбой и инциденты сценария; скрыт от обучающегося.
Виды сценариев (движки)	container, document, code	Учебный сервер в контейнере, документный тренажер, задача на код.
Форматы отчетов	csv, json; печатная форма - средствами браузера; импорт графа - json, csv	Экспорт отчетности с обязательными хронологическими фильтрами.
LLM-режимы	external, local, template	Выбор режима генерации и деградации.
Sandbox-профили	small (1 CPU, 1024 МБ, 128 процессов, 512 МБ), medium (2 CPU, 2048 МБ, 256, 1024 МБ), document, code (1 CPU, 512 МБ, 64, 64 МБ)	Лимиты CPU/RAM/процессов/storage и вид учебной среды; code - одноразовые контейнеры запуска кода.
Сетевые политики	deny_all (единственная политика первой очереди); allowlist - вторая очередь	Ограничение сетевого доступа учебных сред.

9. Проектирование API

Публичный API проектируется в REST-стиле с префиксом /api. Все ответы возвращаются в JSON и содержат request_id, status, data или error. Для терминала и потока событий сессии используется WebSocket (WSS в эксплуатационном режиме) с краткоживущим токеном (60 секунд), привязанным к сессии и пользователю. Все обращения к API в эксплуатационном режиме выполняются по HTTPS.

Таблица 7 - Общий формат REST-ответа

Поле	Тип	Описание
request_id	string	Корреляционный идентификатор запроса.

Поле	Тип	Описание
status	success | error	Результат обработки.
data	object | array | null	Полезная нагрузка успешного ответа.
error.code	string	Машинно-читаемый код ошибки.
error.message	string	Понятное сообщение на русском языке.
error.details	object	Дополнительные сведения без раскрытия секретов.

Таблица 8 - REST-контуры API

Контур	Назначение	Операции
/auth	Аутентификация и сессии доступа.	POST /login, POST /refresh, POST /logout, GET /me.
/users	Пользователи, группы, роли.	GET /users, POST /users, PATCH /users/{id}, PUT /users/{id}/roles, POST /users/{id}/revoke-sessions (отзыв всех refresh-токенов), GET /users/{id}/audit.
/skills	Предметные области, граф компетенций и уровни навыков.	GET /graph?domain, GET /domains, POST /domains, PATCH /domains/{code}, POST /, PATCH /{id}, POST /import, PUT /targets (целевые уровни направления), GET /me, GET /recommendations.
/scenarios	Шаблоны, генерация, назначения.	GET /templates, POST /templates, GET /templates/{id}, PATCH /templates/{id}, POST /templates/{id}/validate, POST /templates/{id}/publish, POST /templates/{id}/archive, GET /spec-schema (JSON Schema спецификации для редактора), POST /generate, GET /runs/{id}, GET /assignments, POST /assignments, GET /assignments/me.
/sessions	Учебные сессии.	POST /, GET /mine, GET /{id}, POST /{id}/pause, POST /{id}/resume, POST /{id}/answers, POST /{id}/run (запуск примеров и тестов задачи на код), POST /{id}/ws-token, POST /{id}/hints, POST /{id}/self-check (самопроверка: названия критериев и результат), POST /{id}/submit.
/events	Цифровой след.	GET ?session_id&from&to&type&severity, GET /export?session_id&format.

Контур	Назначение	Операции
/verification	Результат проверки и экспертная оценка.	GET /{session_id}, PATCH /{session_id} (оценка и обязательная причина).
/reports	Отчеты и аналитика.	GET ?group&from&to, GET /mine, GET /by-session/{session_id}, GET /groups/list, GET /{id}, GET /export?from&to&format=csv|json.
/llm	Состояние LLM-контура для всех ролей.	GET /status (режим генерации и причина переключения).
/admin	LLM, песочницы, аудит, состояние.	GET /llm/providers, PATCH /llm/providers/{id}, POST /llm/switch-mode, GET /llm/health, GET /sandbox/profiles, PATCH /sandbox/profiles/{code}, GET /audit?from&to, GET /security-events, GET /logs/export, GET /background, POST /sandboxes/{container_id}/cleanup, GET /health, GET /openapi.json.
служебные	Наблюдаемость.	GET /api/health, GET /metrics.

Таблица 9 - WebSocket-каналы

Канал	Сообщения	Назначение
/ws/sessions/{id}/terminal?token=	двоичные кадры stdin/stdout или JSON {"type":"input"}, JSON-кадр resize; подключение возобновляет сессию на паузе	Интерактивный терминал контейнерной среды.
/ws/sessions/{id}/events?token=	status, incident, hint, security, check_status; служебное сообщение {"type":"ping"} после 25 с без событий. Коды закрытия: 4401 - токен недействителен, 4403 - нет доступа, 4409 - сессия не готова	Поток событий и статусов учебной сессии.

Основные ошибки API.

Система возвращает единые коды ошибок (в скобках - HTTP-статус): AUTH_INVALID_CREDENTIALS (401, в том числе при временной блокировке входа), AUTH_FORBIDDEN (403), SCENARIO_NOT_VALID (422), SESSION_NOT_READY (409), HINT_LIMIT_EXCEEDED (409), VERIFICATION_FAILED (500), LLM_PROVIDER_UNAVAILABLE (503; при недоступности провайдеров генерация переходит в шаблонный режим, поэтому код зарезервирован), SANDBOX_LIMIT_EXCEEDED (503), REPORT_PERIOD_REQUIRED (422), VALIDATION_ERROR (422), NOT_FOUND (404). Непредвиденная ошибка сервера возвращается с HTTP 500 и кодом VERIFICATION_FAILED с общим сообщением без внутренних деталей.

10. Модель структуры интерфейса

Модель структуры интерфейса показана на Рис. 35. Она разделяет общие экраны и рабочие области по ролям: обучающийся, преподаватель, методист, администратор и системный администратор; навигация по разделам роли - верхнее меню.

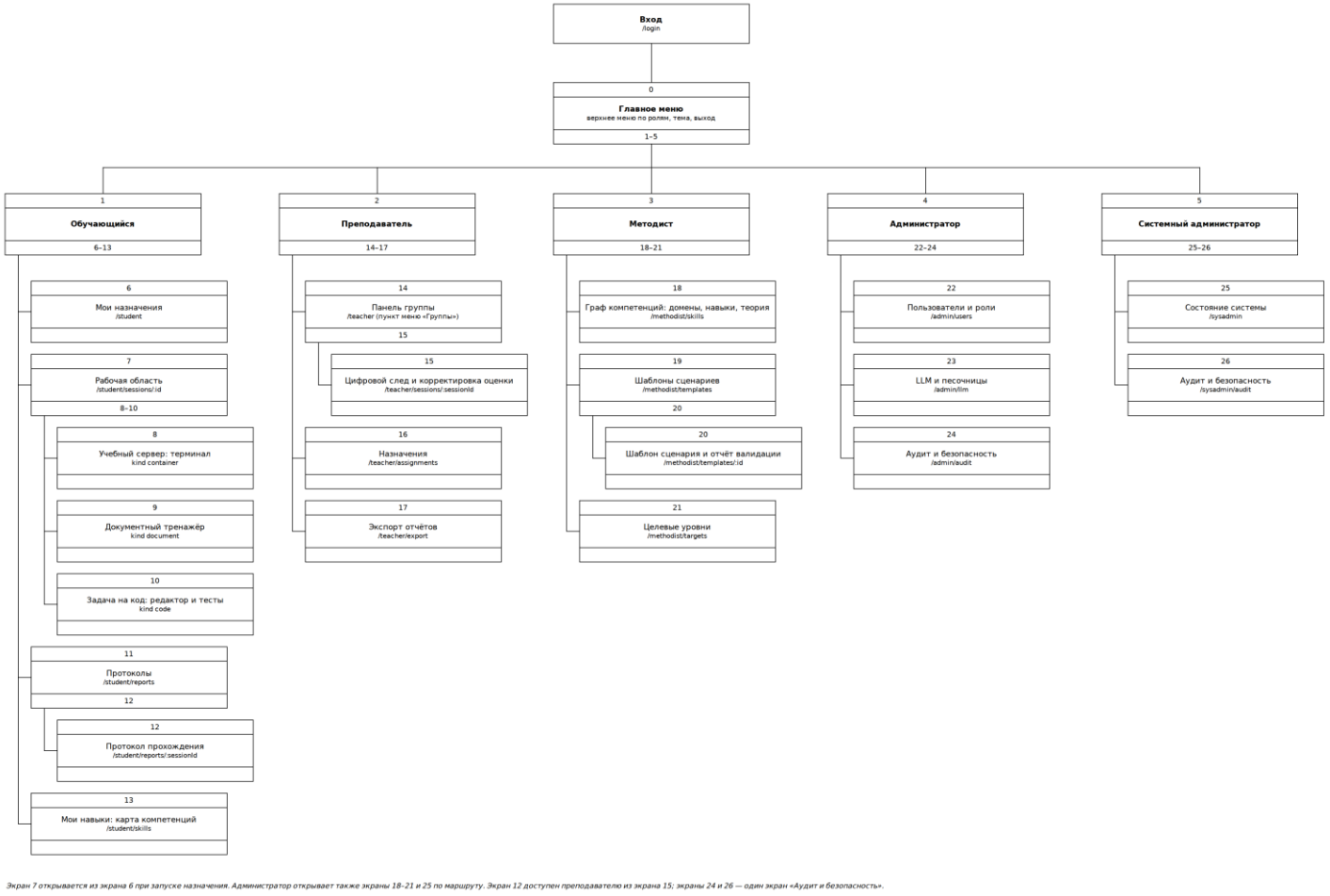


Рис. 35. Модель структуры интерфейса системы

Обязательные интерфейсные решения.

- Метафора интерфейса: учебная диспетчерская, где сценарий выглядит как назначение, рабочая среда - как рабочее место, а отчет - как протокол прохождения.

- Оформление «мягкий современный»: верхнее меню разделов роли в виде вкладок-«пилюль», карточки со скругленными углами, теплая нейтральная основа и оранжевый акцент для основных действий; зеленый - для успешных проверок, желтый - для предупреждений, красный - для ошибок и событий безопасности. Интерфейс поддерживает светлую и темную тему; по умолчанию тема выбирается по настройке операционной системы, переключатель - в шапке.
- При вводе информации используются справочники: роли, группы, предметные области, типы сценариев, статусы, severity, форматы отчетов, LLM-режимы, sandbox-профили.
- Сообщения состояния обязательны для генерации, запуска среды, ожидания проверки, экспорта отчета и переключения LLM-контура.
- Сообщения об ошибках должны быть понятны пользователю и не раскрывать системные промпты, секреты провайдеров, внутренние checks и пути хостовой файловой системы.
- В формах ввода сохраняется пропорция: подпись занимает около 30-35 % ширины строки, поле ввода - 65-70 %.
- Отчетность всегда содержит хронологические фильтры «Дата начала» и «Дата окончания»; без периода экспорт не запускается.

11. Макеты интерфейсов системы

Макеты экранов выполнены в Claude Design как интерактивный холст с артбордами: экраны связаны переходами по пунктам меню и кнопкам, для каждого артборда доступно переключение светлой и темной темы. Макеты являются эталоном внешнего вида реализации. Общий стиль: верхнее меню разделов роли, карточки на теплой нейтральной основе, оранжевый акцент действий, статусные метки в цветах из раздела 10. Макеты показывают состав элементов и сообщения

состояния на обобщенных данных: вместо конкретных значений используются заполнители (N и NN - числа, ДД.ММ.ГГГГ и ЧЧ:ММ - дата и время, «Сценарий N», «Навык N», «Пользователь N» и т. п.). Исходные файлы холста и изображения экранов приложены к проекту.

Экран входа показан на Рис. 36. Он содержит поля учетных данных, сообщение об ошибке входа (в том числе о временной блокировке после серии неудачных попыток) и переключатель темы.

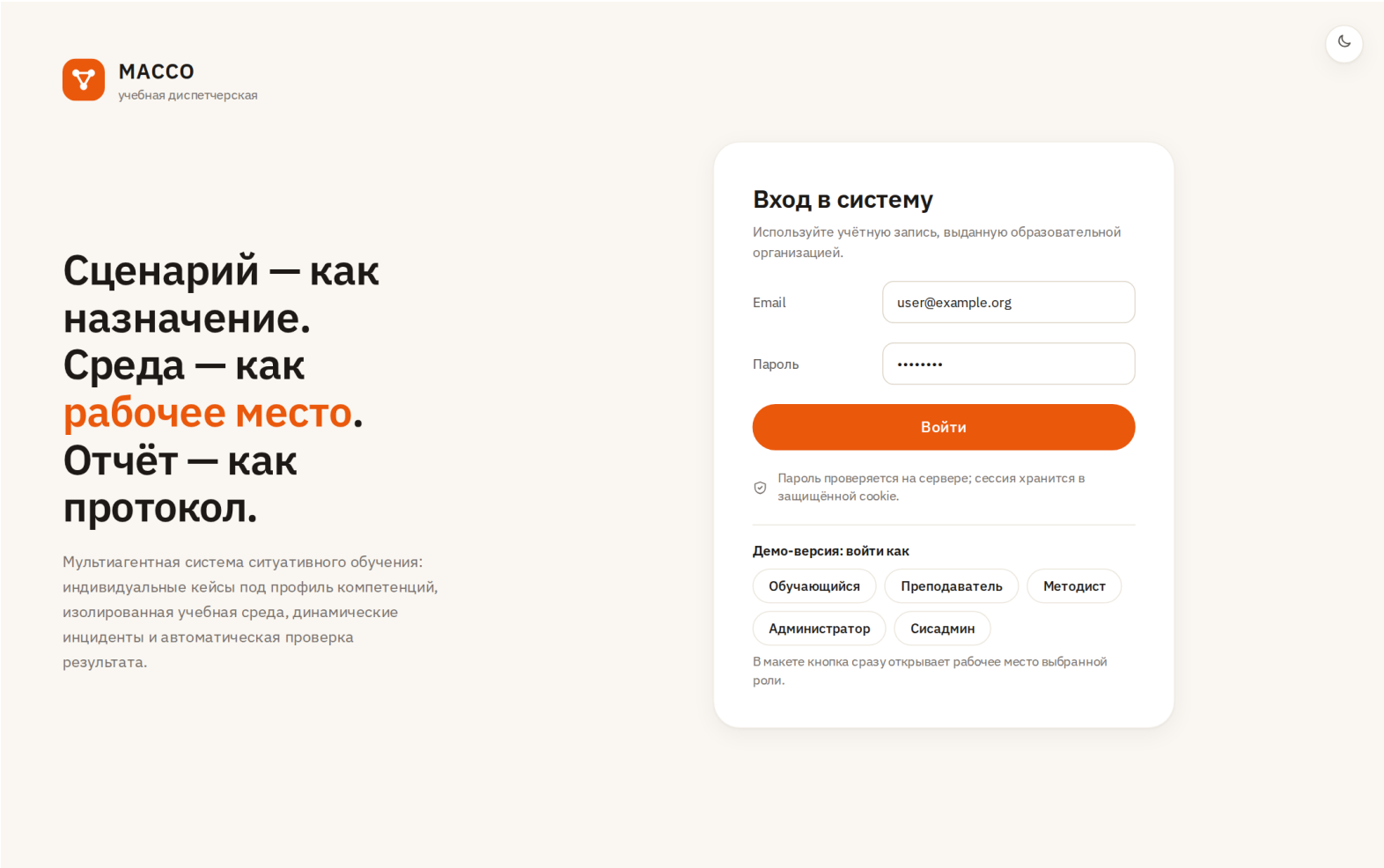


Рис. 36. Макет интерфейса входа в систему

Панель обучающегося показана на Рис. 37. В ней отображаются назначенные сценарии со сроками и статусами, рекомендации ProfileAgent с сообщением о ходе генерации и последние протоколы.

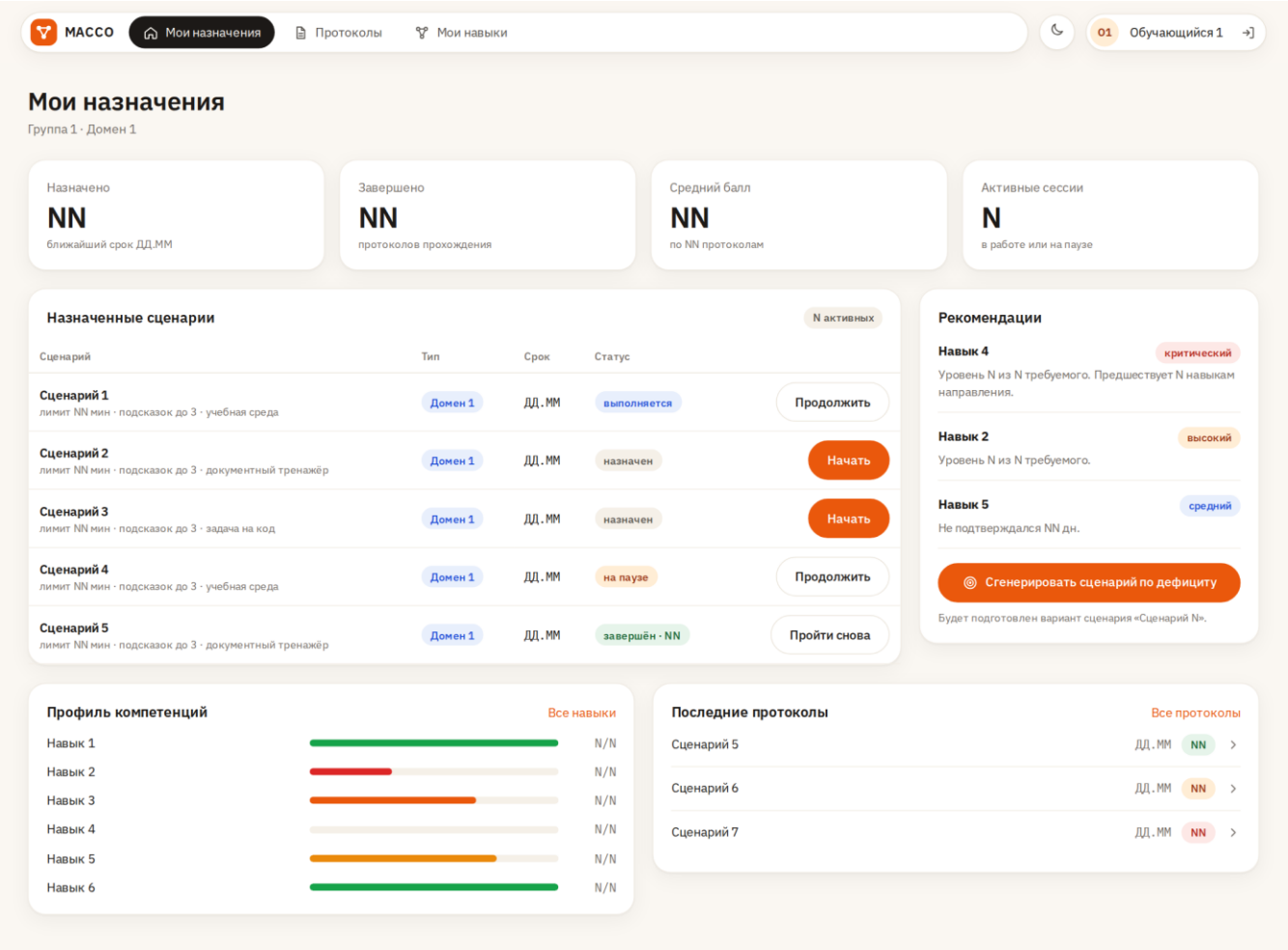


Рис. 37. Макет панели обучающегося

Рабочая область сценария с учебным сервером показана на Рис. 38. Она объединяет легенду, критерии оценивания с результатами самопроверки и теорию по навыкам сценария, веб-терминал учебного сервера, журнал инцидентов и подсказок, таймер с кнопкой «Пауза/Продолжить» и действие «Завершить и отправить на проверку».

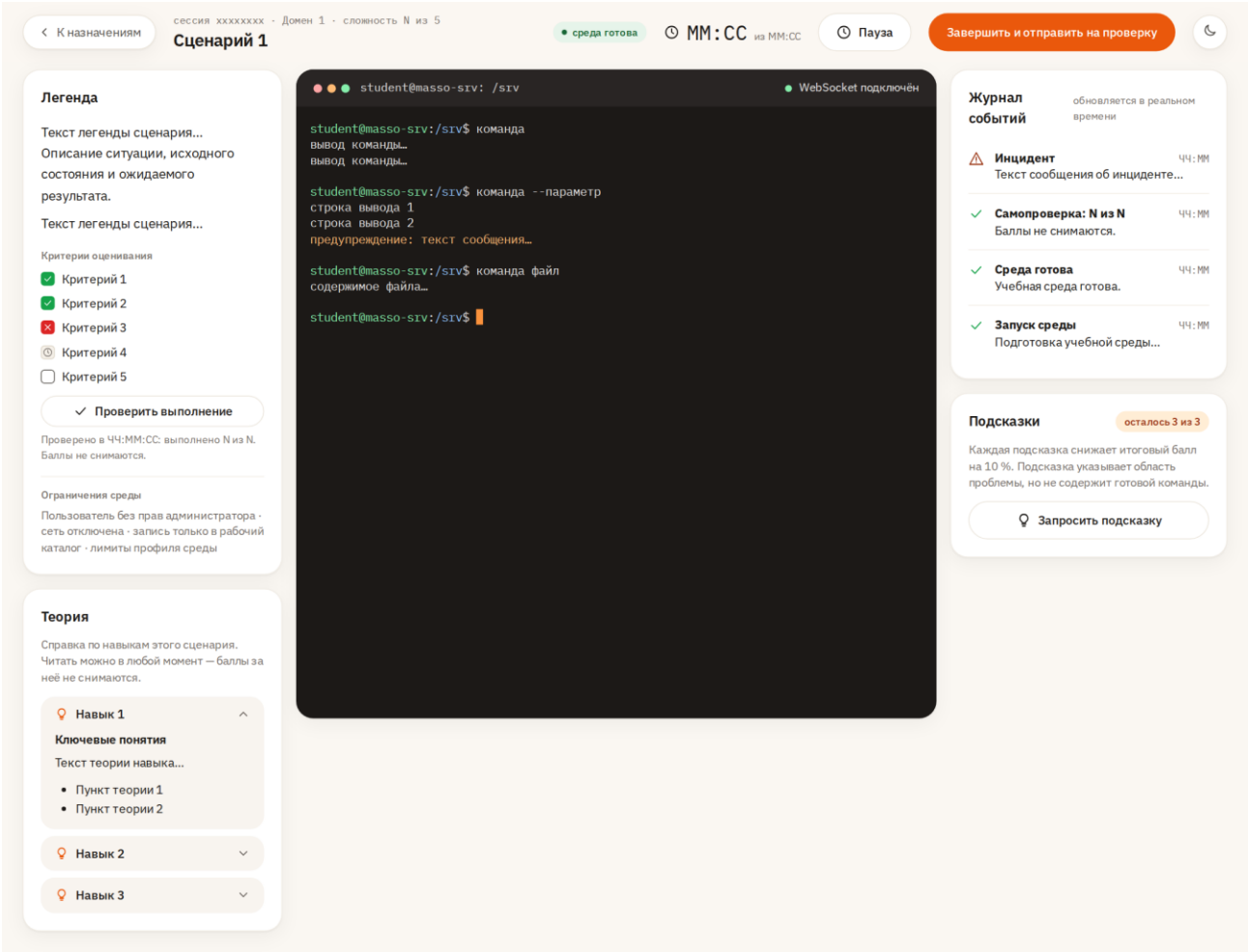


Рис. 38. Макет рабочей области: сценарий с учебным сервером

Рабочая область документного сценария показана на Рис. 39. Слева - пакет документов, справа - форма замечаний с выбором типа дефекта; ответ сохраняется автоматически через 1,5 с после изменения, о чем сообщает отметка «Сохранено».

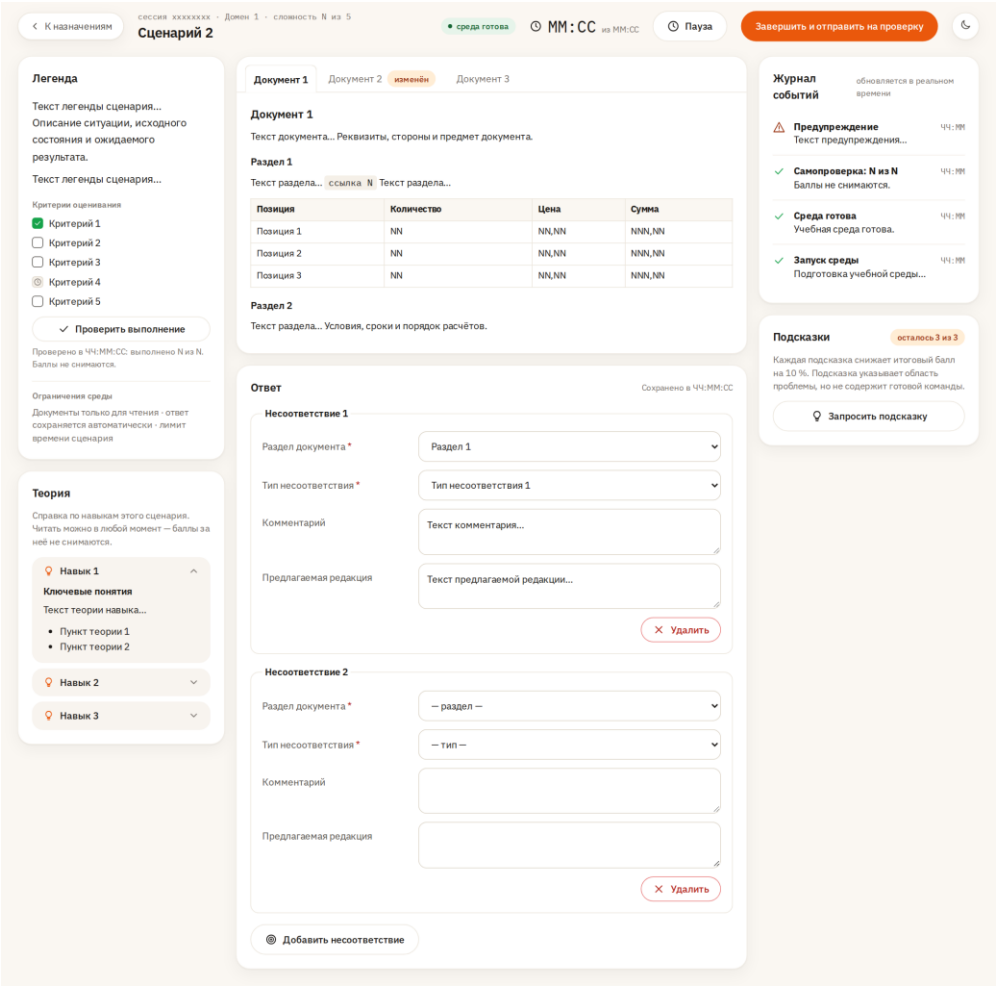


Рис. 39. Макет рабочей области: документный сценарий

Рабочая область задачи на код показана на Рис. 40. Она содержит условие с публичными примерами, редактор с файлами solution.py и test_solution.py и кнопку «Запустить тесты» с результатами по примерам и тестам обучающегося; критерии, самопроверка, подсказки и теория - как в других сценариях.

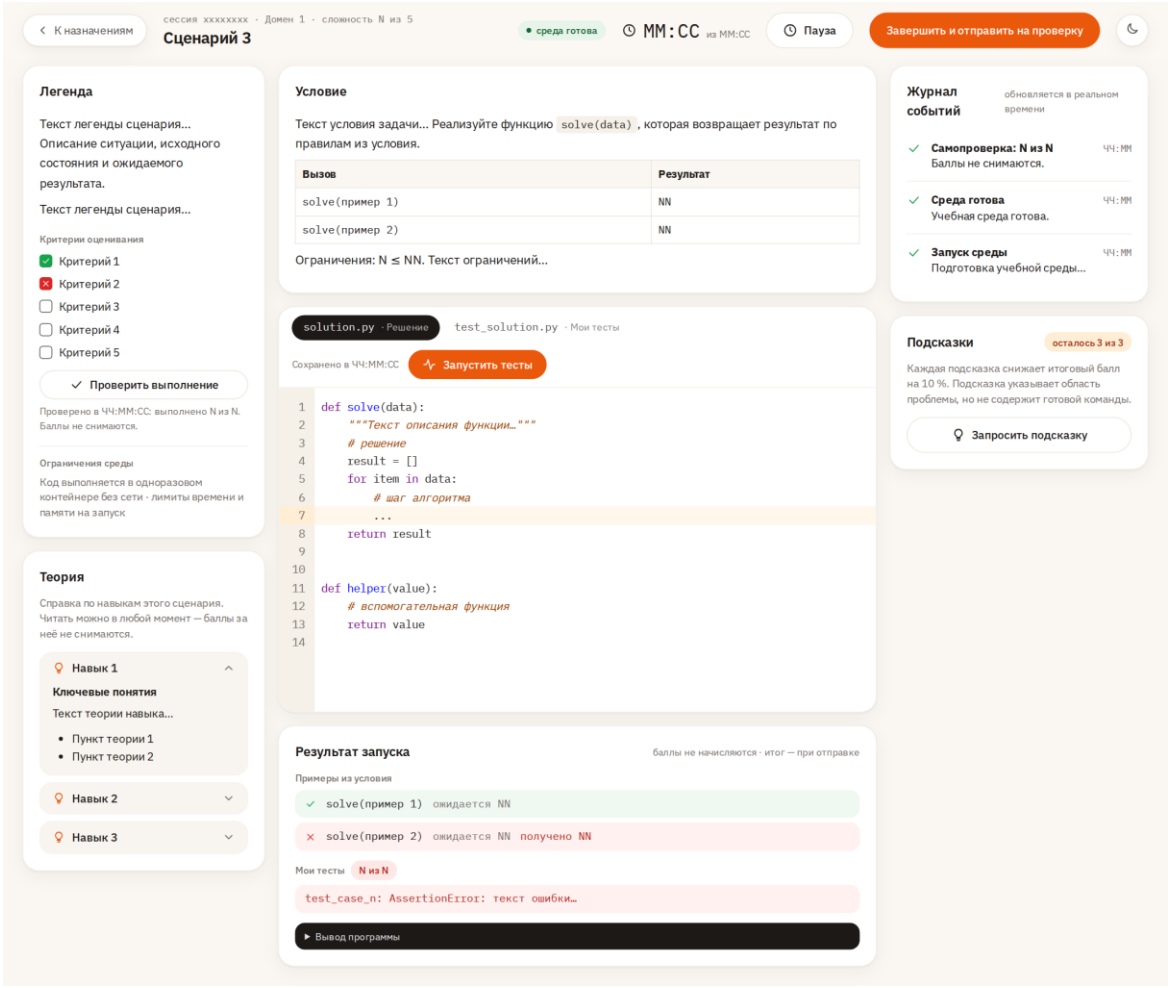


Рис. 40. Макет рабочей области: задача на код

Протокол прохождения показан на Рис. 41. На экране выводятся балл с расчетом, результаты проверок с весами, ошибки, изменения профиля, рекомендация и блок «Повторить теорию» по навыкам с непройденными проверками; печатная форма сохраняется в PDF средствами браузера.

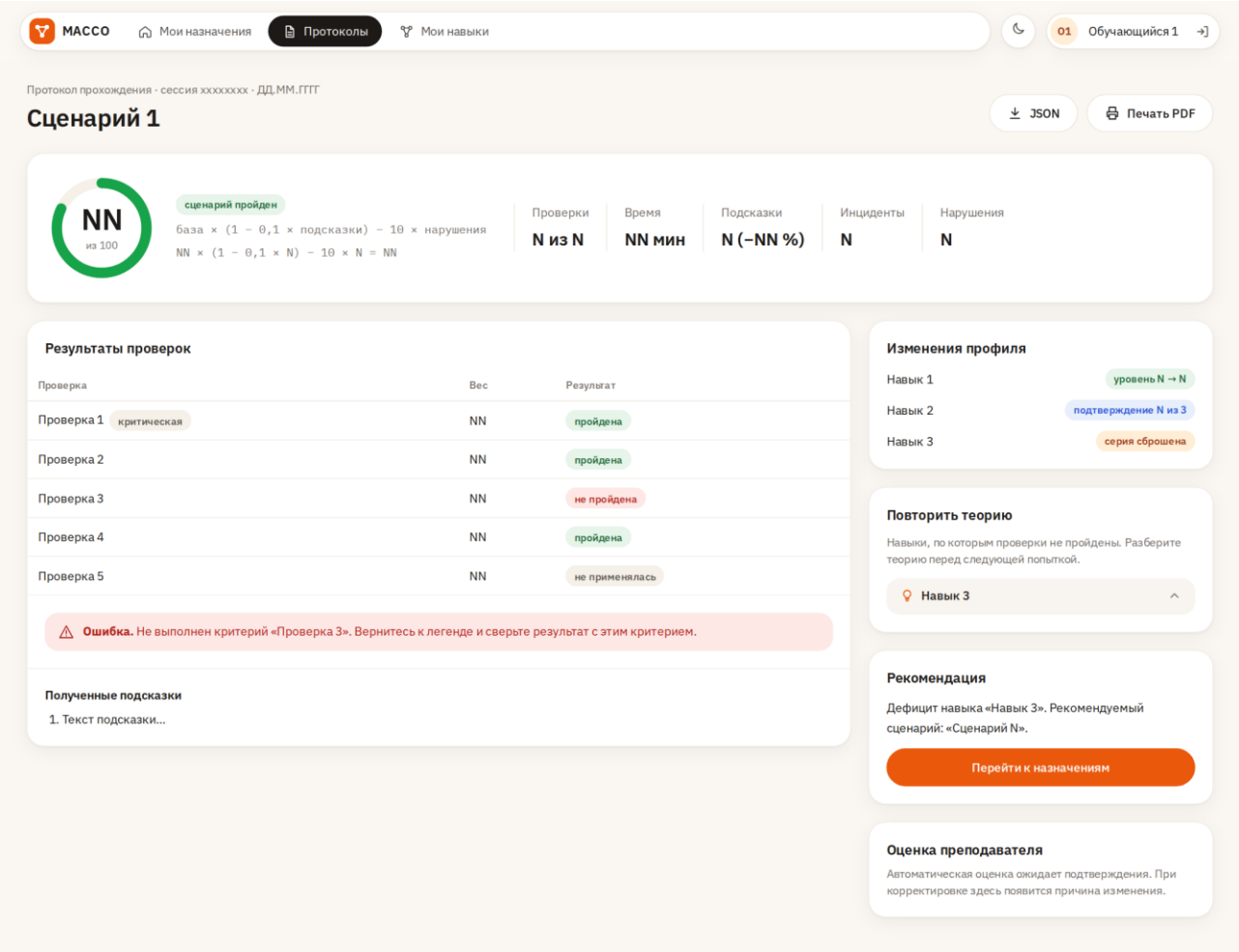


Рис. 41. Макет протокола прохождения

Экран «Мои навыки» показан на Рис. 42. Он показывает карту компетенций направления: граф навыков с prerequisite-связями, где каждый навык отмечен состоянием (освоен, изучается, доступен для изучения, закрыт до

освоения предыдущих), выделены дефициты и указан процент освоения направления; по выбору навыка выводятся его описание, текущий и требуемый уровни, серия успешных прохождений, связанные навыки и теория.

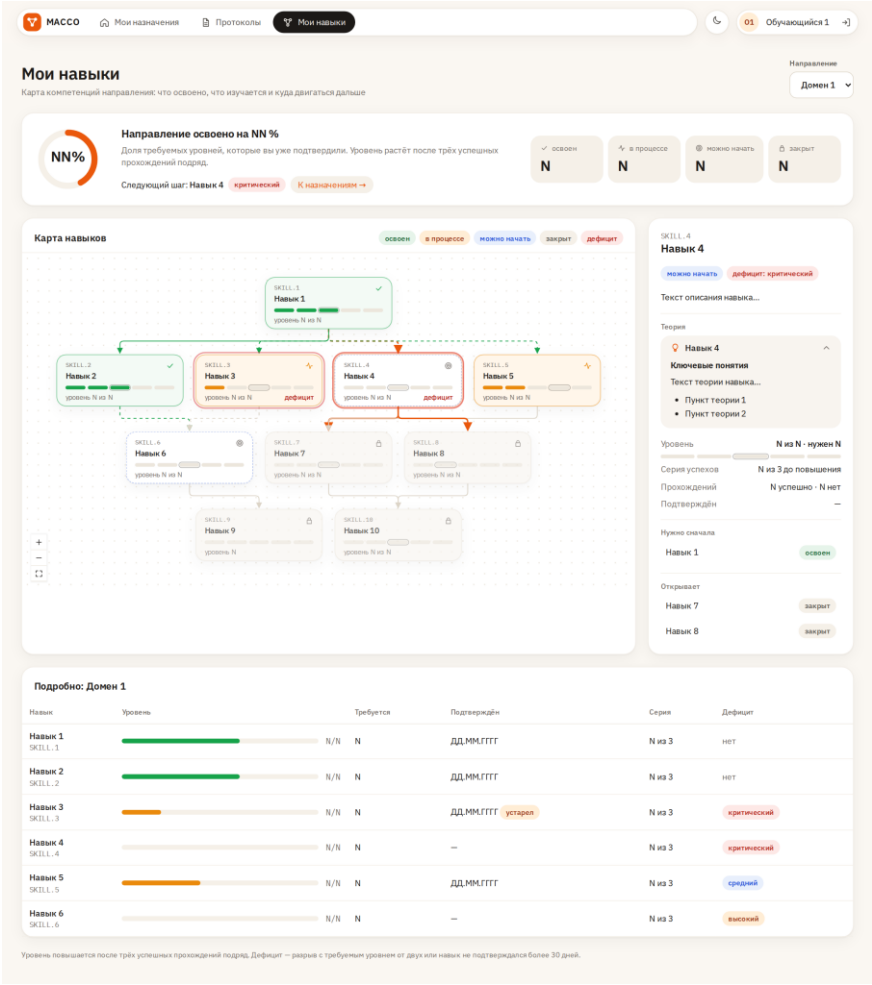


Рис. 42. Макет карты компетенций обучающегося

Панель группы показана на Рис. 43. Обязательными являются фильтры группы и периода; экран содержит агрегаты, список сессий со статусом оценки (строка открывается целиком, ссылка «Открыть»), проблемные навыки и баннер режима LLM.

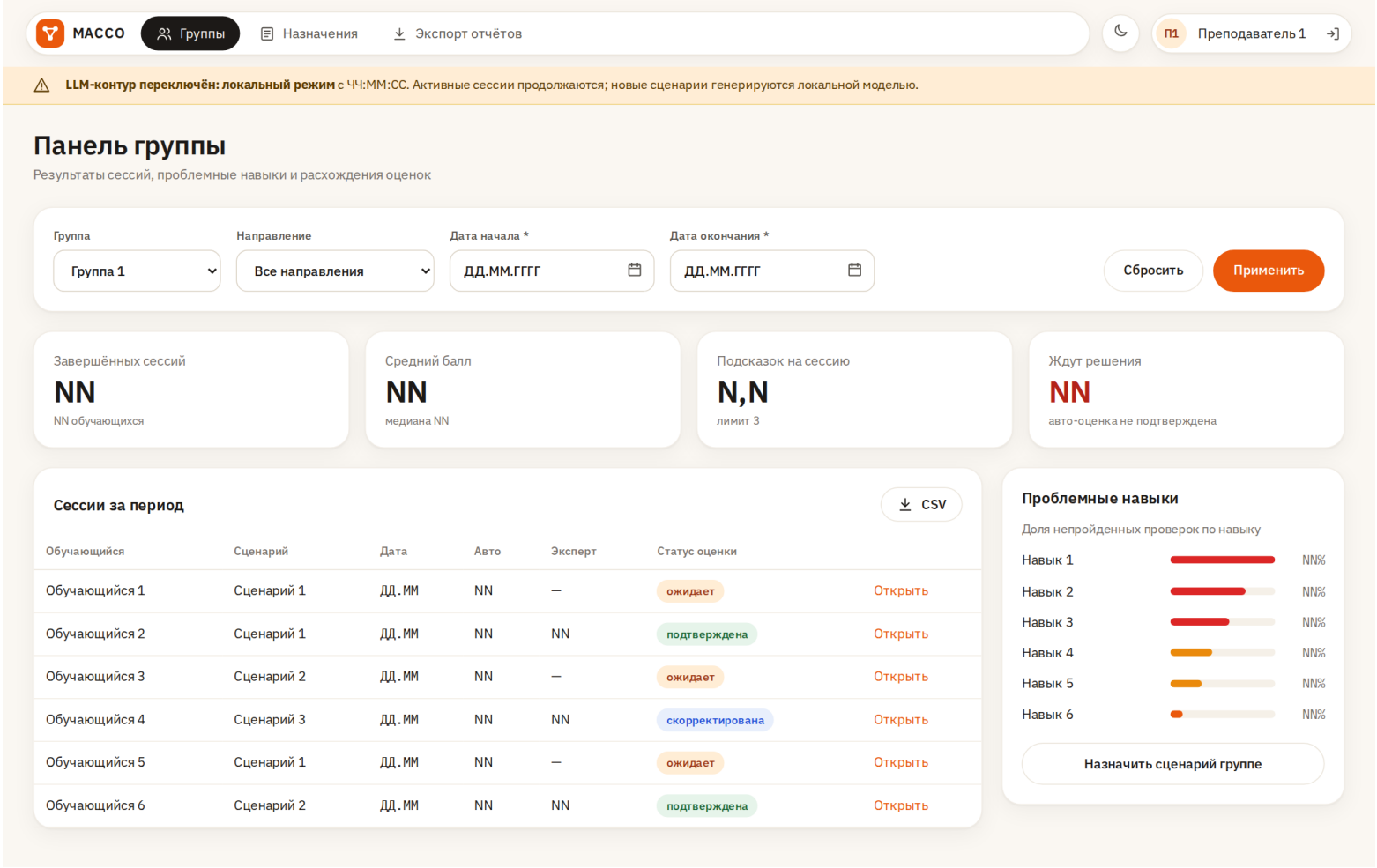


Рис. 43. Макет панели группы

Экран цифрового следа показан на Рис. 44. Он позволяет фильтровать события по времени, типу и severity, анализировать проверки, отмечать альтернативное решение и корректировать оценку с обязательной причиной.

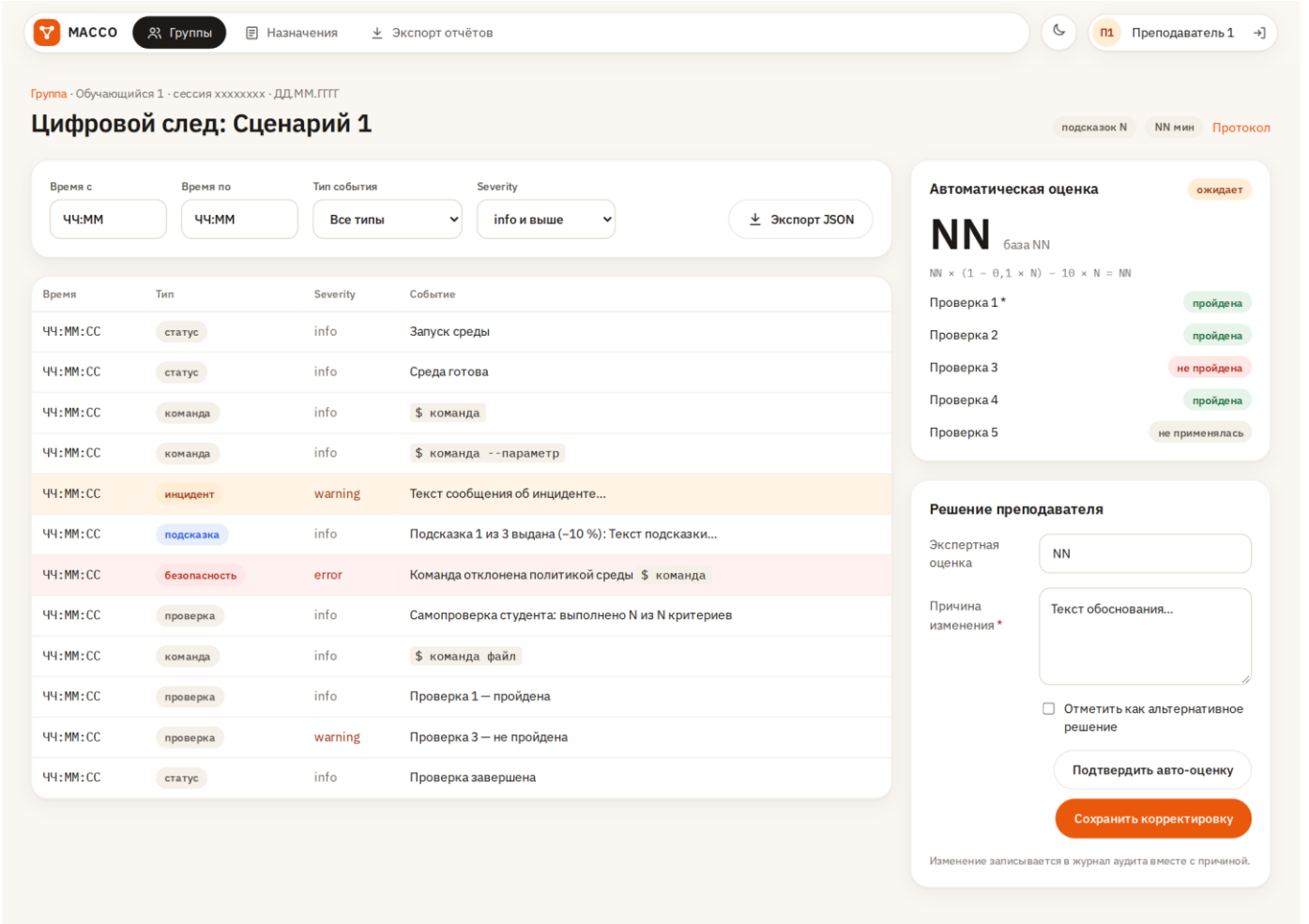


Рис. 44. Макет просмотра цифрового следа преподавателем

Экран графа компетенций показан на Рис. 45. Он содержит выбор предметной области с созданием и редактированием доменов, граф навыков с весами связей, свойства выбранного навыка с полем «Теория (Markdown)» и предпросмотром и импорт CSV/JSON; контроль ацикличности отображается постоянно.

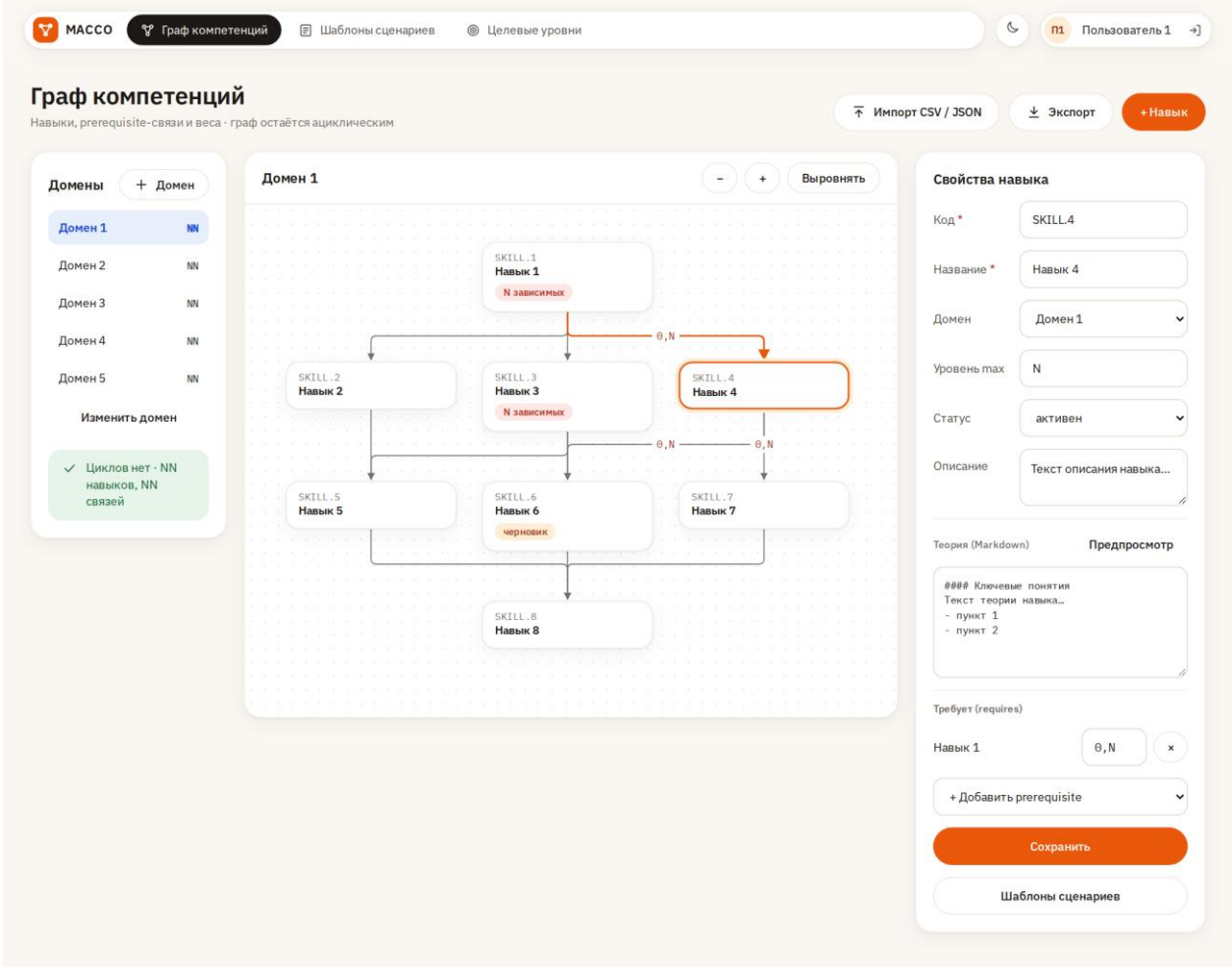


Рис. 45. Макет управления графом компетенций

Экран шаблона сценария показан на Рис. 46. Методист задает параметры, каталог сбоев с эталонными исправлениями, пул инцидентов и checks, запускает валидацию прогоном и публикует шаблон после успешного отчета валидации.

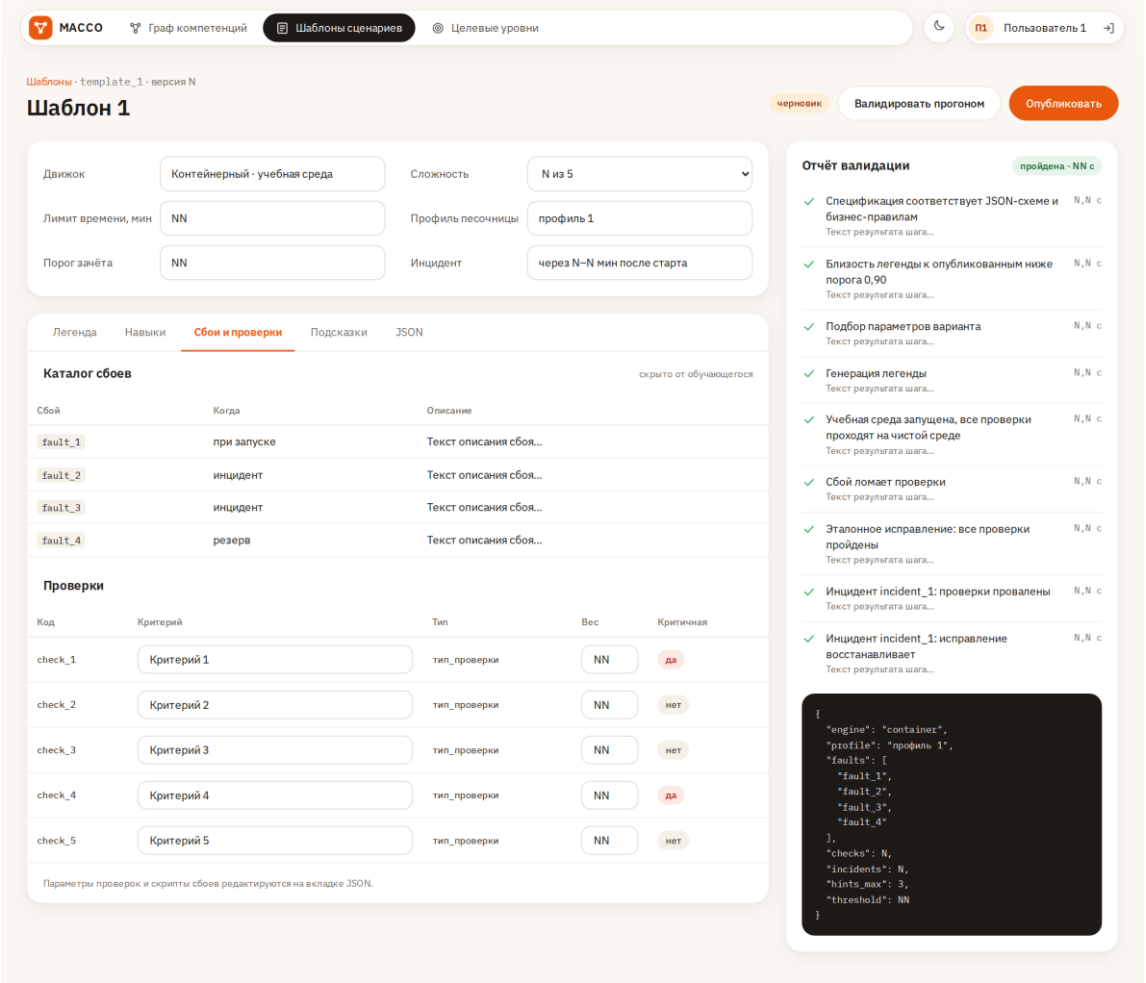


Рис. 46. Макет редактирования шаблона сценария

Экран целевых уровней направления показан на Рис. 47. Методист задает для навыков направления требуемый уровень, по которому ProfileAgent рассчитывает дефициты.

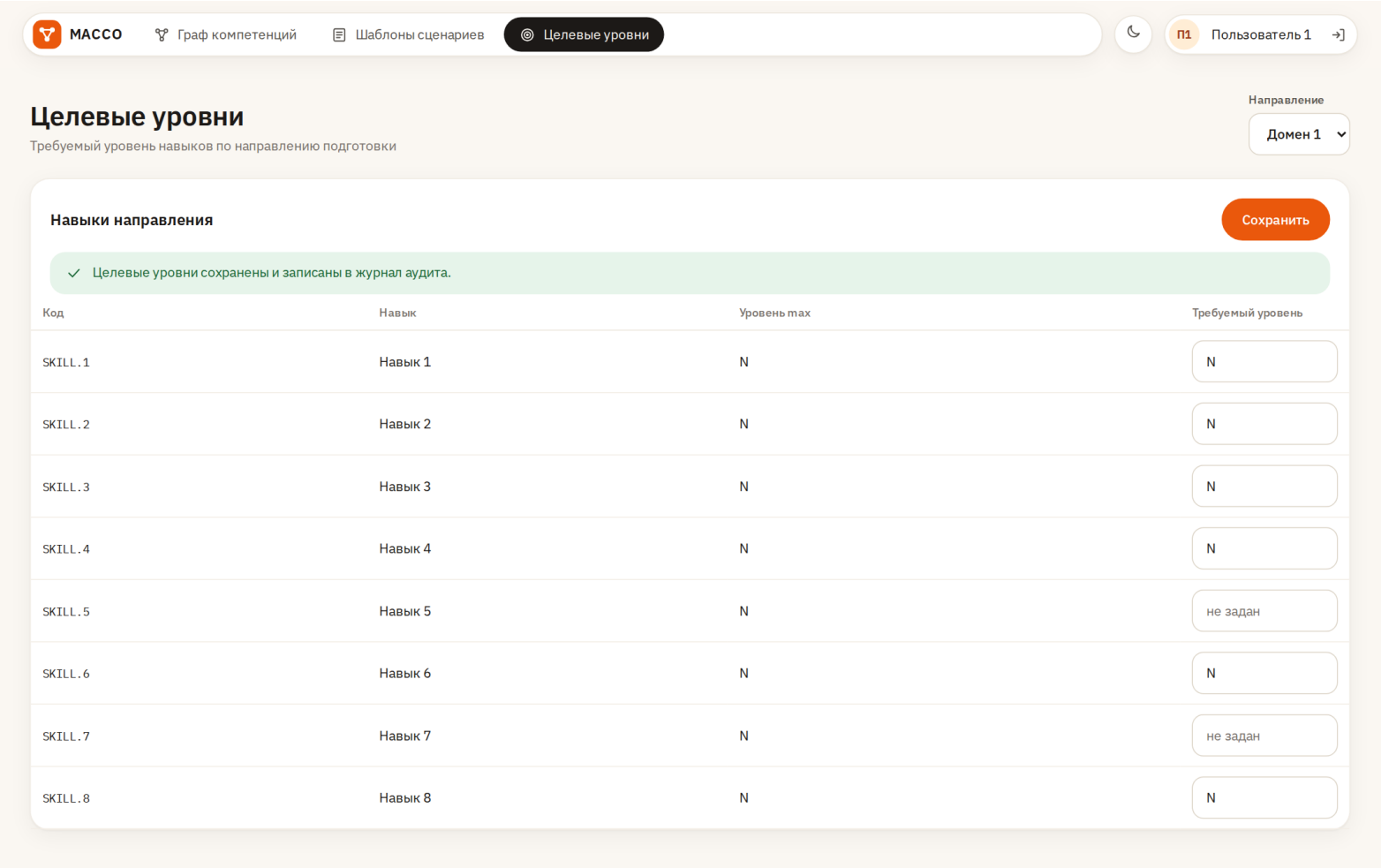


Рис. 47. Макет целевых уровней направления

Экран пользователей и ролей показан на Рис. 48. Он поддерживает поиск, создание учетной записи, изменение ролей и статуса, отзыв сессий входа с подтверждением и показывает историю изменений.

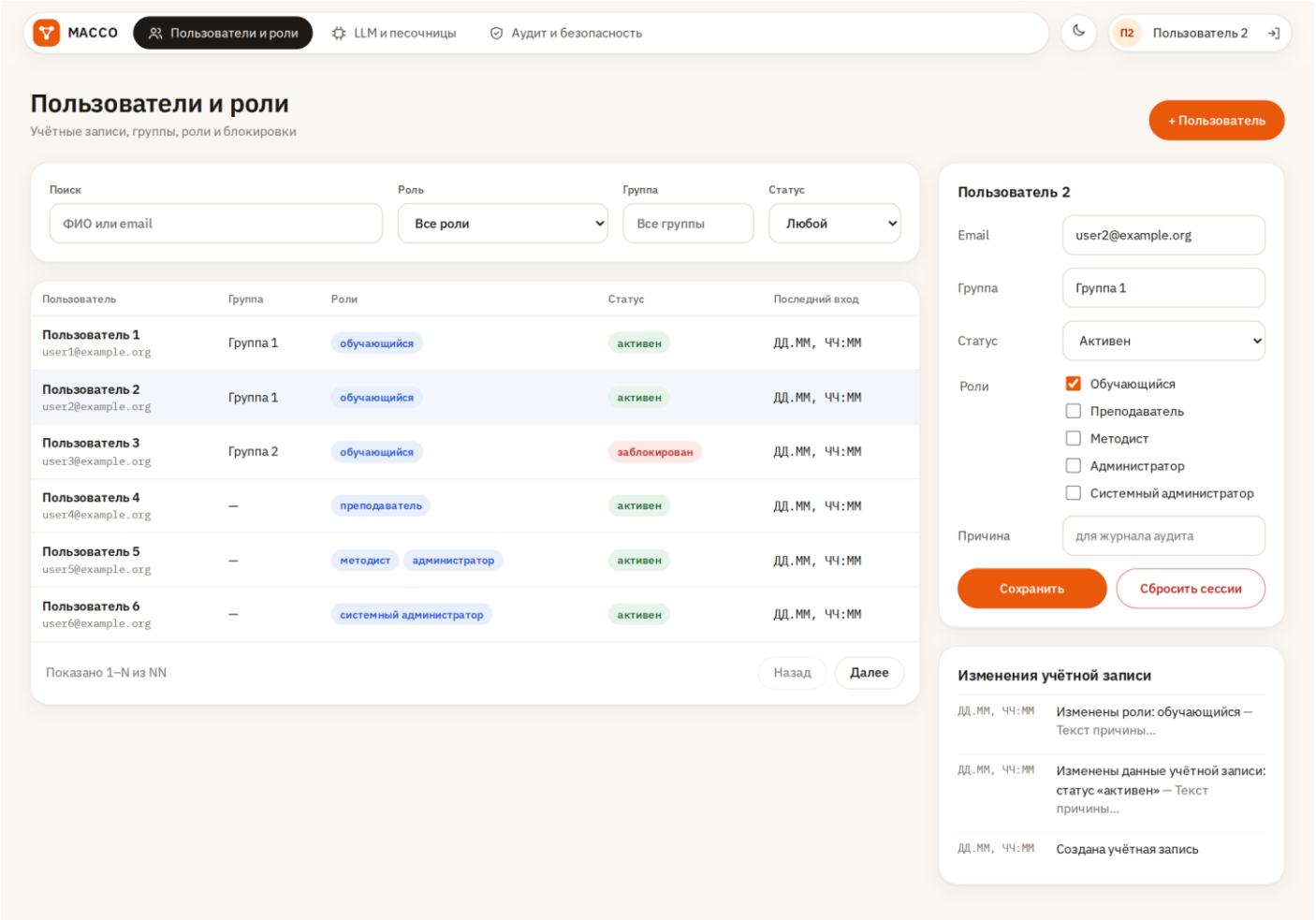


Рис. 48. Макет управления пользователями и ролями

Экран LLM и песочниц показан на Рис. 49. Администратор видит состояние провайдеров, переключает режим с обязательной причиной, просматривает профили песочниц и фоновые задачи.

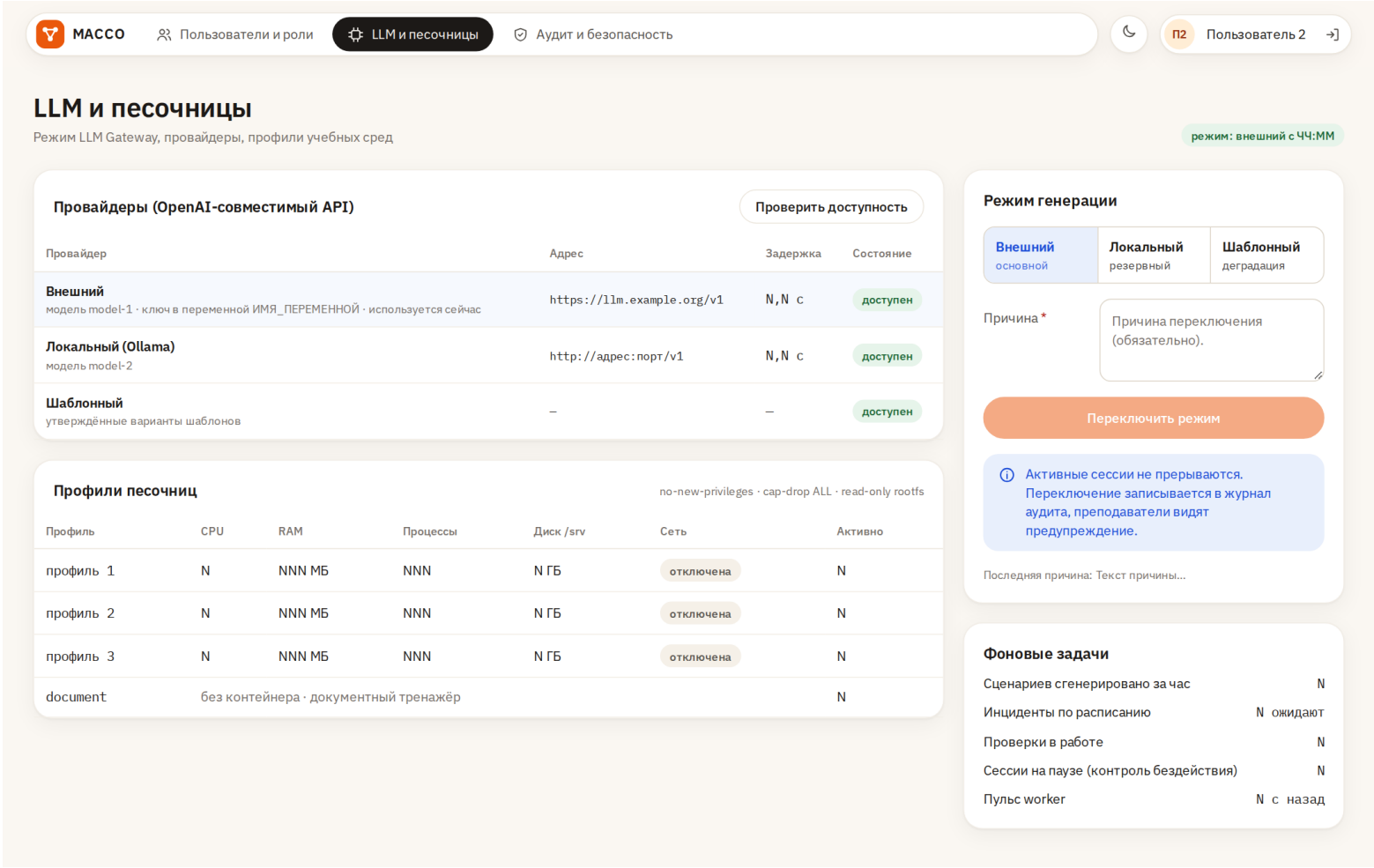


Рис. 49. Макет администрирования LLM и песочниц

Экран состояния системы показан на Рис. 50. Он содержит состояние компонентов, ресурсы стенда, список учебных сред с ручной очисткой, ссылку на метрики (/metrics) и журнал событий безопасности с периодом и экспортом; полный журнал аудита и безопасности - на отдельном экране «Аудит и безопасность». Резервное копирование выполняется по регламенту (pg_dump по расписанию).

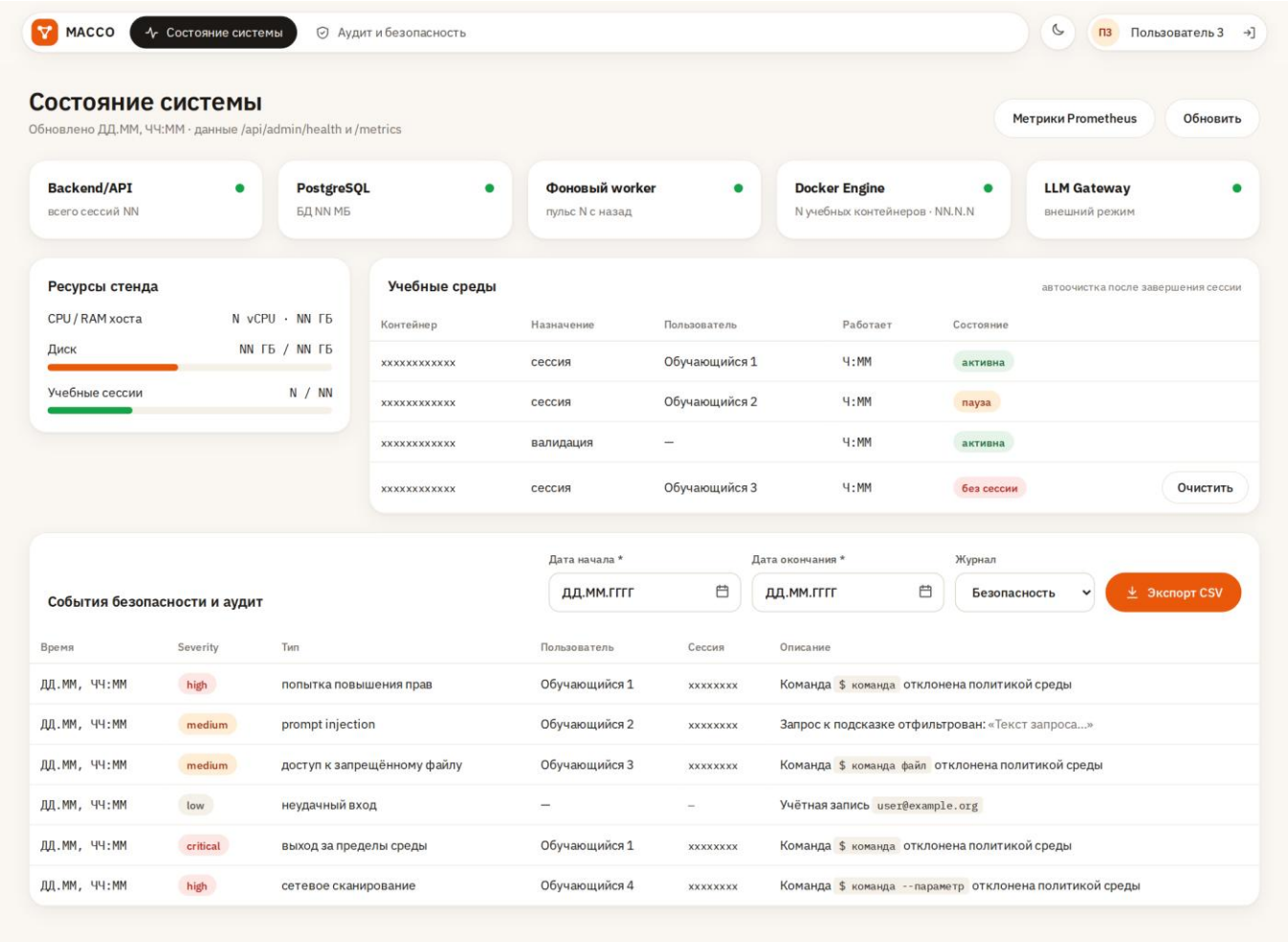


Рис. 50. Макет состояния системы

Рабочая область в темной теме показана на Рис. 51. Темная тема меняет только цветовые токены; состав и расположение элементов совпадают со светлой.

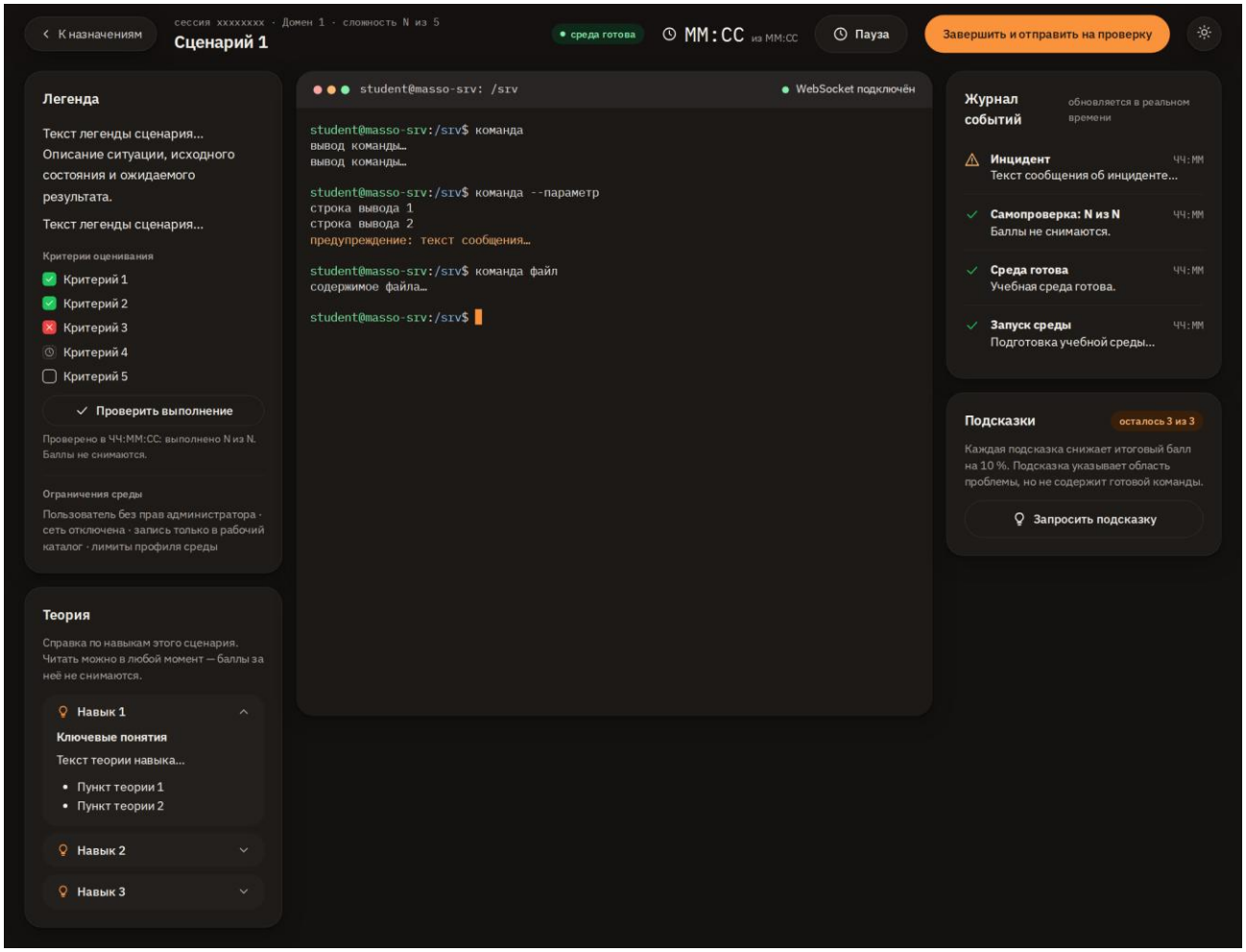


Рис. 51. Макет рабочей области в темной теме

12. Контрольные проектные сценарии

Сценарий 1. Обучающийся проходит DevOps-сценарий.

Обучающийся входит в систему, выбирает назначение восстановления веб-сервиса и запускает среду. В терминале учебного сервера он проверяет состояние сервисов и журналы, исправляет параметры подключения к базе данных, после инцидента из пула шаблона (например, неверный upstream в конфигурации Nginx) восстанавливает сервис, при необходимости запускает самопроверку и отправляет результат. Система фиксирует команды, инцидент, подсказки и события безопасности, выполняет checks, формирует отчет и обновляет профиль компетенций.

Сценарий 2. Обучающийся проходит юридический документный кейс.

Обучающийся открывает пакет документов, находит противоречия, учитывает новую нормативную вводную и заполняет форму ответа, которая сохраняется автоматически. Пакет документов и дефекты сформированы программно по параметрам шаблона, легенду пишет языковая модель (или шаблон в режиме деградации). AssessmentAgent выполняет правила проверки по ключу варианта и семантическое сравнение предложенных формулировок с эталоном.

Сценарий 3. Преподаватель анализирует результаты группы.

Преподаватель выбирает группу и период, просматривает средний балл, частоту подсказок, проблемные навыки и сессии с расхождениями. При корректировке результата причина сохраняется в журнале аудита.

Сценарий 4. Методист добавляет шаблон сценария.

Методист добавляет навык в граф, указывает prerequisite-связи, задает каталог сбоев, checks, инциденты, лимит времени и правила подсказок. Шаблон публикуется только после успешной валидации прогоном.

Сценарий 5. Администратор переключает LLM-контур.

Администратор получает предупреждение о лимите внешнего провайдера и переводит новые генерации в локальный или шаблонный режим с указанием причины. Запущенные сессии продолжают работу, переключение аудитируется.

Сценарий 6. Обучающийся решает задачу на программирование.

Обучающийся открывает задачу направления «Алгоритмы и структуры данных», читает условие и теорию по навыкам, пишет решение и собственные тесты в редакторе кода и запускает их кнопкой «Запустить тесты». Код выполняется в одноразовом изолированном контейнере. После отправки Verification Engine прогоняет решение на скрытых тестах по группам и оценивает собственные тесты обучающегося на эталоне и типичных ошибках; протокол показывает результат по группам и теорию для повторения.

13. Вывод

В техническом проекте выполнена детализация системы МАССО на основе технического задания редакции 3: определены сущности, процессы, последовательности взаимодействия, модели данных, архитектура C4, информационные потоки, справочники, API, структура интерфейса и макеты экранов. Проектные решения соответствуют реализованному опытному образцу.

Проектные решения описывают первую очередь создания системы: модульный монолит на FastAPI, единое хранилище PostgreSQL с pgvector, фоновый worker и учебные среды в контейнерах Docker без сети и привилегий; сценарии обслуживают три движка - учебный сервер, документный тренажер и задачи на код. Такая конфигурация реализует все функциональные требования ТЗ на одном сервере и сохраняет границы компонентов для перехода ко второй очереди. Диаграммы выполнены в draw.io и приложены к проекту исходными файлами и изображениями, макеты экранов выполнены в Claude Design как интерактивный холст с артбордами.

СОСТАВИЛ

Наименование организации, предприятия	Должность исполнителя	Фамилия, имя, отчество	Подпись	Дата
ИРНИТУ	Студент гр. ИСИб-23-1	Л. А. Демонов		

СОГЛАСОВАНО

Наименование организации, предприятия	Должность исполнителя	Фамилия, имя, отчество	Подпись	Дата
ИРНИТУ	Руководитель	Д. С. Шипицин		